Syrian Private University
Faculty of Artificial Intelligence
Software Engineering and Intelligent Information Systems

(BeeFast)

Delivery Web Application

This thesis was prepared to complete the requirements of the semester project in department of software engineering

Prepared by:

Fajer Rafat Abo Trabi

Supervised by:

Eng. Shadi Blaidy

Academic Year 2025/2026

الملخص

في ضوء التقدم التكنولوجي السريع، أصبحت تطبيقات التوصيل عند الطلب ضرورة ملحّة، ويهدف هذا المشروع إلى تطوير نظام توصيل متكامل وذكي يربط المتاجر والمستخدمين والسائقين، ويتصدى للتحديات التقليدية مثل عدم الكفاءة في إدارة الطلبات وصعوبة التتبع. يعتمد النظام على إطار العمل Laravelوهندسة MVC ، ويستخدم **المنهجية الشلالية (Waterfall)** في تطويره لضمان التخطيط المحكم والتنفيذ المنظم عبر مراحل متسلسلة وواضحة تبدأ بالتحليل الدقيق وتنتهي بالاختبار والتسليم. يشمل النظام واجهات مخصصة لثلاثة أدوار رئيسية: المستخدم (لوضع الطلبات وتتبعها)، والسائق (لتوصيل الطلبات وتحديث حالتها)، والمدير (لإدارة النظام والمستخدمين والمتاجر والمنتجات). تم تحليل المتطلبات الوظيفية وغير الوظيفية بدقة، وتصميم قاعدة بيانات علائقية، ورسم مخططات UML) حالات الاستخدام، الأنشطة، التسلسل) لتوضيح تفاعلات النظام قبل الانتقال لمرحلة البرمجة. كما تم توثيق بيئة التنفيذ البرمجي، إلى جانب نماذج من كود المصدر للوظائف الحرجة، وإعداد حالات اختبار شاملة للتحقق من جودة النظام قبل إطلاقه. لقد نجح المشروع في تحقيق أهدافه بأتمتة عملية الطلب والتوصيل، وتوفير واجهات سهلة الاستخدام، ودعم أدوار متعددة، مع دمج ميزات ذكية مثل نظام إشعارات فعّال. وأخيراً، يوصي المشروع بتضمين تحسينات مستقبلية مثل دمج نظام تحديد المواقع(GPS) ، وتطوير خوارزميات تحسين المسار، ونظام توصيل المنتجات باستخدام تعلّم الآلة، مما يعزز كفاءة النظام وقابليته للتوسع.

الكلمات المفتاحية: تطبيقات التوصيل الذكية المتكاملة، MVC , Laravel.

Abstract

In light of rapid technological advancements, on-demand delivery applications have become an urgent necessity, and this project aims to develop an integrated and smart delivery system that connects stores, users, and drivers while addressing traditional challenges such as inefficiency in order management and difficulty in tracking. The system relies on the Laravel framework and MVC architecture, and utilizes the **Waterfall methodology** in its development to ensure meticulous planning and structured execution through sequential, well-defined phases from initial analysis to final deployment. It includes dedicated interfaces for three main roles: the User for placing and tracking orders, the Driver for delivering orders and updating their status, and the Administrator for managing the system, users, stores, and products. Functional and non-functional requirements were thoroughly analyzed, a relational database designed, and UML diagrams (Use Case, Activity, Sequence) drawn to fully map out system interactions prior to the implementation phase. The software implementation environment has also been documented, along with source code samples for critical functions and comprehensive test cases prepared to validate system quality before release. The project has successfully achieved its objectives in automating the ordering and delivery process, providing user-friendly interfaces, and supporting multiple roles while integrating smart features such as an effective notification system. Finally, the project recommends future enhancements including GPS integration, development of route optimization algorithms, and a product recommendation system using machine learning, thereby enhancing the system's efficiency and scalability.

Keywords: Smart integrated delivery system, Laravel, MVC.

Table of Contents

الملخص II
Abstract III
Table of Contents IV
List of Figures VII
List of Tables VIII
Terms List VIII
Chapter 1: Introduction 2
1-1 Project Overview and Background 2
1-2 Problem Statement and Academic Significance 2
1-3 Project Objectives (Main and Sub-Objectives) 2
1-4 Project Scope, Constraints, and Assumptions 3
1-5 Project Contribution to Knowledge and Field 3
1-6 Research Methodology and Scientific Methods Used 4
Chapter 2: Project Development and Management Methodology 6
2-1 Chosen Development Methodology and Justifications 6
2-2 Project Timeline (Gantt Chart) 6
2-3 Risk Management (Risk Log and Mitigation Strategies) 7
Chapter 3: Literature Review and Previous Studies 9
3-1 Similar Studies 9
3-1-1 Talabat Application: 9
3-1-2 Uber Eats Application: 9
3-1-3 Jahez Application in Saudi Arabia: 9
3-1-4 Toters Application in Iraq and Lebanon: 9
3-2 Comparative Analysis of Existing Systems and Products 9
3-3 Research Gap and Proposed Innovation 10
Chapter 4: Requirements Modeling and Analysis 12
4-1 Feasibility Study 12
4-2 Stakeholder Identification & Needs 13
4-3 Requirements Elicitation Process 13
4-4 Functional Requirements (FRs) 14
FR-01: User Management and Authentication 14
•**FR-01-1** 14
FR-02: Profile Management and Delivery Location 14

- **FR-02-1**……14
FR-03: Store Browsing ……14
- **FR-03-1**……14
FR-04: Product Browsing and Shopping Cart Management ……14
- **FR-04-1**……14
FR-05: Store Search ……14
- **FR-05-1**……14
FR-06: Order Placement and Payment……14
- **FR-06-1**……14
FR-07: Order Status Tracking ……14
- **FR-07-1**……14
FR-08: Driver Order Management ……15
- **FR-08-1**……15
FR-09: User Management (for Admin)……15
- **FR-09.1 (View All Users):**…… ……15
- **FR-09.2 (View User Details):**……15
- **FR-09.3 (Delete Users):**……15
FR-10: Store Management (for Admin)……15
- **FR-10.1 (Add New Store):**……15
- **FR-10.2 (Modify Store Information):**……15
- **FR-10.3 (Delete Store):**……15
FR-11: Product Management (for Admin) ……15
- **FR-11.1 (Add New Product):**……15
- **FR-11.2 (Modify Product Information):**……15
- **FR-11.3 (Delete Product):**……15
FR-12: Order Management (for Admin)……15
- **FR-12.1 (View All Orders):**…… ……15
- **FR-12.2 (Modify Order Status):**……16
- **FR-12.3 (Assign Driver to Order):**……16
- **FR-12.4 (Cancel Order):**……16
FR-13: Notifications……16
- **FR-13-1**……16

4-5 Non-Functional Requirements (NFRs)……16
4-6 Use Case Diagram ……16
4-7 Use Case Descriptions ……18
4-8 Activity Diagram ……24

4-9 Sequence Diagram....31
4-10 Modeling Based on Adopted Methodology....38
4-11 Requirements Analysis and Documentation....38
4-12 Expanded Requirements Traceability Matrix (RTM)....39
4-13 Initial Test Cases....40
Chapter 5: System Architectural Design....42
5.1 High-Level Design....43
5.1.1 Architectural Block Diagram....43
5.1.2 Used Architectural Design Pattern:....44
5.1.3 System Architecture Design....45
5.2 Database Design....45
5.2.1 Entity-Relationship Diagram (ERD)....46
5.2.2 Relational Schema....48
5.2.3 Constraints and Keys....51
5.3 Security and Permissions Design....52
5.3.1 Authentication....52
5.3.2 Authorization (Role-Based Access Control - RBAC)....52
5.3.3 Data Protection....53
Chapter 6: Execution Environment and Tools....55
6-1 Tools Used....55
6.2 Development Environment and Project Structure....58
6.3 Programming Languages and Frameworks....58
6.4 Database Server and Operating System....58
6.5 Libraries and Dependencies....58
6.6 Version Control and Source Management Tools....59
6.8 Testing Environment....59
Chapter 7: Programming Implementation....60
7-1 Project File Structure....60
7-2 Modules Implementation....61
7-2-1 Authentication Module....61
7-2-2 Store & Product Module....62
7-2-3 Cart & Order Module....62
7-2-4 Notification Module....62
7-2-5 Favorite Module....63
7-2-6 Admin Module....63
7-3 APIs Implementation....63

7-4 Data Access Layer.....65
7-5 Business Logic Layer.....66
7-6 UI Implementation.....68
7-7 Smart Algorithms Implementation.....74
Chapter 8: Testing and Evaluation.....76
8-1 Test Plan.....76
8-2 Types of Testing.....76
8.2.1 Unit Testing.....76
8.2.2 Integration Testing.....76
8.2.3 System Testing.....76
8.2.4 Acceptance Testing.....77
8.2.5 Usability Testing.....77
8.2.6 Compatibility Testing.....77
8-3 Test Case Diagrams.....77
8-3-1 Test Cases.....77
8-3-2 Test Results Summary.....78
8-4 Test Results & Analysis.....78
8-5 Quality Metrics.....79
8-6 Usability Testing.....79
Chapter 9: Results and Analysis.....82
9-1 System Implementation Results.....82
9-2 Comparative Analysis with Existing Systems.....95
9-3 Goals Achievement.....95
9-4 Strengths and Weaknesses Analysis.....96
9-5 Key Performance Indicators (KPIs).....97
Chapter 10: Conclusion and Recommendations.....99
10-1 Project Summary and Key Achievements.....99
10-2 Challenges and Difficulties.....99
10-3 Future Development Suggestions.....100
10-4 Recommendations.....101
Appendices and References.....103

List of Figures

Figure 2.1:Gantt chart.....6
Figure 4.2 : Use Case Diagram.....17

Figure 4.3-15 Activity Diagram.....30-24
Figure 4.16-28 Sequence Diagram.....31-37
Figure 5.29 : system Block Diagram.....41
Figure 5.30 : Entity-Relationship Diagram 45
Figure 7.31 : Directory tree 60
Figure 5.30 : Entity-Relationship Diagram 45

List of Tables

Table 2.1 : Risk Management.....7
Table 3.2 : compare between BeeFast and other websites.....
Table 4.3: UC-01: Register.....18
Table 4.4 : **UC-02: Login** 19
Table 4.5 : **UC-03: Place Order** 19
Table 4.6 : **UC-04: Manage Cart** 19
Table 4.7 : **UC-05: View His Own Orders** 20
Table 4.8 : **UC-06: Manage Favorite** 20
Table 4.9 : **UC-07: View Profile** 21
Table 4.10 : **UC-08: Change Assigned Orders Status** 21
Table 4.11 : UC-09: Manage Users 22
Table 4.12 : UC-10: Manage Products 22
Table 4.13 : **UC-11: Manage Stores** 23
Table 4.14 : UC-12: View Orders 23
Table 4.15 : RTM 39
Table 4.16 : Initial Test Cases 40
Table 5.17 : **Relationships Summary** 48
Table 5.18 – 5.26 : **Relational Database Schema Tables** 49-51
Table 5.27 – 5.30 : Constraints and Keys Table 51-52
Table 7.31 : **Eloquent Relationships** 65
Table 8.32 : **Test Cases** 77
Table 8.33 : **Test Results Summary** 78
Table 9.34 : **Unit Execution Results Table** 82
Table 9.35 : **Competitive Comparison Table** 95
Table 9.36 : **Goals Achievement Table** 95

Terms List

#	Technical Term	Abbreviation	Category	Definition and Concept
1	Model-View-Controller	**MVC**	Architecture	An architectural pattern that separates data, user interface, and logic.

2	Application Programming Interface	**API**	Communication	A set of protocols for exchanging data between different systems.
3	Hypertext Transfer Protocol	**HTTP**	Network	The fundamental protocol for transferring data over the web.
4	Representational State Transfer	**REST**	Architecture	An architectural style for building APIs based on the HTTP protocol.
5	Database Management System	**DBMS**	Database	Software responsible for storing and manipulating data (e.g., MySQL).
6	Entity-Relationship Diagram	**ERD**	Database	A diagram showing the tables and the relationships between them.
7	Structured Query Language	**SQL**	Database	The standard language for dealing with relational databases.
8	Object-Relational Mapping	**ORM**	Programming	A technique for converting database data into code objects (e.g., Eloquent).
9	User Interface	**UI**	Design	The visual design and elements that the user interacts with.
10	User Experience	**UX**	Design	How easy and smooth the system is from the user's perspective.
11	Integrated Development Environment	**IDE**	Tools	Software used for writing code (e.g., VS Code).
12	Version Control System	**VCS**	Tools	A system for tracking changes in source code (e.g., Git).
13	Continuous Integration	**CI**	DevOps	Automating the process of merging and testing code periodically.
14	Continuous Deployment	**CD**	DevOps	Automating the process of deploying updates to production servers.
15	JavaScript Object Notation	**JSON**	Data Format	A lightweight format for exchanging data between server and app.
16	Cross-Site Request Forgery	**CSRF**	Security	A vulnerability protected in Laravel via security "Tokens".
17	Create, Read, Update, Delete	**CRUD**	Operations	The four basic operations for any record in a database.
18	Single Page Application	**SPA**	Web Dev	A web app that loads once and updates dynamically (e.g., React).
19	Cascading Style Sheets	**CSS**	Frontend	The language responsible for the design and look of web pages.
20	HyperText Markup Language	**HTML**	Frontend	The core language for building the structure of web pages.
21	Software Development Kit	**SDK**	Tools	A set of tools that help developers build apps for a specific platform.
22	Backend	-	Architecture	The part responsible for logic and the database (Server-side).
23	Frontend	-	Architecture	The visible part that the user interacts with (Client-side).
24	Middleware	-	Backend	Filters that check requests before they reach the controller.
25	Migration	-	Database	A system in Laravel for managing database structure via code.
26	Seeding	-	Database	The process of filling the database with initial dummy data.

27	Routing	**-**	Backend	The process of linking specific URLs to specific code functions.
28	Controller	-	MVC	The part that receives and processes user requests.
29	View	-	MVC	The template responsible for displaying data to the user (Blade).
30	Model	-	MVC	The code object that represents a table in the database.
31	Primary Key	**PK**	Database	A unique field that identifies each record in a table.
32	Foreign Key	**FK**	Database	A field that links a record in one table to a record in another.
33	Authentication	**Auth**	Security	The process of verifying a user's identity (Login).
34	Authorization	-	Security	The process of verifying permissions (What the user can do).
35	Deployment	-	DevOps	The process of uploading the system to a server for public access.
36	Responsive Design	-	Frontend	The ability of a site to adapt its layout to different screen sizes.
37	Dependency Manager	-	Tools	A tool for managing external libraries (e.g., Composer or NPM).
38	Validation	-	Backend	Ensuring that input data follows required rules (e.g., email format).
39	Encryption	-	Security	Converting data into unreadable symbols for protection.
40	Debugging	-	Programming	The process of finding and fixing programming errors.

Chapter 1

Introduction

Chapter 1: Introduction

1-1 Project Overview and Background

In light of rapid technological development and the digital transformation the world is witnessing, on-demand delivery applications have become an integral part of daily life, meeting the growing need of consumers to obtain goods and services easily and quickly. This project aims to develop an integrated and smart delivery system that connects various stores (restaurants, cafes, grocery stores), users, and drivers, providing a seamless and efficient experience for all parties. The system relies on the latest web development technologies and seeks to offer innovative solutions to traditional challenges facing delivery operations.

1-2 Problem Statement and Academic Significance

Traditional delivery systems suffer from several challenges, most notably inefficiency in order management, difficulty in tracking delivery status, and a lack of effective coordination among stores, drivers, and users. These issues lead to delivery delays, a poor user experience, and increased operational costs. This project highlights its academic significance by providing an integrated software solution that addresses these challenges through process automation, intuitive user interfaces, and the application of smart mechanisms to improve delivery efficiency. It also represents an opportunity to apply concepts learned in software engineering and the development of intelligent systems.

1-3 Project Objectives (Main and Sub-Objectives)

This project aims to achieve a set of main and sub-objectives, formulated according to SMART criteria (Specific, Measurable, Achievable, Relevant, and Time-bound):

Main Objectives:

- Develop an integrated web system for order delivery connecting stores, users, and drivers.
- Improve the efficiency of the delivery process and reduce order response time.
- Provide a seamless and intuitive user experience for all parties.

Sub-Objectives:

- Provide an admin dashboard that allows effective management of stores, products, and orders.
- Develop a user interface for drivers that enables them to receive orders and track delivery routes.

- Create a user interface for users that allows browsing stores, adding products to the cart, and placing orders.
- Implement an instant notification system to update order status for all parties.
- Provide a comprehensive admin dashboard to manage users, stores, products, and orders.

Integrate smart features such as categorizing stores based on popularity and suggesting drivers.

1-4 Project Scope, Constraints, and Assumptions

Project Scope: The project includes the development of a Web Application that supports three types of users: regular user (Customer), and driver (Driver), in addition to an admin dashboard where the Administrator manages stores and products. The system covers the entire order lifecycle from browsing products to receiving the order, focusing on specific cities and geographical areas.

Constraints:

- Limited Budget: May affect the use of some paid external services.
- Available Time: The project must be completed within the timeframe specified for the graduation project.
- Human Resources: The project relies on a limited team.
- Technologies Used: The project is limited to using the Laravel framework and Bootstrap interface.

Assumptions:

- Users and drivers have internet access to reach the application.
- Initial data for stores and products is provided for testing purposes.
- Users and drivers have a minimum basic knowledge of using web applications.

1-5 Project Contribution to Knowledge and Field

This project contributes to the field of artificial intelligence engineering and web applications by providing a delivery system based on a modern software architecture (Laravel) and including smart features to improve user experience and operational efficiency. The main contribution is the integration of mechanisms for categorizing stores based on specific criteria, the development of an effective notification system, and the potential for future expansion to integrate more complex machine learning algorithms to optimize delivery routes and demand forecasts. The project also offers a practical model for applying Domain-Driven Design principles in the context of e-commerce applications and delivery services.

1-6 Research Methodology and Scientific Methods Used

This project adopted an iterative and incremental development methodology, inspired by Agile principles, to ensure flexibility and responsiveness to changes. The project was divided into short development cycles (Sprints), where a set of features is planned, designed, implemented, and tested in each cycle. Descriptive and analytical research methods were used to gather information about requirements, analyze similar systems, and evaluate appropriate technologies. Experimental methodology was also relied upon in the development and testing phases to evaluate system performance and continuously improve it.

Chapter 2

Project Development and Management Methodology

Chapter 2: Project Development and Management Methodology

2-1 Chosen Development Methodology and Justifications

The Waterfall Model was adopted for this project due to the clear and well-defined requirements established at the outset. This sequential approach allowed for a structured development process, moving from comprehensive analysis to the detailed design of the MVC architecture, followed by implementation and testing. This clear separation between phases ensured code stability and technical organization, facilitating a seamless and well-documented integration between the Models, Views, and Controllers.

2-2 Project Timeline (Gantt Chart)

A project timeline was developed using a Gantt Chart to divide tasks and determine their durations, as well as track progress. The following table summarizes the main phases and expected tasks:

BeeFast Delivery System — Project Gantt Chart

Faculty of Artificial Intelligence Engineering · Syrian Private University · Feb – Apr 2026 · Methodology: Waterfall

Phase	Task / Activity	Start Date	End Date	Duration
Planning & Analysis	► Planning & Analysis (Overview)	01/02/2026	05/02/2026	5 days
	• Requirements Gathering	01/02/2026	02/02/2026	2 days
	• Feasibility Study	02/02/2026	03/02/2026	2 days
	• Stakeholder Analysis	03/02/2026	04/02/2026	2 days
Requirements	► Requirements (Overview)	06/02/2026	20/02/2026	15 days
	• Functional Requirements	06/02/2026	10/02/2026	5 days
	• Non-Functional Requirements	10/02/2026	14/02/2026	5 days
	• UML Diagrams	14/02/2026	18/02/2026	5 days
System Design	► System Design (Overview)	21/02/2026	01/03/2026	9 days
	• Architecture Design	21/02/2026	23/02/2026	3 days
	• Database Design (ERD)	23/02/2026	25/02/2026	3 days
	• UI Wireframes	25/02/2026	27/02/2026	3 days
Implementation	► Implementation (Overview)	02/03/2026	01/04/2026	31 days
	• Authentication Module	02/03/2026	09/03/2026	8 days
	• Store & Product Module	09/03/2026	16/03/2026	8 days
	• Cart & Order Module	16/03/2026	23/03/2026	8 days
	• Notification & Map Module	23/03/2026	30/03/2026	8 days
Testing	► Testing (Overview)	02/04/2026	05/04/2026	4 days
	• Unit & Integration Testing	02/04/2026	03/04/2026	2 days
	• Acceptance Testing	03/04/2026	04/04/2026	2 days
	• Bug Fixing	04/04/2026	05/04/2026	2 days
Documentation	► Documentation (Overview)	06/04/2026	25/04/2026	20 days
	• Thesis Writing	06/04/2026	12/04/2026	7 days
	• Code Documentation	12/04/2026	18/04/2026	7 days
	• Final Review	18/04/2026	24/04/2026	7 days

February 2026				March 2026					April 2026			
01/02	08/02	15/02	22/02	01/03	08/03	15/03	22/03	29/03	05/04	12/04	19/04	25/04

LEGEND: Planning & Analysis · Requirements · System Design · Implementation

Project Duration: 84 days | Start: 01 Feb 2026 | End: 25 Apr 2026 | Phases: 6 | Total Tasks: 16 | Methodology: Waterfall | Status: Completed ✓

Figure 2.1 : Gantt Chart

2-3 Risk Management (Risk Log and Mitigation Strategies)

Potential risks that may face the project have been identified, and strategies to mitigate their effects have been developed:

Table 2.1 : Risk Management

Potential Risk	Probability	Impact	Mitigation Strategy
Lack of Technical Expertise	Medium	High	Continuous training, expert consultation, self-learning and research
Changing Requirements	High	Medium	Adopt Agile methodology, continuous communication with user and supervisor
Database Issues	Medium	High	Robust database design, regular backups, performance testing
Integration Difficulties	Medium	Medium	Use standardized APIs, early integration testing
Schedule Delays	Medium	High	Define clear tasks, daily progress monitoring, reschedule tasks when necessary
Performance Issues	Low	Medium	Code optimization, use of caching techniques, load testing

Chapter 3
Literature Review and Previous Studies

Chapter 3: Literature Review and Previous Studies

3-1 Similar Studies

Several successful and similar platforms were utilized to benefit from their experience, including:

3-1-1 Talabat Application:

It is one of the most prominent delivery applications in the region, providing food ordering service from a wide range of restaurants. It features a simple user interface, an order tracking system, and various payment options.

3-1-2 Uber Eats Application:

A global platform that connects users with a large selection of restaurants and provides an efficient delivery system. It is known for its advanced algorithms for assigning orders to drivers as quickly as possible.

3-1-3 Jahez Application in Saudi Arabia:

A successful local platform that offers food and store product delivery service, with a focus on supporting local stores and providing a customized user experience.

3-1-4 Toters Application in Iraq and Lebanon:

It is not limited to food delivery only, but also extends to include grocery, medicine, and other daily needs delivery, making it a comprehensive platform.

3-2 Comparative Analysis of Existing Systems and Products

The following table provides a comparison between the proposed system and some leading systems in the market, highlighting the points of strength and distinction:

Table 3.2 : compare between BeeFast and other websites

Feature / System	BeeFast	Talabat	Uber Eats	Jahez
Centralized Store & Product Management	Yes (via Admin)	No	No	No
Support for Local Stores	Yes	Yes	Yes	Yes
Live Order Tracking	Yes	Yes	Yes	Yes

Smart Notification System	Yes	Yes	Yes	Yes
Multi-Role User Interface	Yes (Customer, Driver, Admin)	Yes (Customer, Restaurant, Driver)	Yes (Customer, Restaurant, Driver)	Yes (Customer, Restaurant, Driver)
Driver Assignment Algorithms	Yes (Simplified)	Advanced	Advanced	Advanced
Product Recommendations	Yes (Simplified)	Advanced	Advanced	Advanced
Multi-language Support	No	Yes	Yes	Yes
Various Payment Options	Yes (Cash)	Yes (Cash, Electronic)	Yes (Cash, Electronic)	Yes (Cash, Electronic)

From the table above, it is clear that the proposed system is distinguished by providing an integrated solution for managing stores and products centrally through the administrator, with a focus on supporting local stores and providing an effective user experience. While other systems excel in some advanced aspects such as driver assignment algorithms, product recommendations, and multi-language support, the proposed system provides a strong and scalable foundation to meet these future needs.

3-3 Research Gap and Proposed Innovation

Despite the availability of many delivery applications, there is a clear gap in providing integrated and easy-to-deploy software systems for local communities or small and medium-sized stores that do not have the resources to integrate into major global platforms. The research gap lies in how to build a system that combines the features of large platforms (such as instant notifications and multi-role management) with the simplicity of management and customization of the experience for the local user. This project innovates by providing an "All-in-one" solution that includes customized interfaces for three parties (customer, driver, and administrator) on a single platform, with a focus on automating the Order Workflow and ensuring transparency at every stage of delivery. This makes it a scalable model for integrating artificial intelligence algorithms to rank stores and distribute orders automatically in the future.

Chapter 4
Requirements Modeling and Analysis

Chapter 4: Requirements Modeling and Analysis

4-1 Feasibility Study

The feasibility study is a fundamental step to ensure the project's viability and sustainability. I conducted a rigorous analysis of the project from both technical and financial perspectives as follows:

- **Technical Feasibility:**
  - **Advanced Technologies:** I selected the **Laravel** framework for its robust infrastructure and high security, coupled with **MySQL** for efficient database management, ensuring system stability and future scalability.
  - **System Compatibility:** The system is designed as a responsive web platform using Bootstrap, ensuring seamless access across various devices (PCs, smartphones, tablets) without the immediate need for separate native applications.
  - **Maintainability and Development:** By adopting the **MVC** design pattern, I can manage updates and develop new features independently and systematically, minimizing technical downtime risks.
  - **Tool Availability:** All development tools and environments used (e.g., XAMPP, VS Code) are open-source and readily available, removing any technical barriers related to licensing or accessibility.
- **Financial Feasibility:**
  - **Low Initial Costs:** I built the project entirely using open-source software and free technologies, bringing the software development cost close to zero, excluding the human effort and time invested.
  - **Low Operational Costs:** The system does not require expensive server resources initially; it can be hosted on economical cloud servers, with the ability to upgrade resources gradually as the number of users and orders increases.
  - **Return on Investment (ROI):** The system reduces reliance on costly manual and paper-based processes. It provides a business model that generates revenue through fixed delivery fees, ensuring the recovery of operational costs and achieving a financial surplus in the medium term.
  - **Administrative Efficiency:** Through the central admin dashboard, the need for a large staff to manage stores and orders is reduced, significantly lowering administrative operational costs.

4-2 Stakeholder Identification & Needs

Identifying stakeholders and understanding their needs is crucial for the system's success. I analyzed the groups interacting with the system and defined their requirements as follows:

- **Stakeholder Identification:**
  1.1 **Administrator:** The central entity with full authority to manage the system, stores, products, and orders.
  1.2 **Customers:** The end-users who interact with the frontend to discover stores and order products.
  1.3 **Drivers:** Those responsible for the logistics and field delivery, updating order statuses in the system.
- **Stakeholders' Needs:**
  - **Admin Needs:** Centralized control, user management, order oversight, and access to accurate reports and statistics for performance evaluation.
  - **Customers' Needs:** Easy navigation, accurate information regarding product prices and delivery fees, real-time order tracking, and simple profile/address management.
  - **Drivers' Needs:** A dedicated task management interface, a simple mechanism to update delivery status (On the Way, Delivered), and access to precise customer location and order details.

4-3 Requirements Elicitation Process

Requirements elicitation is the foundation of any successful software system. I followed a systematic methodology to extract actual needs and transform them into precise technical requirements using several techniques:

- **Similar Systems Analysis:** I studied leading delivery platforms (e.g., Talabat, Jahez, Uber Eats) to understand standard workflows. This helped identify core features such as browsing by category, easy cart management, and real-time tracking.
- **Brainstorming & Gap Analysis:** Through intensive brainstorming sessions, I identified the "Research Gap" my project addresses: providing a simple, effective central management system that empowers the Admin to fully control local stores and delivery fees.
- **Field Observation:** I monitored traditional order lifecycles in local stores, from phone reception to manual delivery. This identified "Pain Points" like status update delays and address data loss, leading to the design of an instant notification system.

- **Prototyping:** I created initial wireframes for user interfaces and the admin dashboard. This helped visualize stakeholder interactions before actual coding, improving the User Experience (UX) and simplifying programming paths.

Based on the results of the feasibility study, stakeholder needs analysis, and the elicitation process mentioned above, the following list of functional requirements has been derived:

4-4 Functional Requirements (FRs)

After analyzing the project idea and studying similar systems, the following functional requirements that the system must achieve were identified:

FR-01: User Management and Authentication

- **FR-01-1**

  Ability to register and log in to the system with three roles (Customer, Driver, Admin).

FR-02: Profile Management and Delivery Location

- **FR-02-1**

  Each user has a personal account containing their profile and delivery location.

FR-03: Store Browsing

- **FR-03-1**

  Browse stores based on their type (restaurant, cafe, supermarket, etc.).

FR-04: Product Browsing and Shopping Cart Management

- **FR-04-1**

  Browse products of each store and add them to the shopping cart.

FR-05: Store Search

- **FR-05-1**

  Search for stores by name.

FR-06: Order Placement and Payment

- **FR-06-1**

  Place a new order and complete the payment process.

FR-07: Order Status Tracking

- **FR-07-1**

Track order status (pending, preparing, on the way, delivered, cancelled).

FR-08: Driver Order Management

- **FR-08-1**
  Ability for the driver to view assigned orders and update their status to "on the way" or "delivered."

FR-09: User Management (for Admin)

- **FR-09.1 (View All Users):**
  The admin must be able to view a list of all registered users in the system.
- **FR-09.2 (View User Details):**
  The admin must be able to view detailed information for each user.
- **FR-09.3 (Delete Users):**
  The admin must be able to delete user accounts from the system.

FR-10: Store Management (for Admin)

- **FR-10.1 (Add New Store):**
  The admin must be able to add new stores to the system, specifying their name, description, type, and fixed delivery fees.
- **FR-10.2 (Modify Store Information):**
  The admin must be able to modify existing store information (e.g., name, description, type, delivery fees).
- **FR-10.3 (Delete Store):**
  The admin must be able to delete stores from the system.

FR-11: Product Management (for Admin)

- **FR-11.1 (Add New Product):**
  The admin must be able to add new products, specifying their name, description, price, quantity, and linking them to a specific store.
- **FR-11.2 (Modify Product Information):**
  The admin must be able to modify existing product information (e.g., name, description, price, quantity, associated store).
- **FR-11.3 (Delete Product):**
  The admin must be able to delete products from the system.

FR-12: Order Management (for Admin)

- **FR-12.1 (View All Orders):**
  The admin must be able to view a list of all orders in the system.

- **FR-12.2 (Modify Order Status):**
  The admin must be able to modify the status of an order (e.g., from "pending" to "preparing").
- **FR-12.3 (Assign Driver to Order):**
  The admin must be able to assign a specific driver to deliver a particular order.
- **FR-12.4 (Cancel Order):**
  The admin must be able to cancel orders.

FR-13: Notifications

- **FR-13-1**
  Display order notifications for both the user and the admin.

4-5 Non-Functional Requirements (NFRs)

In addition to functional requirements, a set of non-functional requirements were identified to ensure system quality:

- NFR-01 (Performance): The system must respond to user requests within 3 seconds.
- NFR-02 (Security): All sensitive data (e.g., passwords) must be encrypted and communication secured (HTTPS).
- NFR-03 (Availability): The system must be available 99% of the time.
- NFR-04 (Usability): User interfaces must be intuitive and easy to use for all user categories.
- NFR-05 (Scalability): The system must be scalable to support an increasing number of users and orders.

4-6 Use Case Diagram

The following diagram illustrates the main use cases in the system and the actors involved in each case:

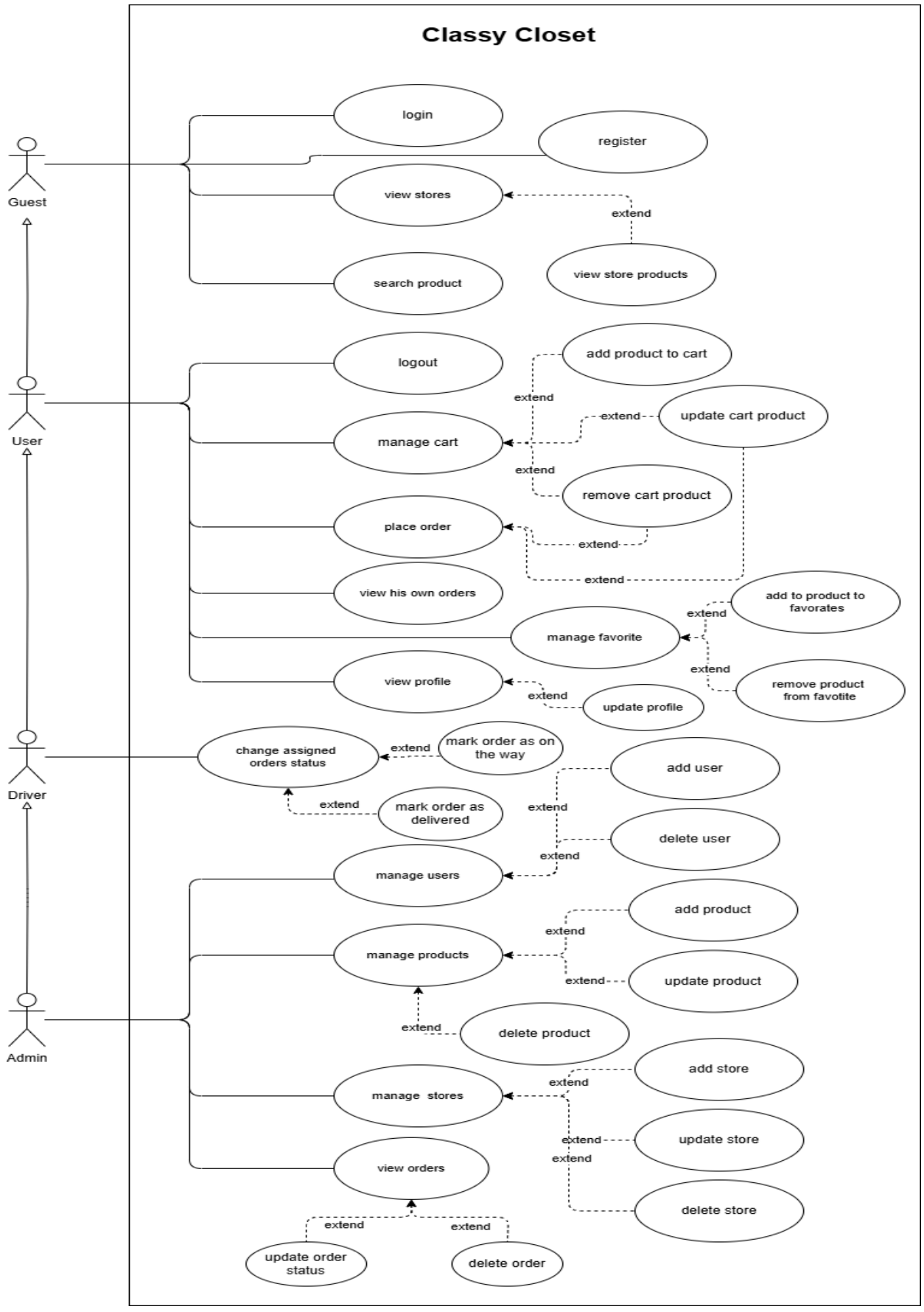


Figure 4.2 : Use Case Diagram

4-7 Use Case Descriptions

Table 4.3: UC-01: Register

UC-01: Register	
Use Case ID	UC-01
Use Case Name	Register
Actor	Guest
Description	A new user creates an account by providing personal information and selecting a role.
Preconditions	- User is not logged in - User has a valid email address
Postconditions	- A new user account is created in the database - User is redirected to the home page
Main Flow	1. Guest clicks Register 2. System displays the registration form 3. Guest fills in name, email, password, address, city, phone 4. Guest selects a role (User or Driver) 5. Guest clicks Submit 6. System validates the data 7. System saves the new user to the database 8. System redirects the user to the home page
Alternative Flow	A1. Invalid data -> System shows validation errors A2. Email already exists -> System shows Email already taken message

Table 4.4: UC-02: Login

UC-02: Login	
Use Case ID	UC-02
Use Case Name	Login
Actor	Guest
Description	A registered user logs into the system using email and password.
Preconditions	- User has a registered account - User is not logged in
Postconditions	- User session is started - User is redirected based on their role
Main Flow	1. Guest clicks Login 2. System displays the login form 3. Guest enters email and password 4. Guest clicks Submit 5. System validates credentials 6. System starts the user session 7. System reads the user role 8. Admin -> redirected to Admin Dashboard Driver -> redirected to Driver Dashboard User -> redirected to Home page
Alternative Flow	A1. Invalid credentials -> System shows error message

	A2. Invalid data format -> System shows validation errors

Table 4.5: UC-03: Place Order

UC-03: Place Order	
Use Case ID	UC-03
Use Case Name	Place Order
Actor	User
Description	The user confirms cart items and submits a new order with a delivery location.
Preconditions	- User is logged in - Cart has at least one product
Postconditions	- A new order is created in the database - Cart is cleared - Admin receives a notification - User order count is updated
Main Flow	1. User opens the cart 2. System displays cart items with subtotal, delivery fee, and discount 3. User enters delivery address and picks location on map 4. User clicks Confirm Order 5. System calculates the total amount 6. System creates the order and order items 7. System clears the cart 8. System sends notification to admin 9. System displays success message
Alternative Flow	A1. Cart is empty -> System shows Cart is empty message A2. Address not provided -> System shows Address required message

Table 4.6: UC-04: Manage Cart

UC-04: Manage Cart	
Use Case ID	UC-04
Use Case Name	Manage Cart
Actor	User
Description	The user manages cart items by adding, updating quantity, or removing products.
Preconditions	- User is logged in
Postconditions	- Cart is updated with the latest changes - Product stock is updated accordingly
Main Flow	1. User browses a store and selects a product 2. User sets quantity and clicks Add to Cart 3. System adds the product to the cart 4. System updates product stock 5. User opens the cart 6. User updates the quantity of a cart item

	7. System updates the item quantity and stock 8. User removes an item from the cart 9. System removes the item and restores stock
Alternative Flow	A1. Product out of stock -> System shows Out of stock message A2. Requested quantity exceeds stock -> System shows insufficient stock message

Table 4.7: UC-05: View His Own Orders

UC-05: View His Own Orders	
Use Case ID	UC-05
Use Case Name	View His Own Orders
Actor	User
Description	The user views a list of their placed orders and tracks the status of each one.
Preconditions	- User is logged in
Postconditions	- Orders are displayed with their current status
Main Flow	1. User opens the home page or My Orders section 2. System retrieves all orders for the logged-in user 3. System displays orders with status and details 4. User selects an order 5. System shows order items, total, driver, and delivery location on map 6. If status is Pending, user can cancel the order 7. System updates the order status to Cancelled
Alternative Flow	A1. No orders found -> System shows No orders yet message A2. Order status is not Pending -> Cancel option is disabled

Table 4.8 UC-06: Manage Favorite

UC-06: Manage Favorite	
Use Case ID	UC-06
Use Case Name	Manage Favorite
Actor	User
Description	The user adds or removes products from their favorites list.
Preconditions	- User is logged in
Postconditions	- Favorites list is updated in the database
Main Flow	1. User browses store products 2. User clicks the favorite icon on a product 3. System checks if the product is already in favorites 4. If not in favorites -> System adds the product to favorites 5. If already in favorites -> System removes the product from favorites 6. System shows a success message 7. User opens the Favorites page 8. System displays all favorite products
Alternative Flow	A1. User is not logged in -> System redirects to login page

Table 4.9 UC-07: View Profile

UC-07: View Profile	
Use Case ID	UC-07
Use Case Name	View Profile
Actor	User
Description	The user views and updates their personal information.
Preconditions	- User is logged in
Postconditions	- Profile information is updated in the database
Main Flow	1. User clicks on their name and selects Profile 2. System displays current profile information 3. User edits name, email, phone, city, address 4. User optionally enters a new password 5. User clicks Save Changes 6. System validates the data 7. System updates user information in the database 8. System shows success message
Alternative Flow	A1. Invalid data -> System shows validation errors A2. Email already taken -> System shows Email already used message

Table 4.10 UC-08: Change Assigned Orders Status

UC-08: Change Assigned Orders Status	
Use Case ID	UC-08
Use Case Name	Change Assigned Orders Status
Actor	Driver
Description	The driver views orders assigned to them and updates their status.
Preconditions	- Driver is logged in - Driver has orders assigned by admin
Postconditions	- Order status is updated - Customer and admin receive a notification
Main Flow	1. Driver opens the driver dashboard 2. System displays all orders assigned to the driver 3. Driver selects an order 4. System shows order details 5. Driver clicks Mark as On the Way 6. System updates order status to On the Way 7. System notifies the customer and admin 8. Driver clicks Mark as Delivered 9. System updates order status to Delivered 10. System notifies the customer and admin
Alternative Flow	A1. No assigned orders -> System shows No assigned orders message

Table 4.11 UC-09: Manage Users

UC-09: Manage Users	
Use Case ID	UC-09
Use Case Name	Manage Users
Actor	Admin
Description	The admin views all registered users and can delete any user account.
Preconditions	- Admin is logged in
Postconditions	- User is deleted from the database
Main Flow	1. Admin opens the admin dashboard 2. Admin clicks Manage Users 3. System displays all users with their information 4. Admin views user details (name, email, role) 5. Admin clicks Delete on a user 6. System deletes the user from the database 7. System shows success message
Alternative Flow	A1. No users found -> System shows No users found message

Table 4.12 UC-10: Manage Products

UC-10: Manage Products	
Use Case ID	UC-10
Use Case Name	Manage Products
Actor	Admin
Description	The admin adds, edits, or deletes products and links them to a specific store.
Preconditions	- Admin is logged in - At least one store exists in the system
Postconditions	- Product is added, updated, or deleted in the database
Main Flow	1. Admin opens Manage Products 2. System displays all products with their stores 3. To add: Admin fills the product form and selects a store, System saves the product 4. To edit: Admin selects a product, updates the form, System updates the product 5. To delete: Admin clicks Delete, System removes the product from the database 6. System shows success message after each operation
Alternative Flow	A1. Invalid data -> System shows validation errors A2. No stores available -> Admin cannot add a product

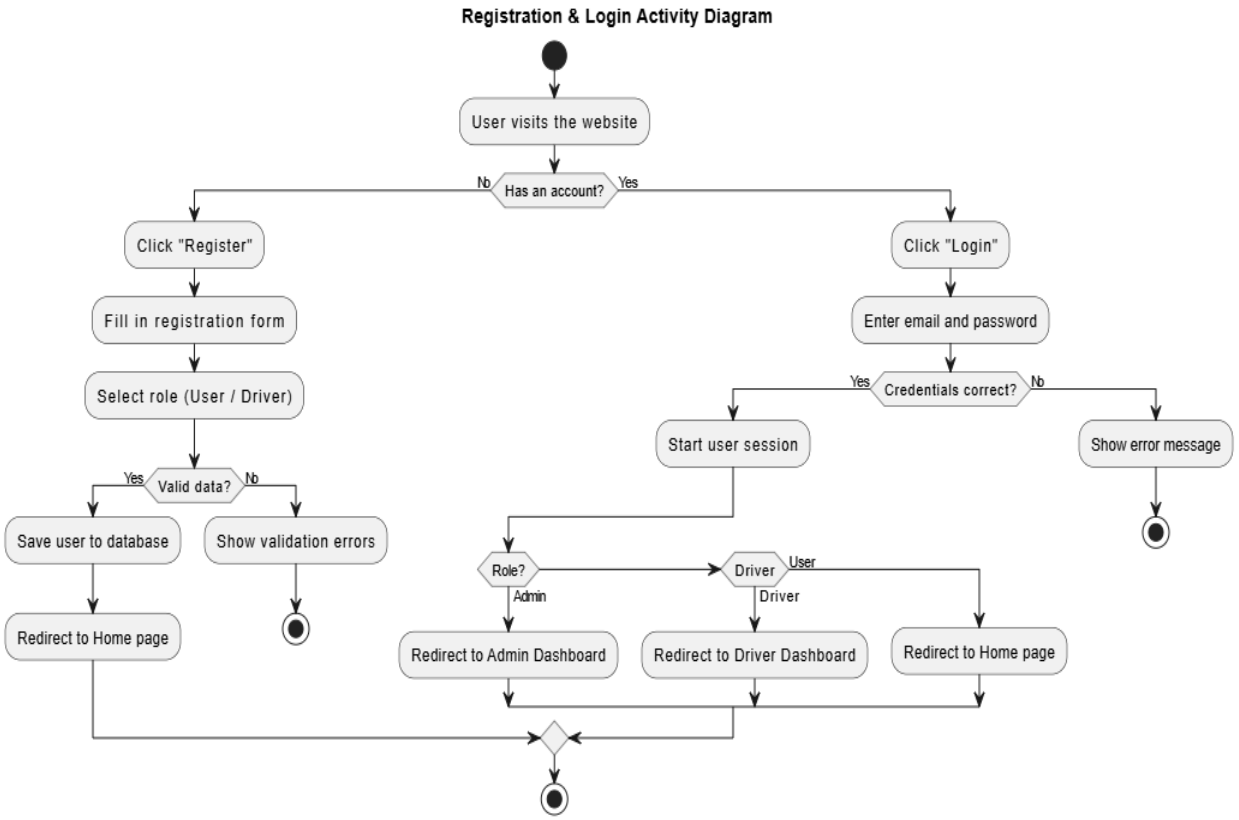
Table 4.13 UC-11: Manage Stores

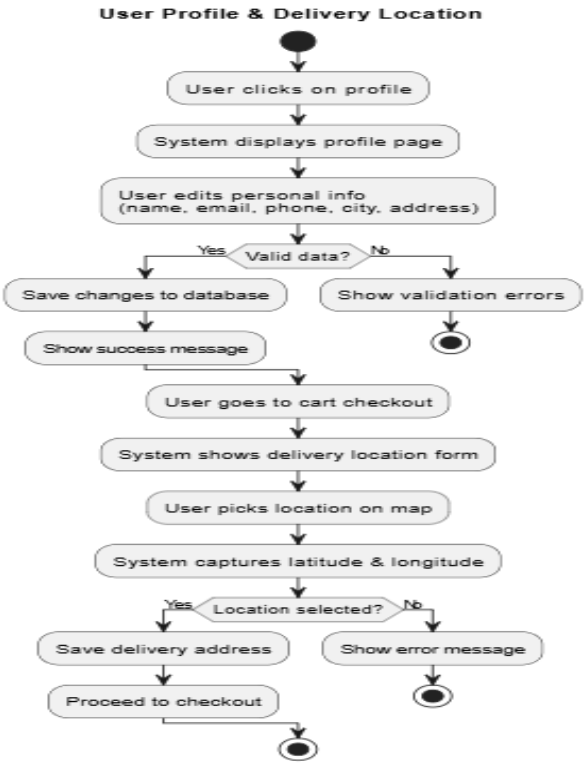
UC-11: Manage Stores	
Use Case ID	UC-11
Use Case Name	Manage Stores
Actor	Admin
Description	The admin adds, edits, or deletes stores including type and delivery fee.
Preconditions	- Admin is logged in
Postconditions	- Store is added, updated, or deleted in the database
Main Flow	1. Admin opens Manage Stores 2. System displays all stores 3. To add: Admin fills store form (name, image, type, delivery fee), System saves the store 4. To edit: Admin selects a store, updates the form, System updates the store 5. To delete: Admin clicks Delete, System removes image from storage and deletes store 6. System shows success message after each operation
Alternative Flow	A1. Invalid data -> System shows validation errors

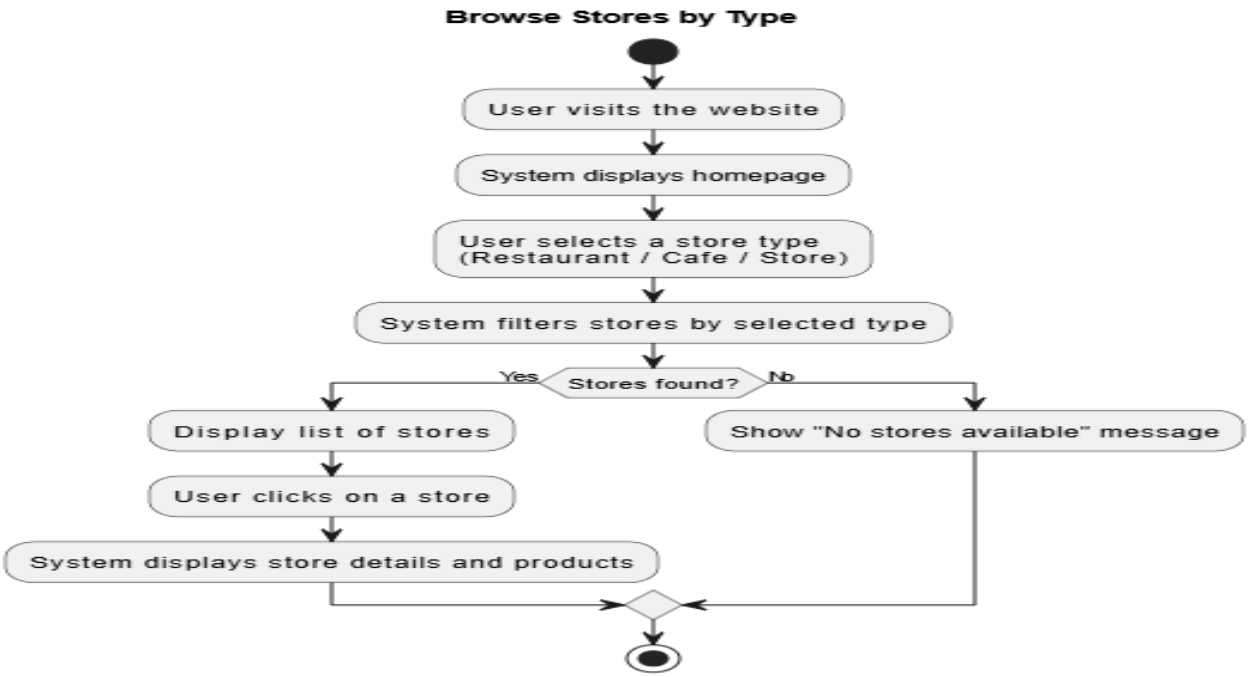
Table 4.14 UC-12: View Orders

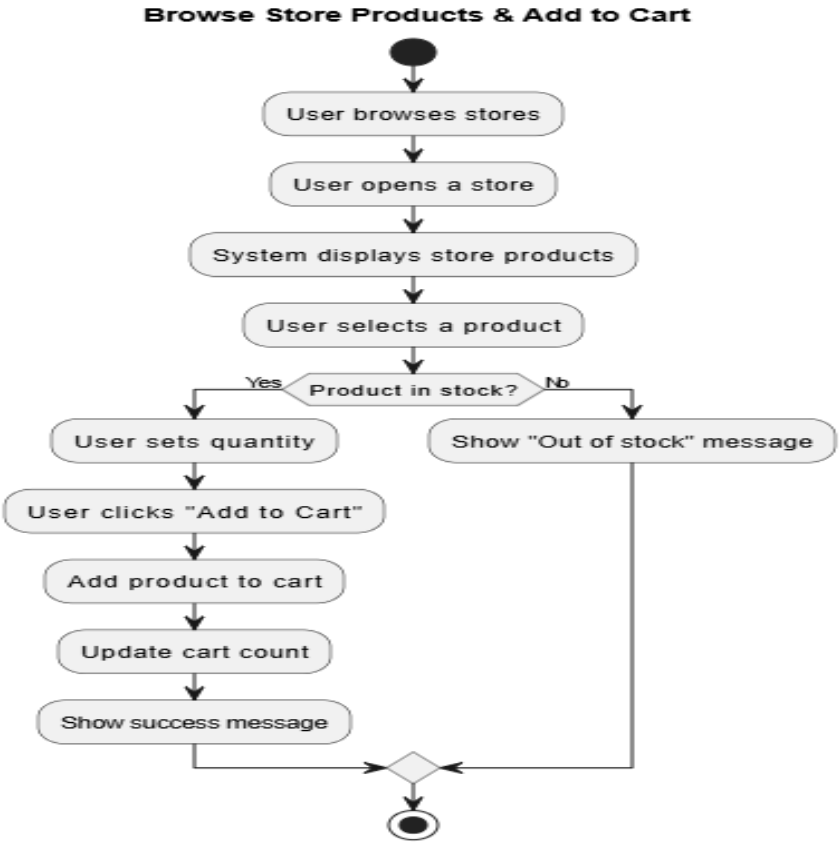
UC-12: View Orders	
Use Case ID	UC-12
Use Case Name	View Orders
Actor	Admin
Description	The admin views all orders, assigns drivers, updates order status, or cancels orders.
Preconditions	- Admin is logged in
Postconditions	- Order is updated with new status or assigned driver - Relevant parties are notified
Main Flow	1. Admin opens Manage Orders 2. System displays all orders with customer and driver info 3. Admin selects an order to view details 4. To assign driver: Admin selects a driver, System assigns driver and sets status to In Progress 5. To update status: Admin selects new status, System updates and notifies customer and driver 6. To cancel: Admin cancels a Pending order, System updates status and notifies customer 7. System shows success message after each operation
Alternative Flow	A1. No driver assigned -> System blocks status update with error message A2. Order is not Pending -> System blocks cancellation with error message

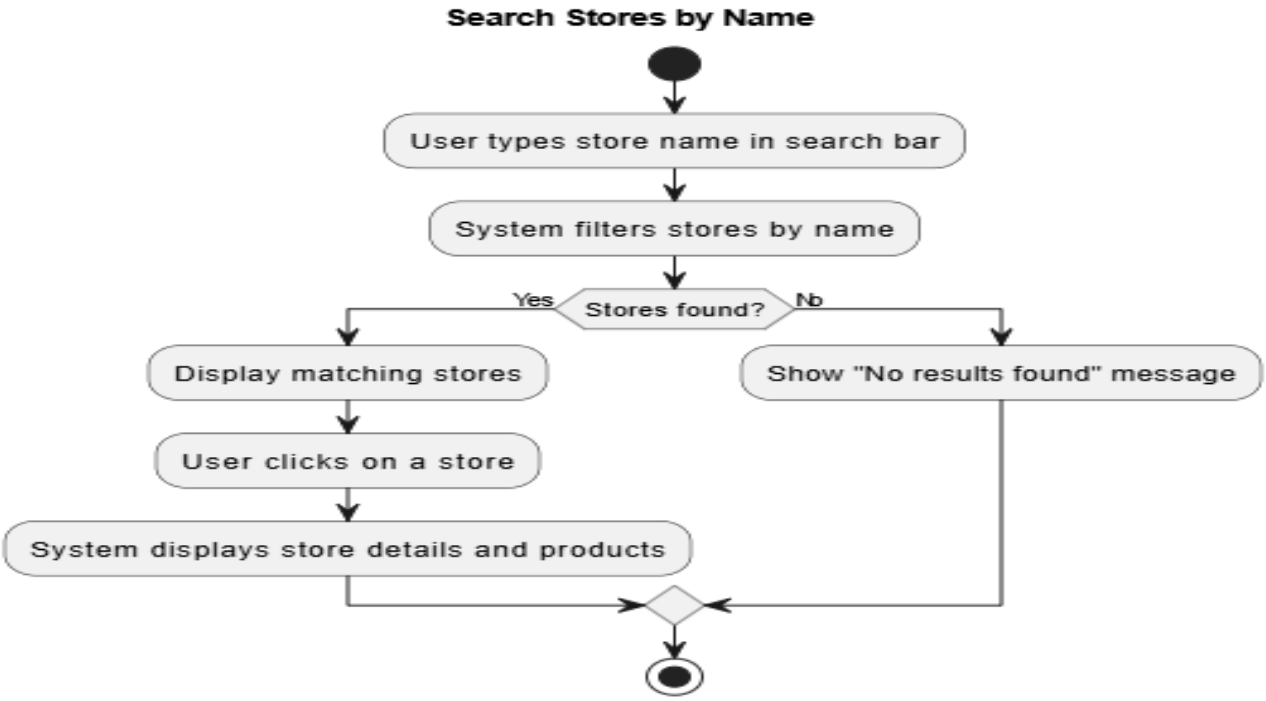
4-8 Activity Diagram

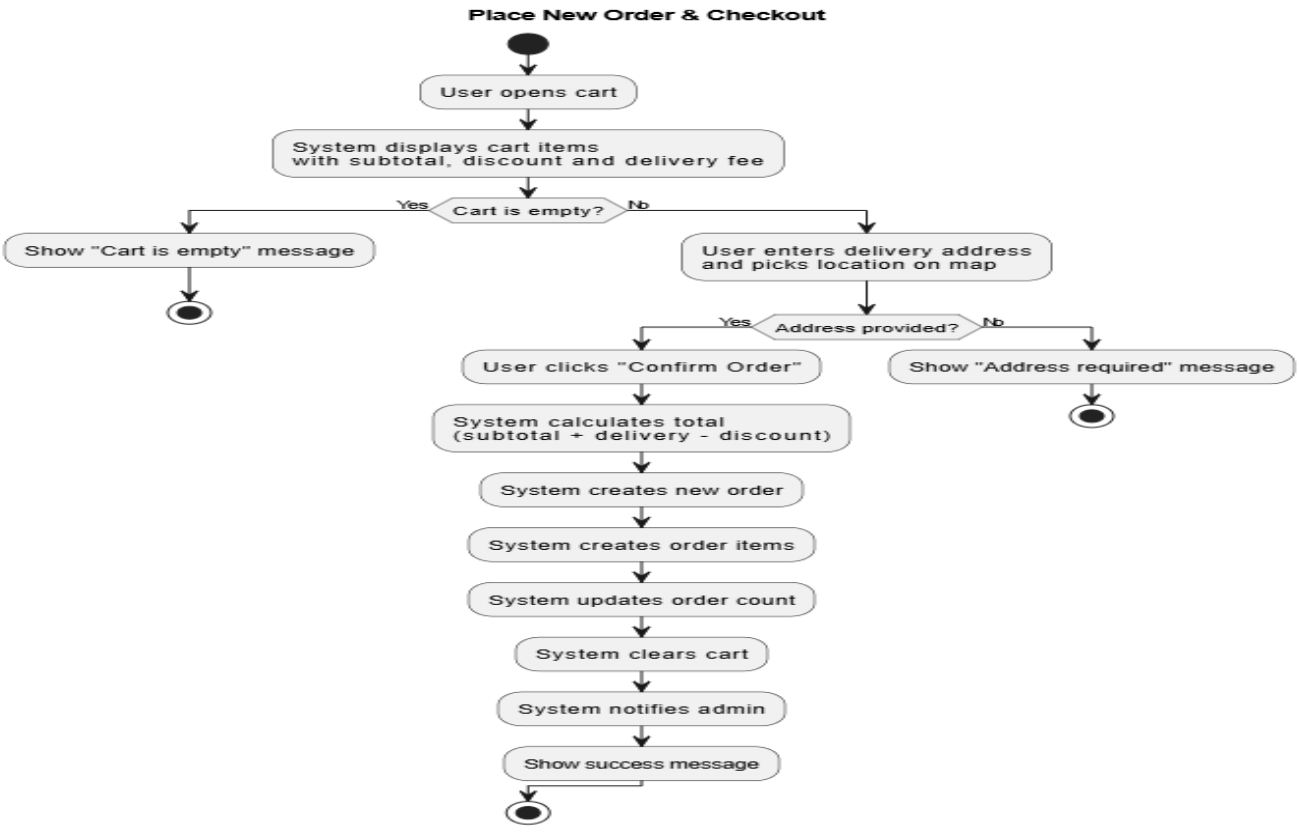


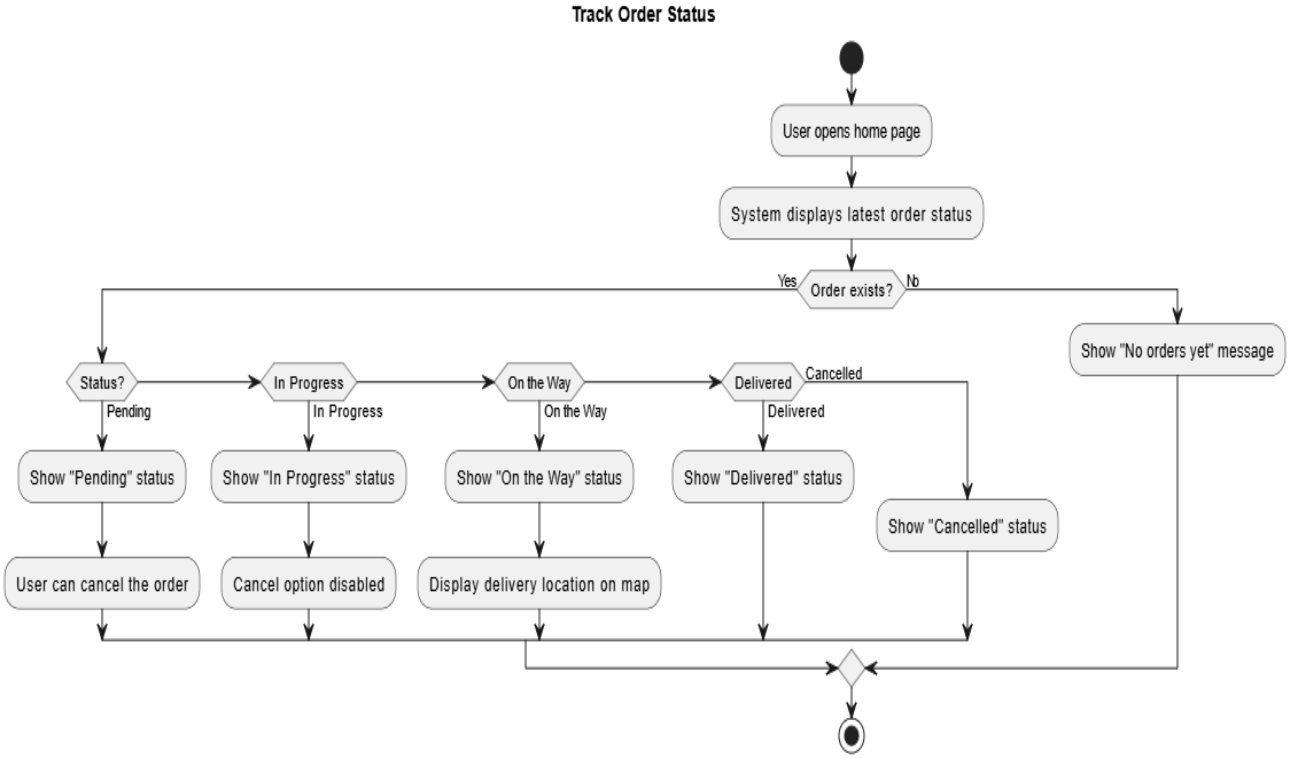


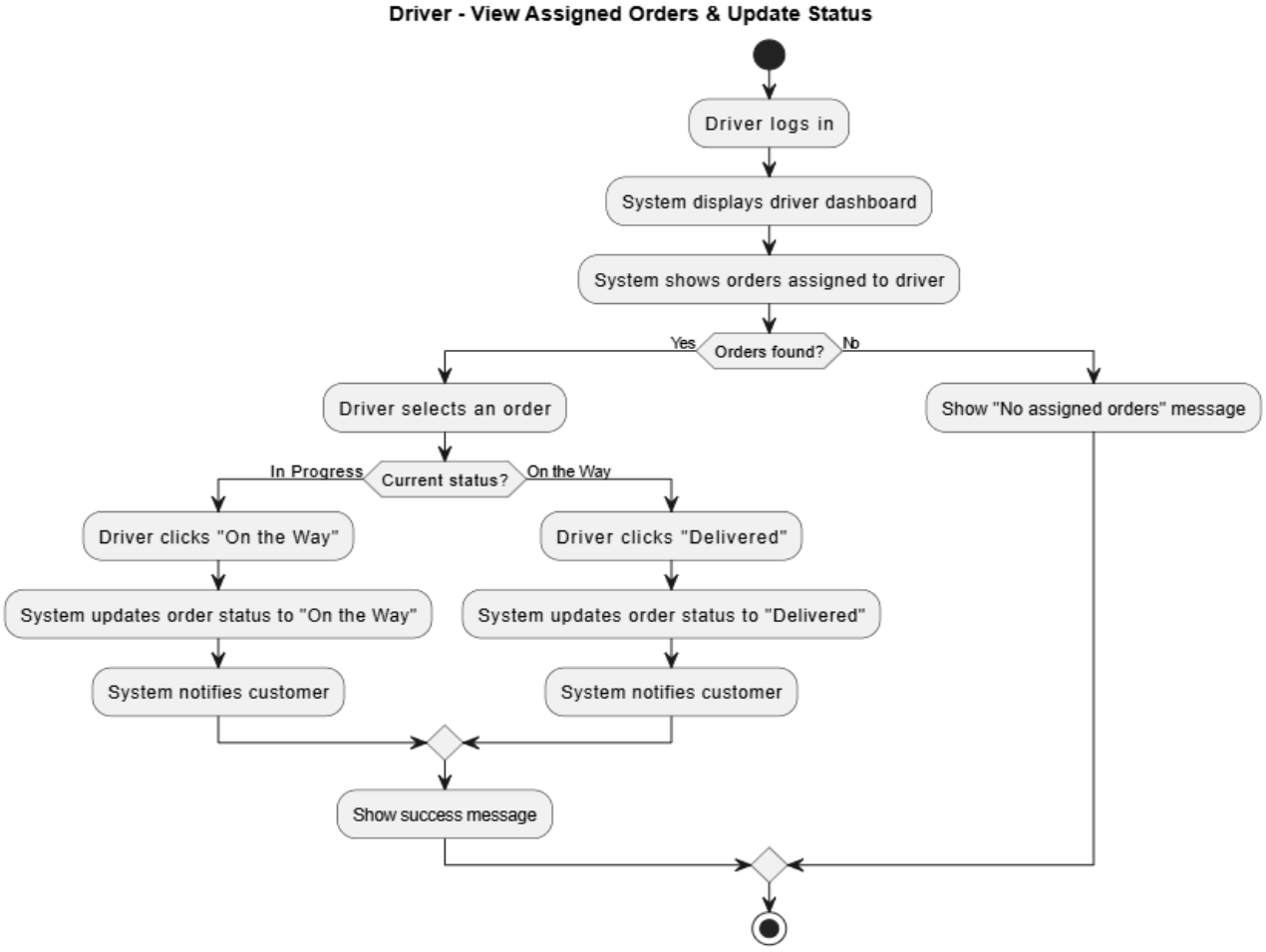

Browse Stores by Type
User visits the website
System displays homepage
User selects a store type (Restaurant / Cafe / Store)
System filters stores by selected type
Stores found?
Yes
No
Display list of stores
User clicks on a store
System displays store details and products
Show "No stores available" message

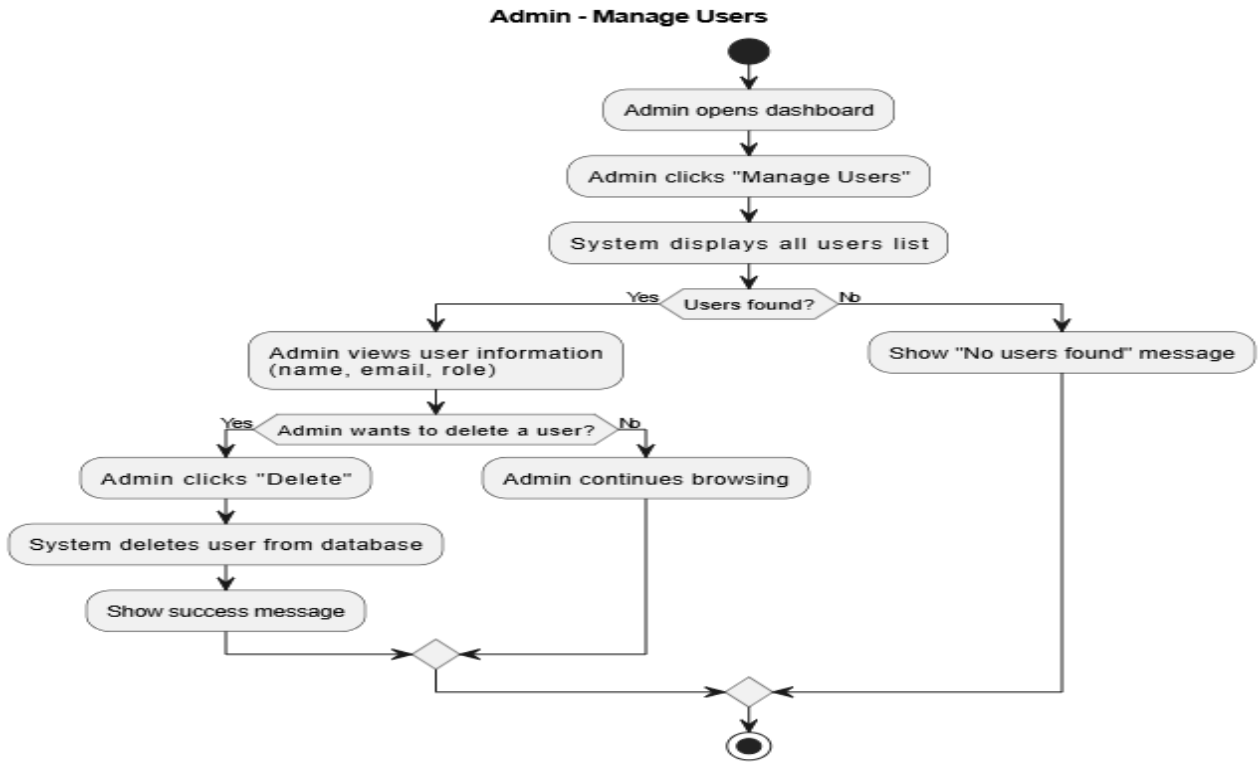

Browse Store Products & Add to Cart
User browses stores
User opens a store
System displays store products
User selects a product
Product in stock?
Yes
No
User sets quantity
User clicks "Add to Cart"
Add product to cart
Update cart count
Show success message
Show "Out of stock" message

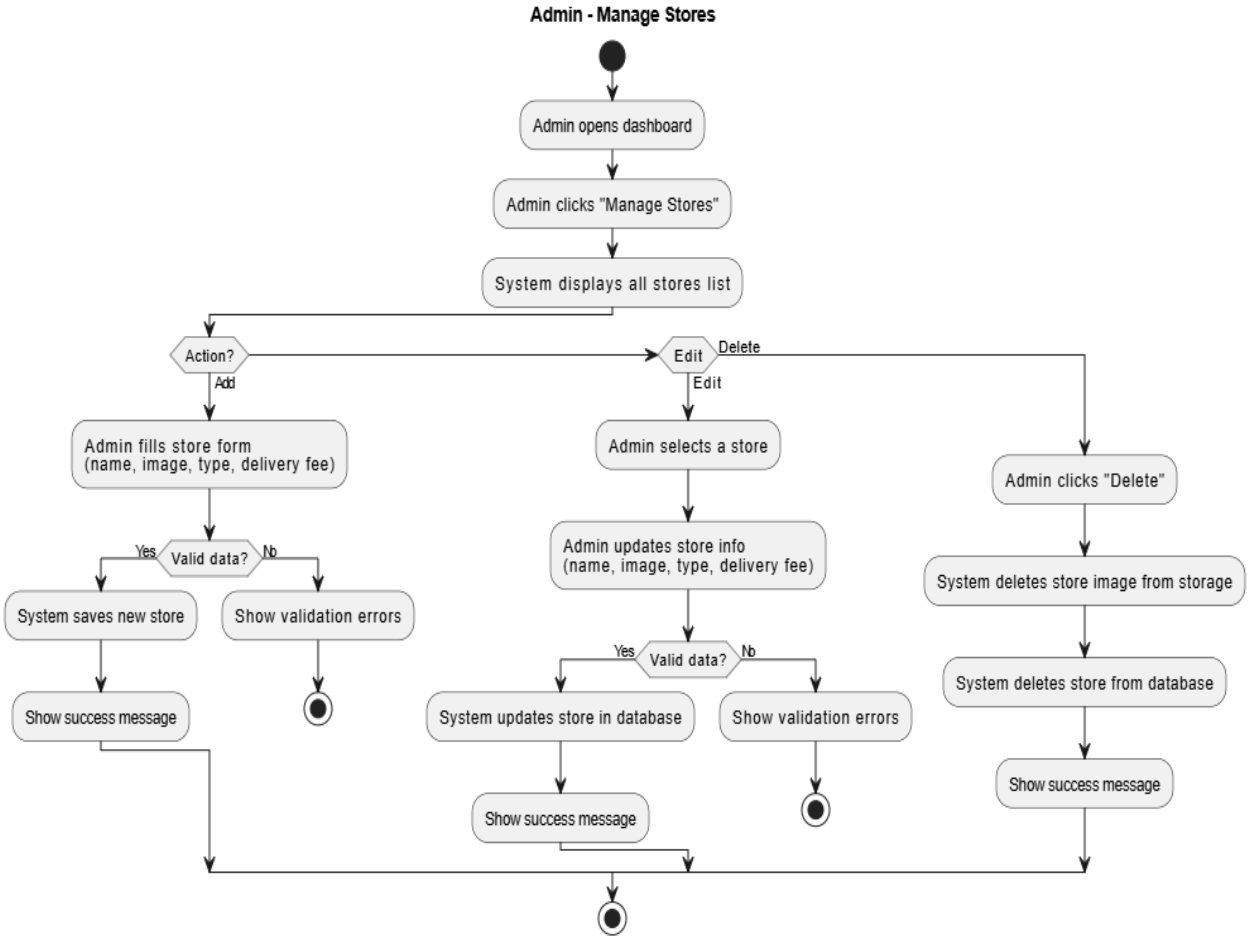

Search Stores by Name
User types store name in search bar
System filters stores by name
Yes
Stores found?
No
Display matching stores
Show "No results found" message
User clicks on a store
System displays store details and products

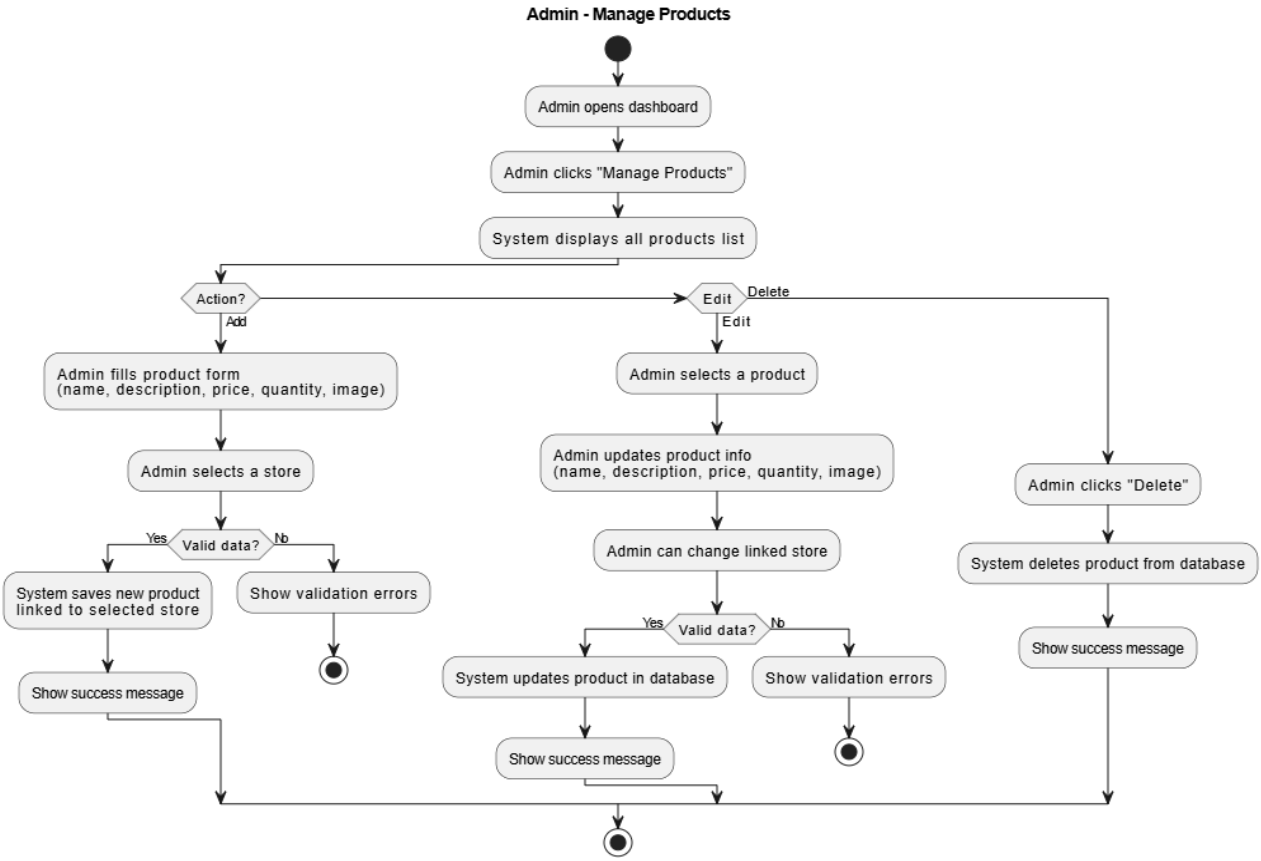

Place New Order & Checkout
User opens cart
System displays cart items
with subtotal, discount and delivery fee
Yes
Cart is empty?
No
Show "Cart is empty" message
User enters delivery address
and picks location on map
Yes
Address provided?
No
User clicks "Confirm Order"
Show "Address required" message
System calculates total
(subtotal + delivery - discount)
System creates new order
System creates order items
System updates order count
System clears cart
System notifies admin
Show success message

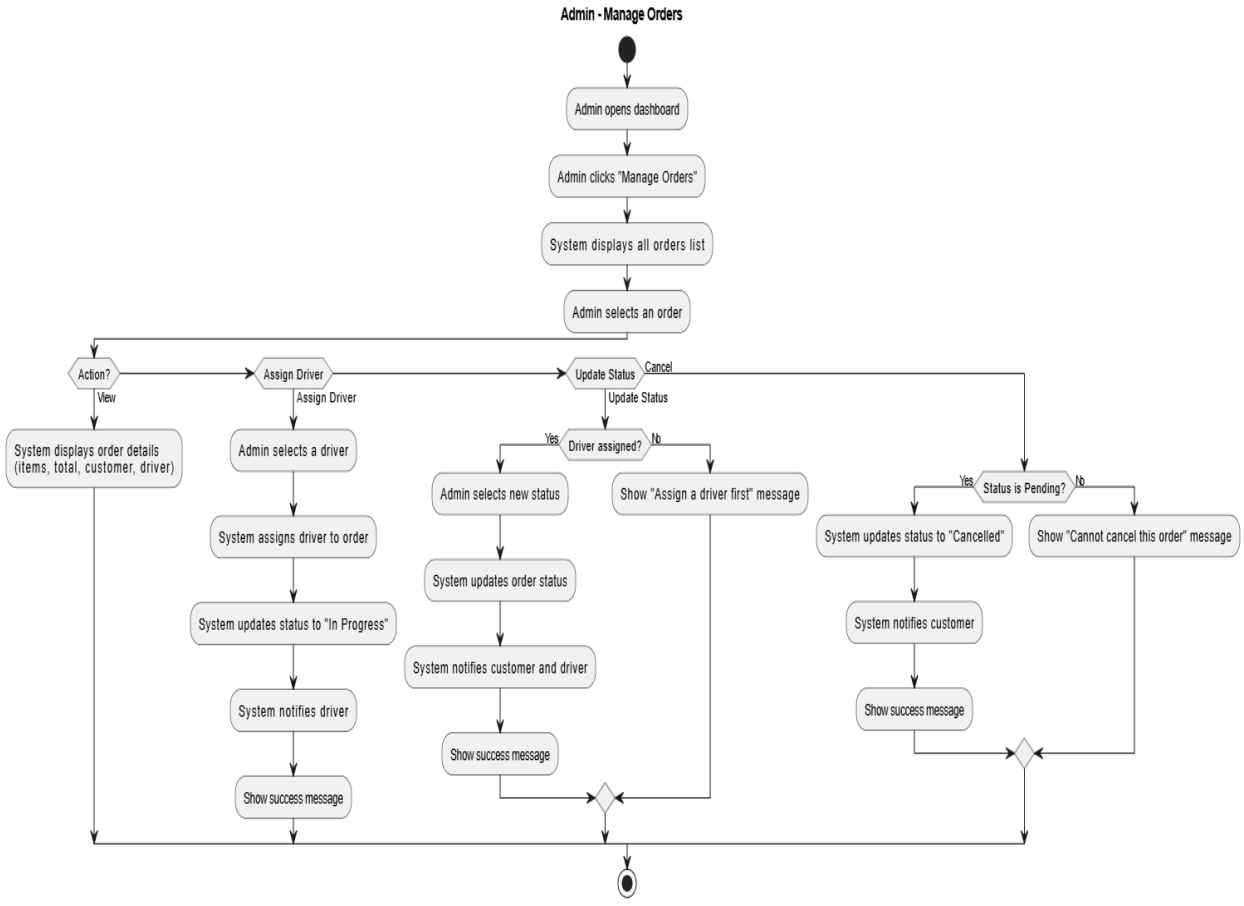

Track Order Status
User opens home page
System displays latest order status
Order exists?
Yes
No
Show "No orders yet" message
Status?
Pending
In Progress
On the Way
Delivered
Cancelled
Show "Pending" status
Show "In Progress" status
Show "On the Way" status
Show "Delivered" status
Show "Cancelled" status
User can cancel the order
Cancel option disabled
Display delivery location on map

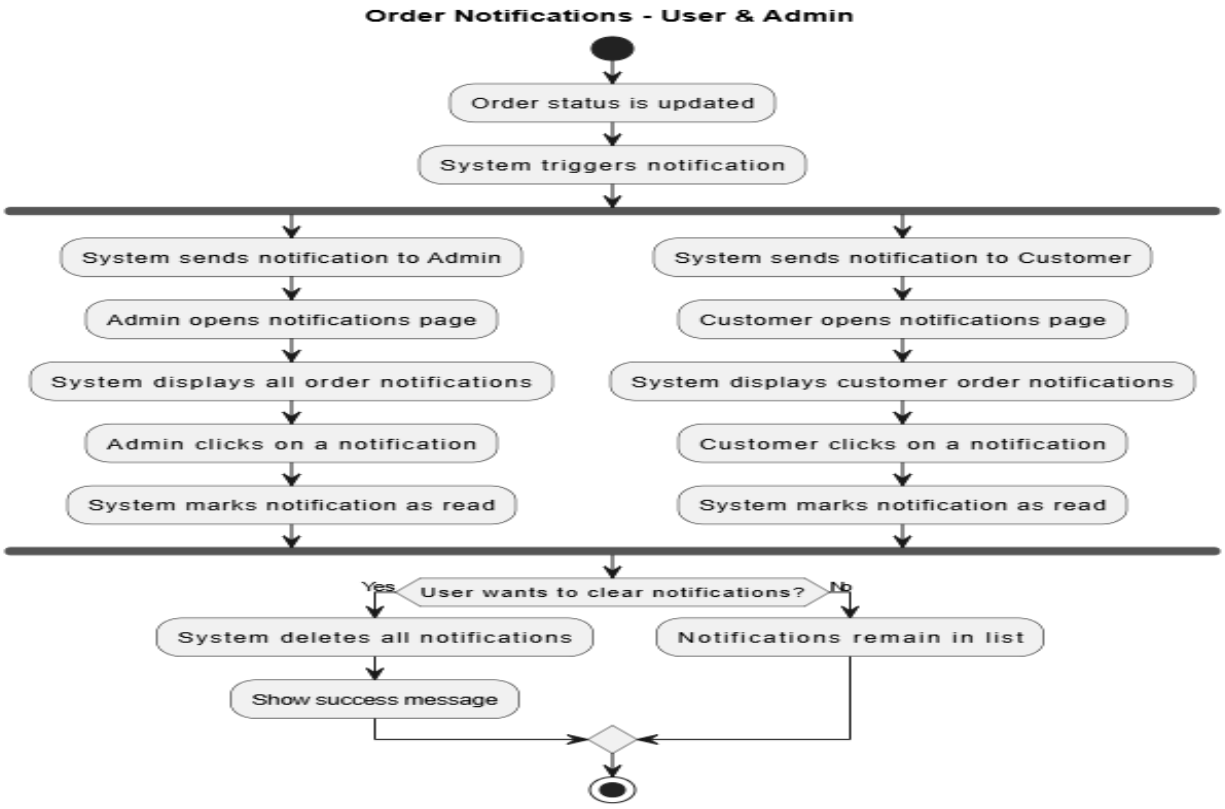

Driver - View Assigned Orders & Update Status
Driver logs in
System displays driver dashboard
System shows orders assigned to driver
Orders found?
Yes
No
Driver selects an order
Show "No assigned orders" message
Current status?
In Progress
On the Way
Driver clicks "On the Way"
Driver clicks "Delivered"
System updates order status to "On the Way"
System updates order status to "Delivered"
System notifies customer
System notifies customer
Show success message


Admin - Manage Users
Admin opens dashboard
Admin clicks "Manage Users"
System displays all users list
Users found?
Yes
No
Admin views user information (name, email, role)
Show "No users found" message
Admin wants to delete a user?
Yes
No
Admin clicks "Delete"
Admin continues browsing
System deletes user from database
Show success message

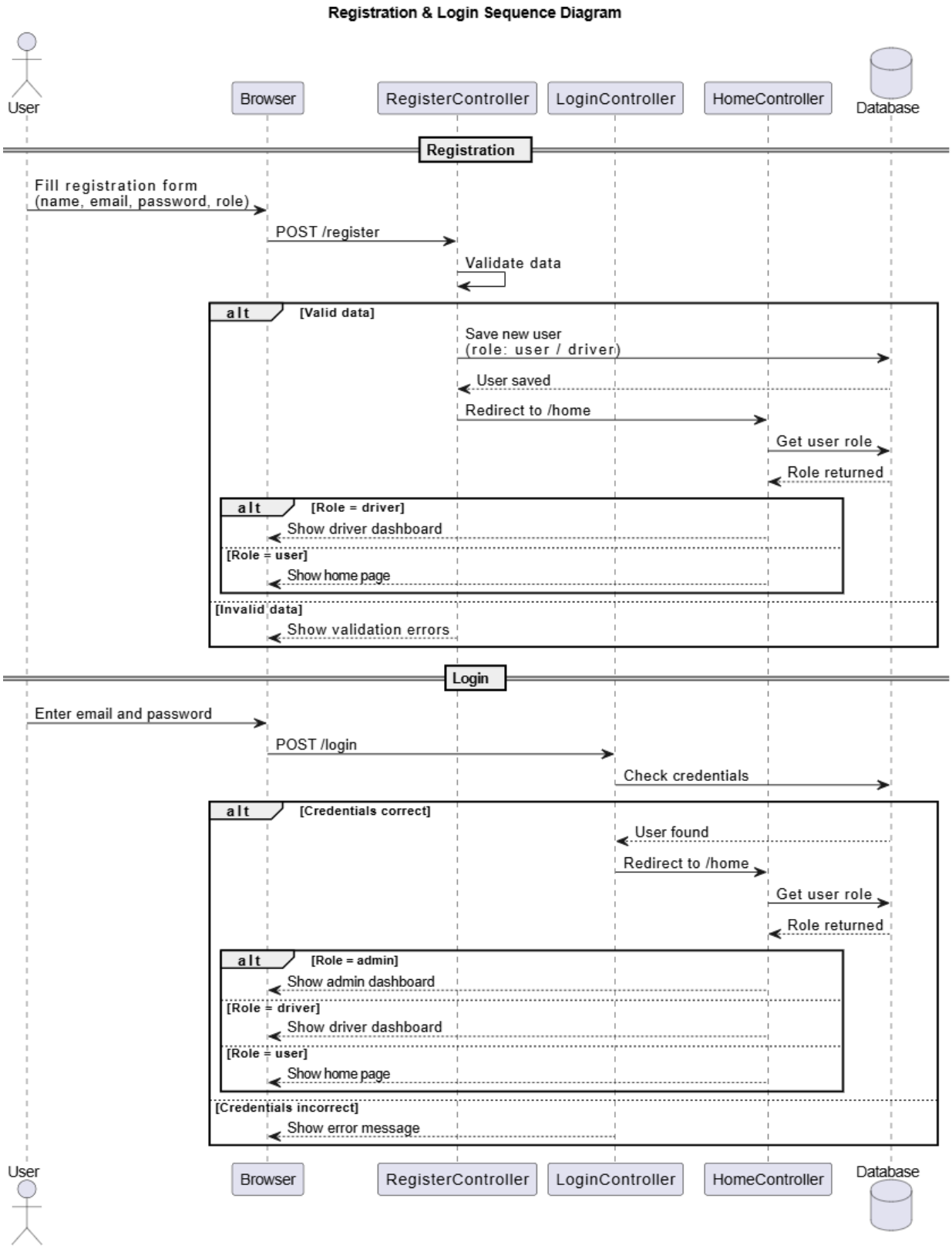

Admin - Manage Stores
Admin opens dashboard
Admin clicks "Manage Stores"
System displays all stores list
Action?
Add
Edit
Delete
Admin fills store form (name, image, type, delivery fee)
Admin selects a store
Admin clicks "Delete"
Valid data?
Yes
No
Admin updates store info (name, image, type, delivery fee)
System deletes store image from storage
System saves new store
Show validation errors
Valid data?
Yes
No
System deletes store from database
Show success message
System updates store in database
Show validation errors
Show success message

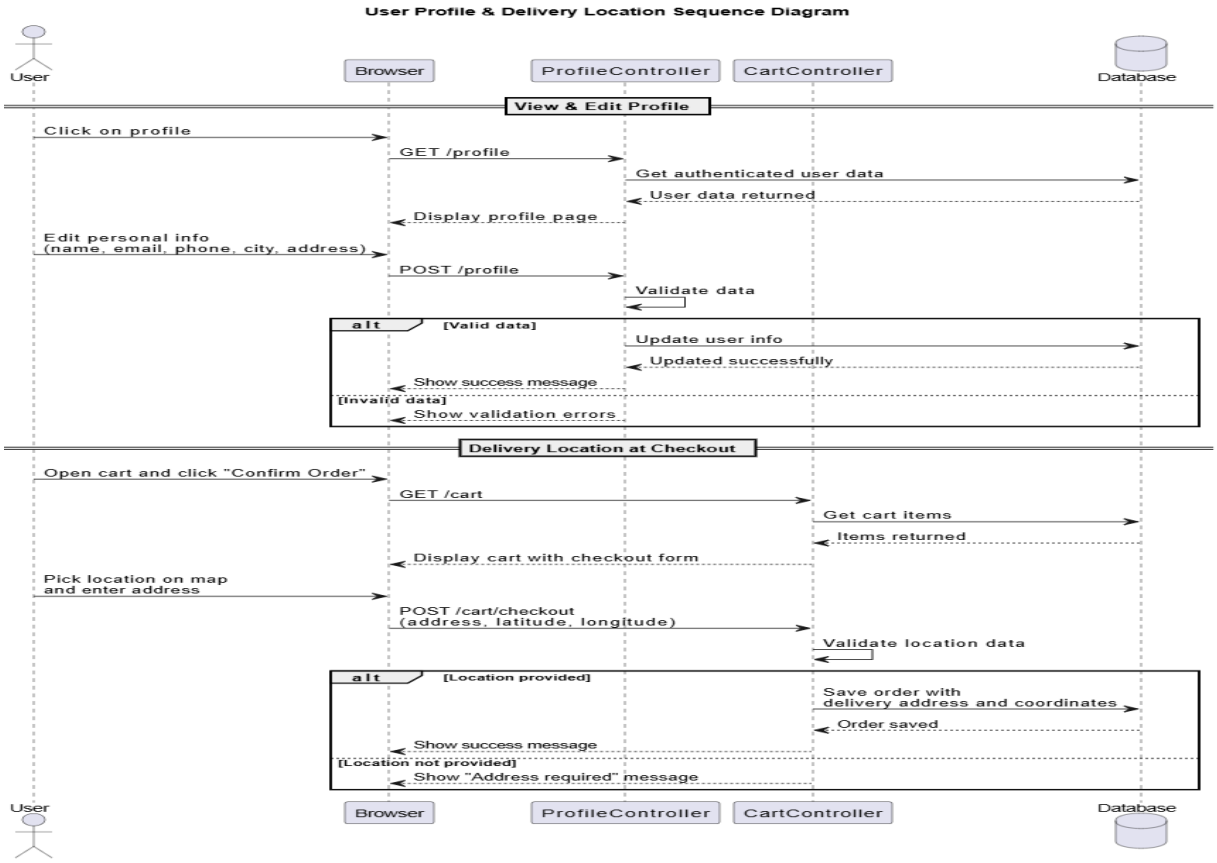

Admin - Manage Products
Admin opens dashboard
Admin clicks "Manage Products"
System displays all products list
Action?
Add
Edit
Delete
Admin fills product form
(name, description, price, quantity, image)
Admin selects a store
Yes
Valid data?
No
System saves new product
linked to selected store
Show validation errors
Show success message
Admin selects a product
Admin updates product info
(name, description, price, quantity, image)
Admin can change linked store
Yes
Valid data?
No
System updates product in database
Show validation errors
Show success message
Admin clicks "Delete"
System deletes product from database
Show success message

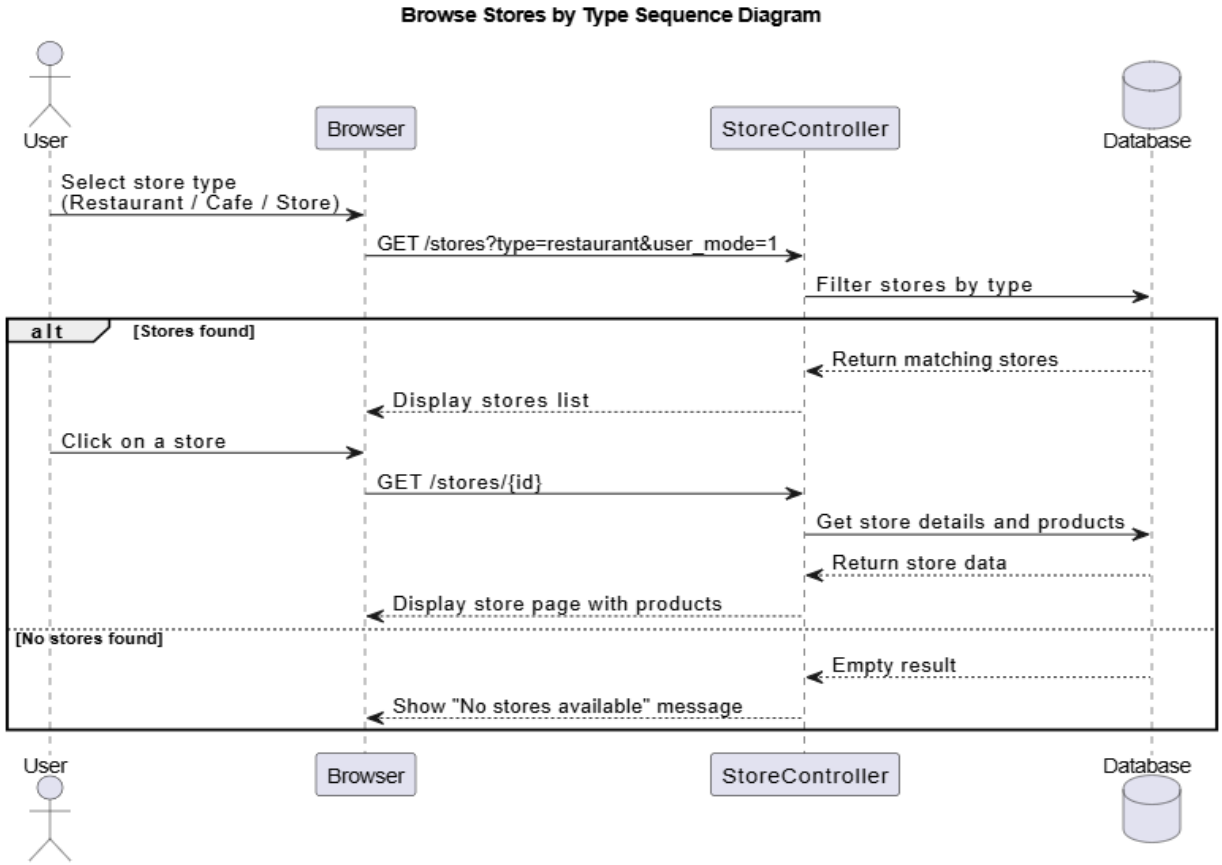

Admin - Manage Orders
Admin opens dashboard
Admin clicks "Manage Orders"
System displays all orders list
Admin selects an order
Action?
View
Assign Driver
Update Status
Cancel
System displays order details
(items, total, customer, driver)
Assign Driver
Admin selects a driver
System assigns driver to order
System updates status to "In Progress"
System notifies driver
Show success message
Update Status
Yes
Driver assigned?
No
Admin selects new status
System updates order status
System notifies customer and driver
Show success message
Show "Assign a driver first" message
Yes
Status is Pending?
No
System updates status to "Cancelled"
System notifies customer
Show success message
Show "Cannot cancel this order" message

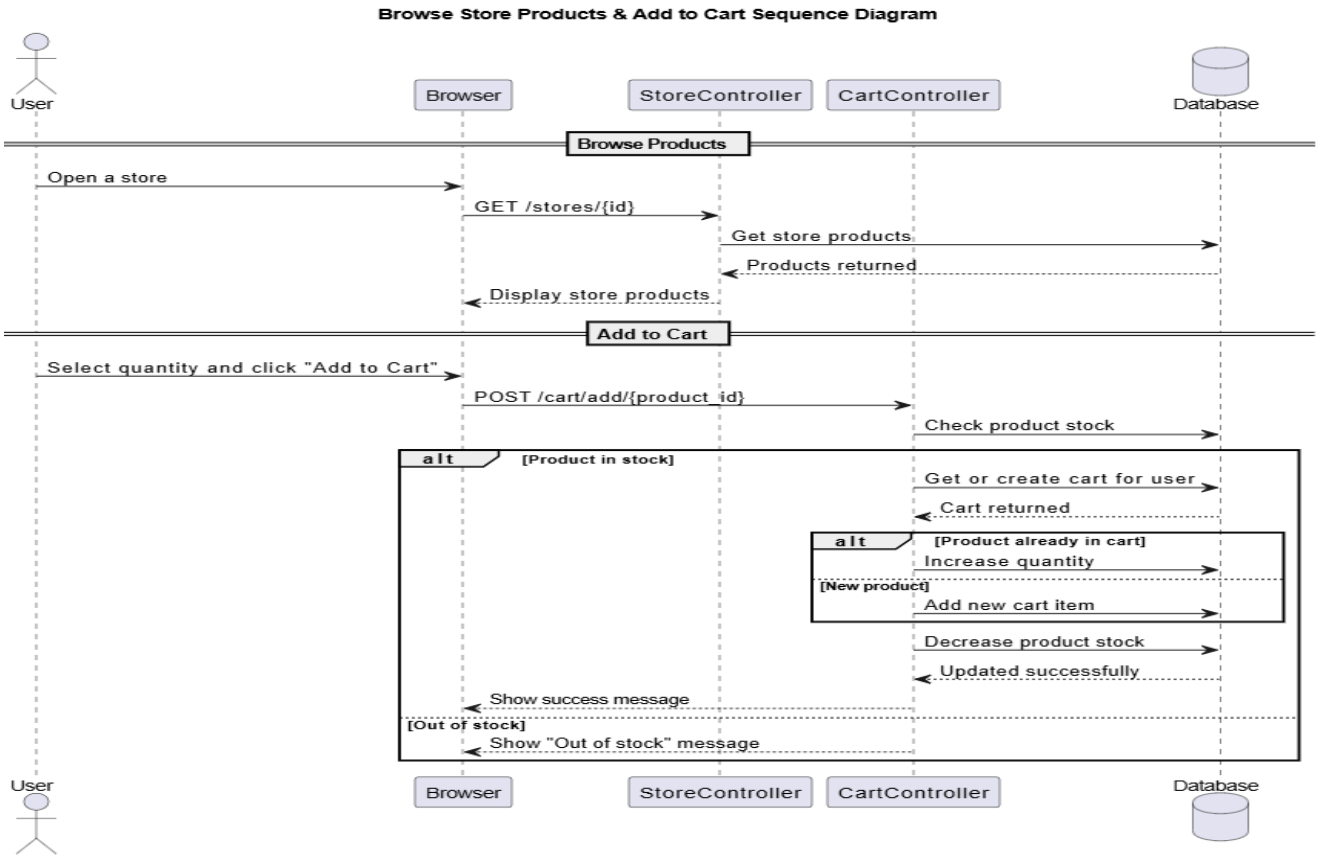

Order Notifications - User & Admin
Order status is updated
System triggers notification
System sends notification to Admin
Admin opens notifications page
System displays all order notifications
Admin clicks on a notification
System marks notification as read
System sends notification to Customer
Customer opens notifications page
System displays customer order notifications
Customer clicks on a notification
System marks notification as read
User wants to clear notifications?
Yes
No
System deletes all notifications
Show success message
Notifications remain in list

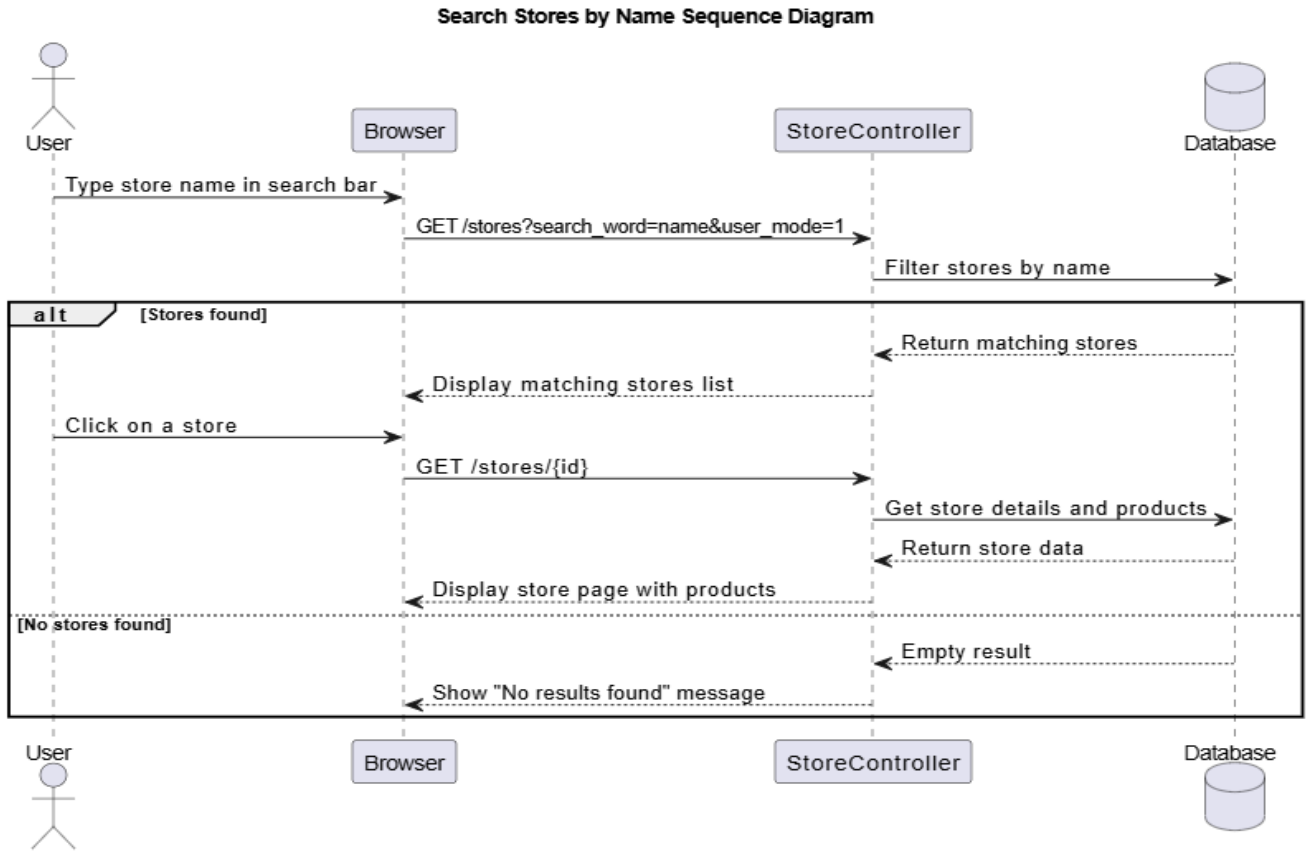
4-9 Sequence Diagram

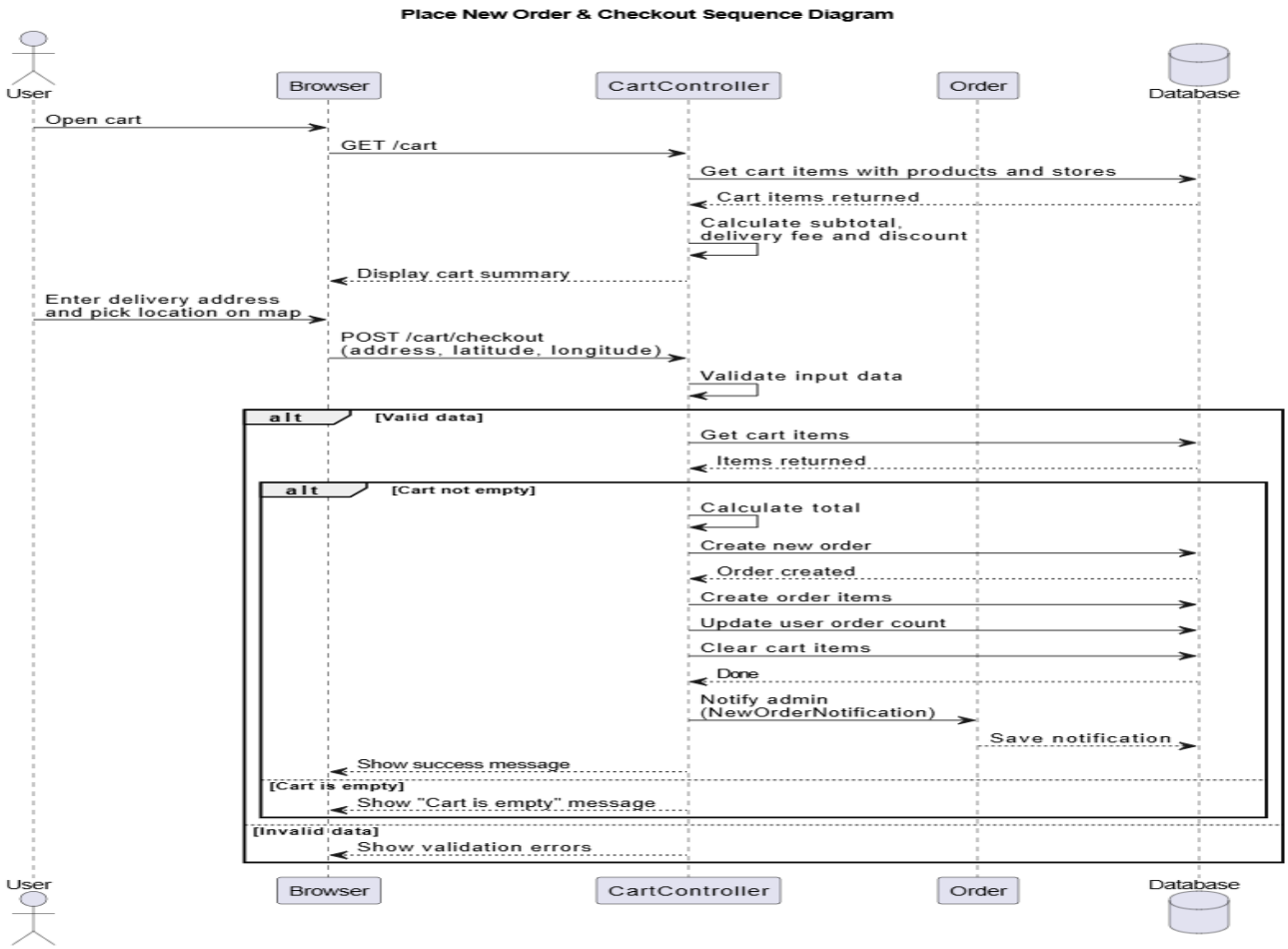


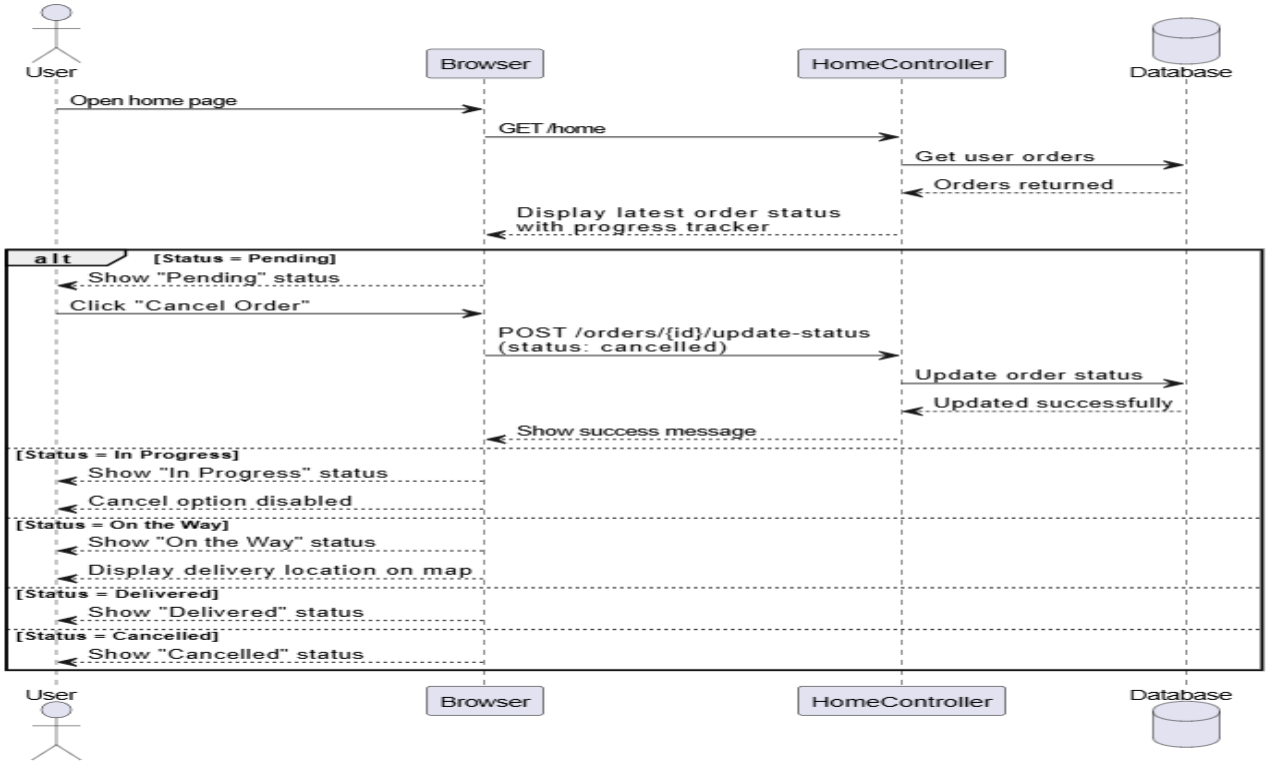

User Profile & Delivery Location Sequence Diagram
User
Browser
ProfileController
CartController
Database
View & Edit Profile
Click on profile
GET /profile
Get authenticated user data
User data returned
Display profile page
Edit personal info
(name, email, phone, city, address)
POST /profile
Validate data
alt
[Valid data]
Update user info
Updated successfully
Show success message
[Invalid data]
Show validation errors
Delivery Location at Checkout
Open cart and click "Confirm Order"
GET /cart
Get cart items
Items returned
Display cart with checkout form
Pick location on map
and enter address
POST /cart/checkout
(address, latitude, longitude)
Validate location data
alt
[Location provided]
Save order with
delivery address and coordinates
Order saved
Show success message
[Location not provided]
Show "Address required" message
User
Browser
ProfileController
CartController
Database

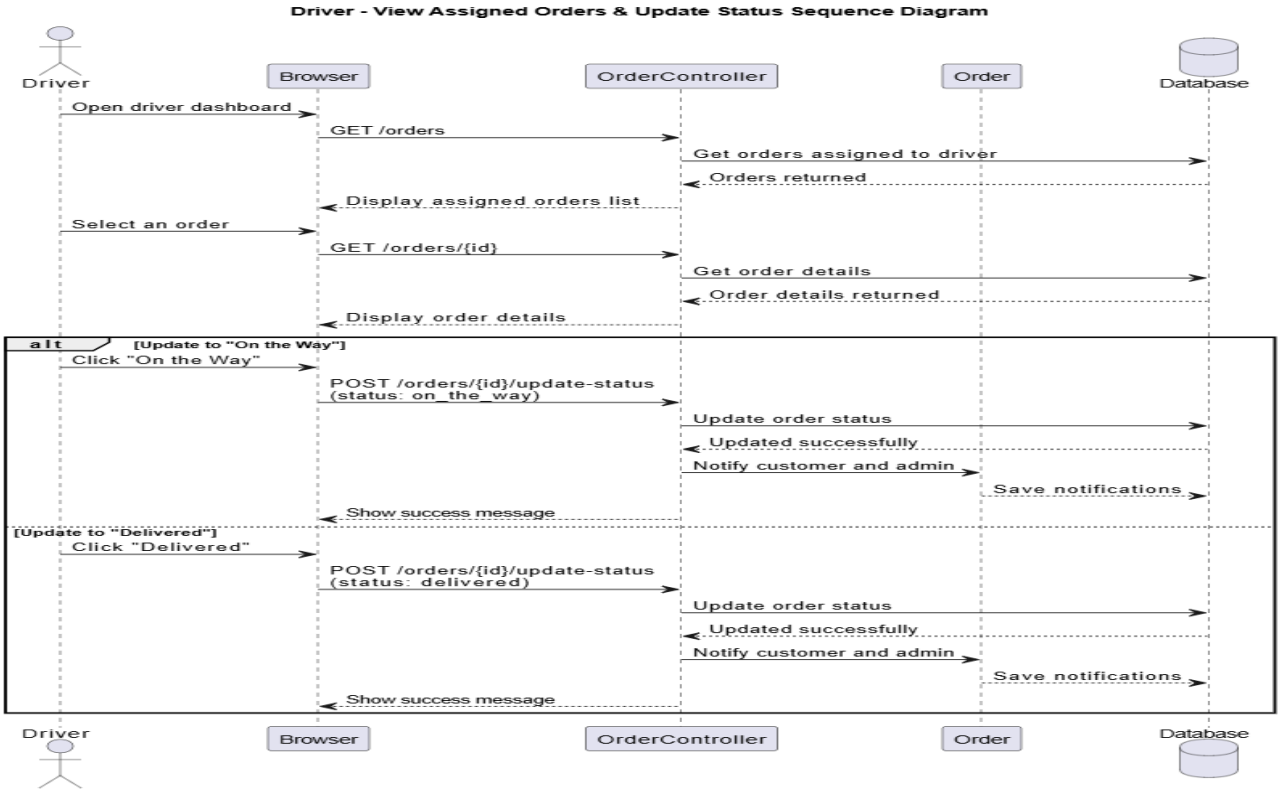

Browse Stores by Type Sequence Diagram
User
Browser
StoreController
Database
Select store type
(Restaurant / Cafe / Store)
GET /stores?type=restaurant&user_mode=1
Filter stores by type
alt
[Stores found]
Return matching stores
Display stores list
Click on a store
GET /stores/{id}
Get store details and products
Return store data
Display store page with products
[No stores found]
Empty result
Show "No stores available" message
User
Browser
StoreController
Database

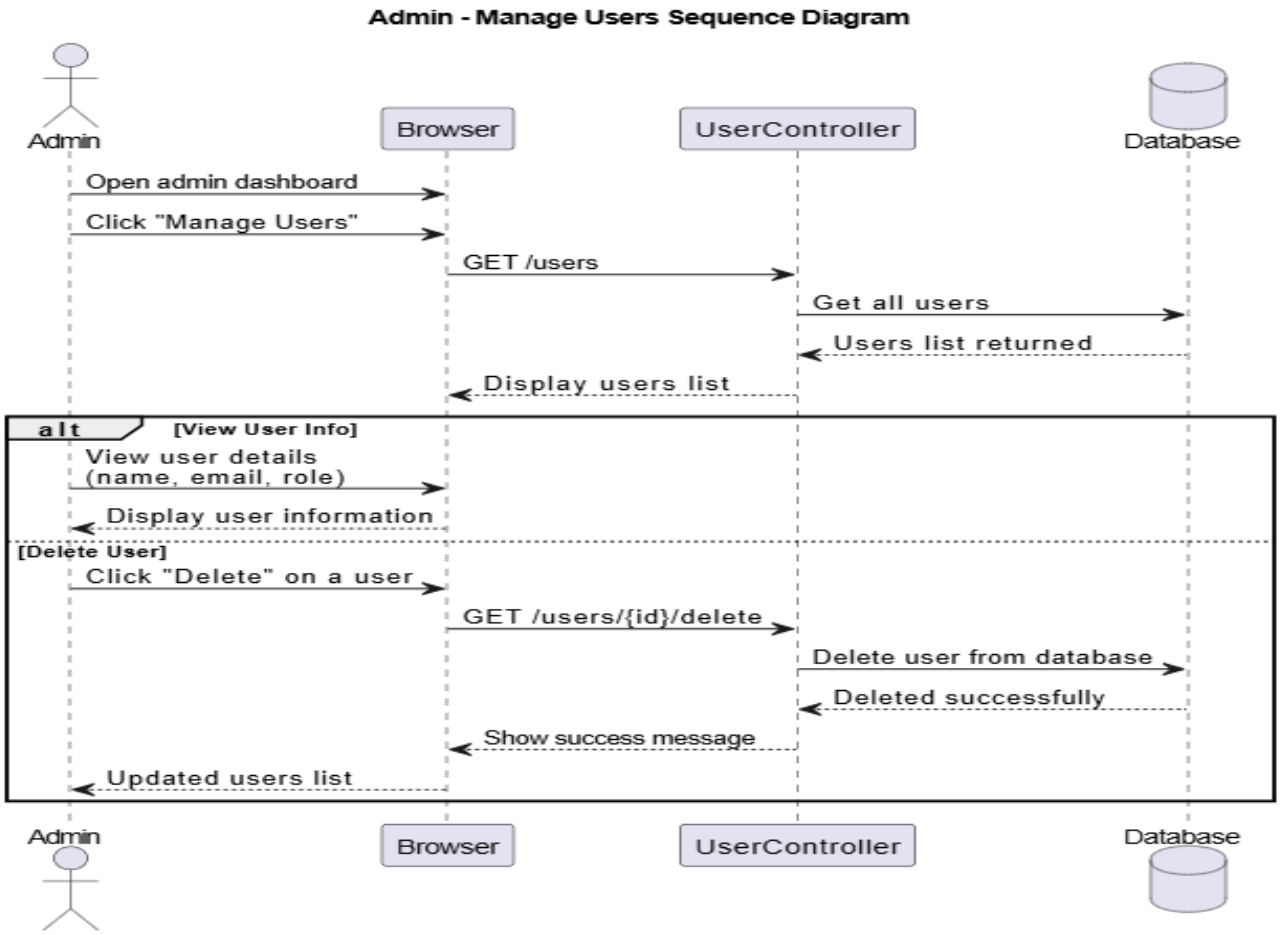

Browse Store Products & Add to Cart Sequence Diagram
User
Browser
StoreController
CartController
Database
Browse Products
Open a store
GET /stores/{id}
Get store products
Products returned
Display store products
Add to Cart
Select quantity and click "Add to Cart"
POST /cart/add/{product_id}
Check product stock
alt
[Product in stock]
Get or create cart for user
Cart returned
alt
[Product already in cart]
Increase quantity
[New product]
Add new cart item
Decrease product stock
Updated successfully
Show success message
[Out of stock]
Show "Out of stock" message

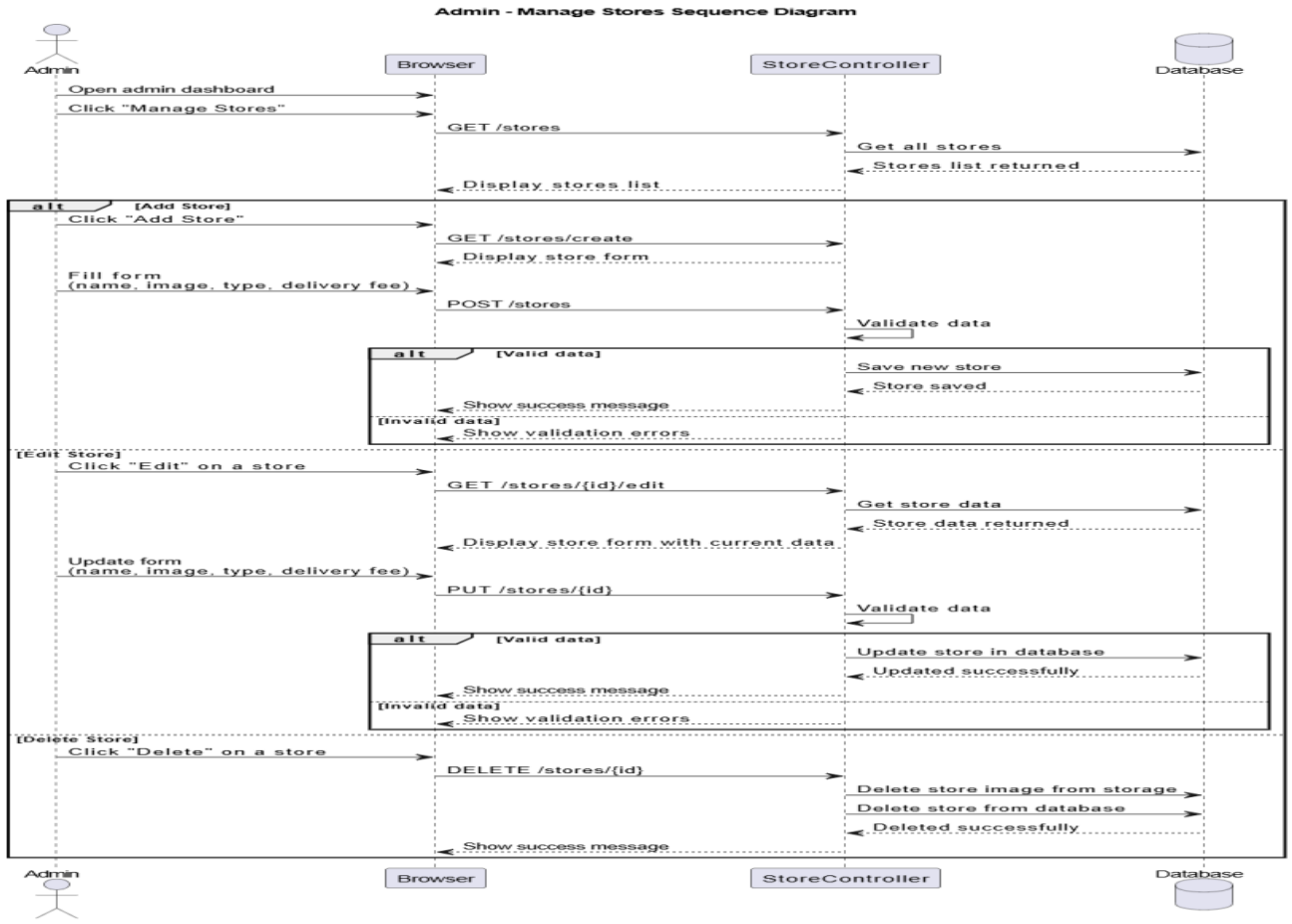

Search Stores by Name Sequence Diagram
User
Browser
StoreController
Database
Type store name in search bar
GET /stores?search_word=name&user_mode=1
Filter stores by name
alt
[Stores found]
Return matching stores
Display matching stores list
Click on a store
GET /stores/{id}
Get store details and products
Return store data
Display store page with products
[No stores found]
Empty result
Show "No results found" message

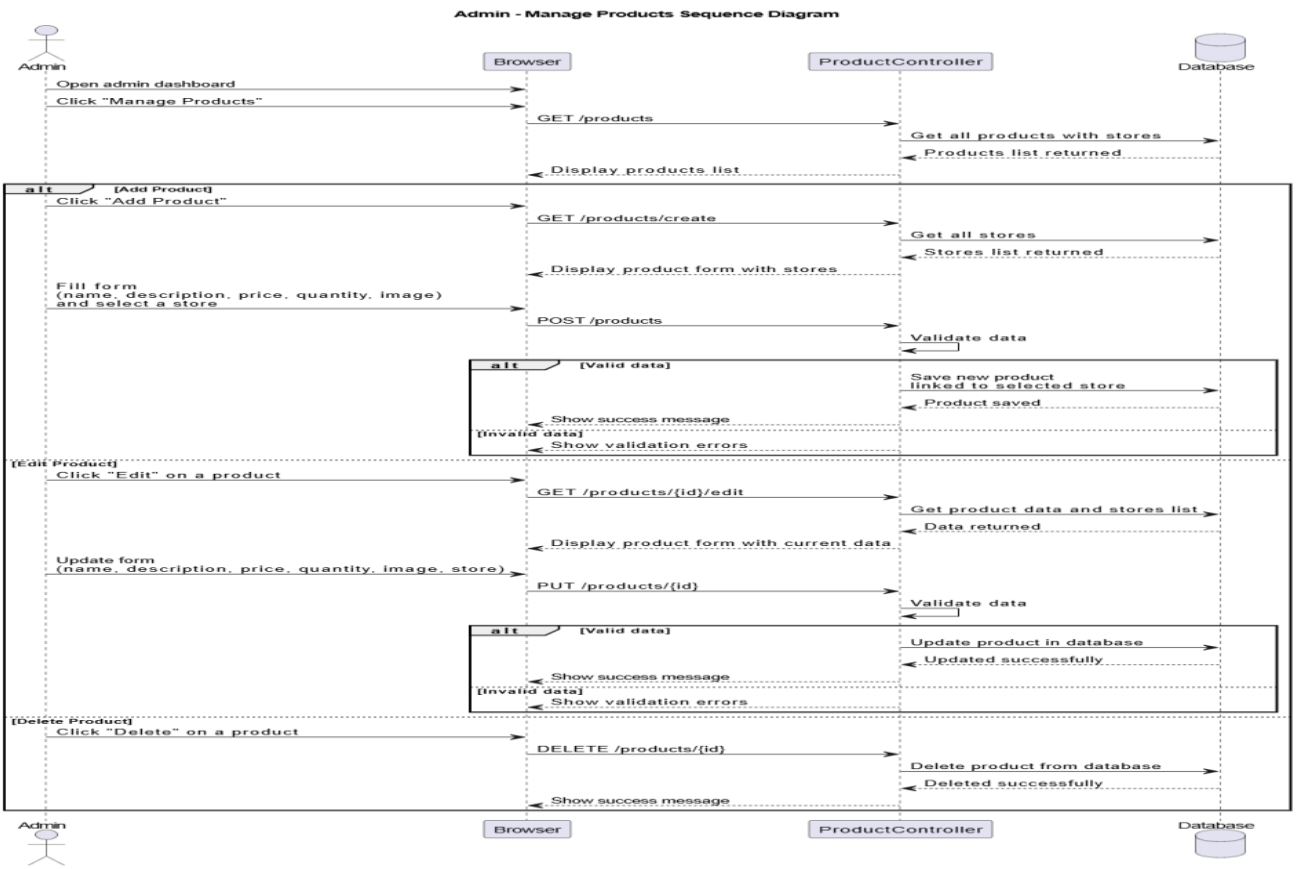

Place New Order & Checkout Sequence Diagram
User
Browser
CartController
Order
Database
Open cart
GET /cart
Get cart items with products and stores
Cart items returned
Calculate subtotal, delivery fee and discount
Display cart summary
Enter delivery address and pick location on map
POST /cart/checkout (address, latitude, longitude)
Validate input data
alt
[Valid data]
Get cart items
Items returned
alt
[Cart not empty]
Calculate total
Create new order
Order created
Create order items
Update user order count
Clear cart items
Done
Notify admin (NewOrderNotification)
Save notification
Show success message
[Cart is empty]
Show "Cart is empty" message
[Invalid data]
Show validation errors

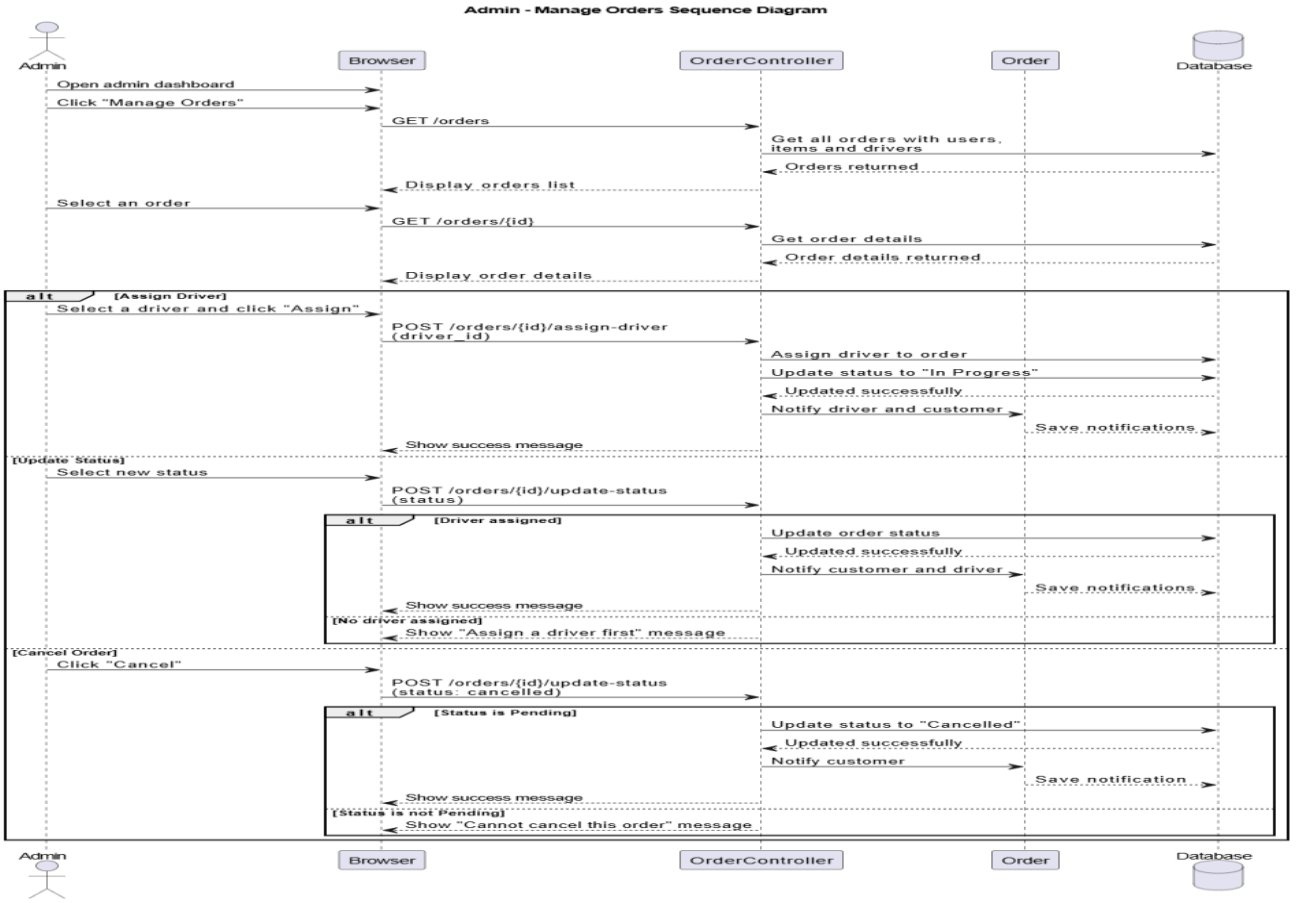

Track Order Status Sequence Diagram
User
Browser
HomeController
Database
Open home page
GET /home
Get user orders
Orders returned
Display latest order status with progress tracker
alt
[Status = Pending]
Show "Pending" status
Click "Cancel Order"
POST /orders/{id}/update-status (status: cancelled)
Update order status
Updated successfully
Show success message
[Status = In Progress]
Show "In Progress" status
Cancel option disabled
[Status = On the Way]
Show "On the Way" status
Display delivery location on map
[Status = Delivered]
Show "Delivered" status
[Status = Cancelled]
Show "Cancelled" status

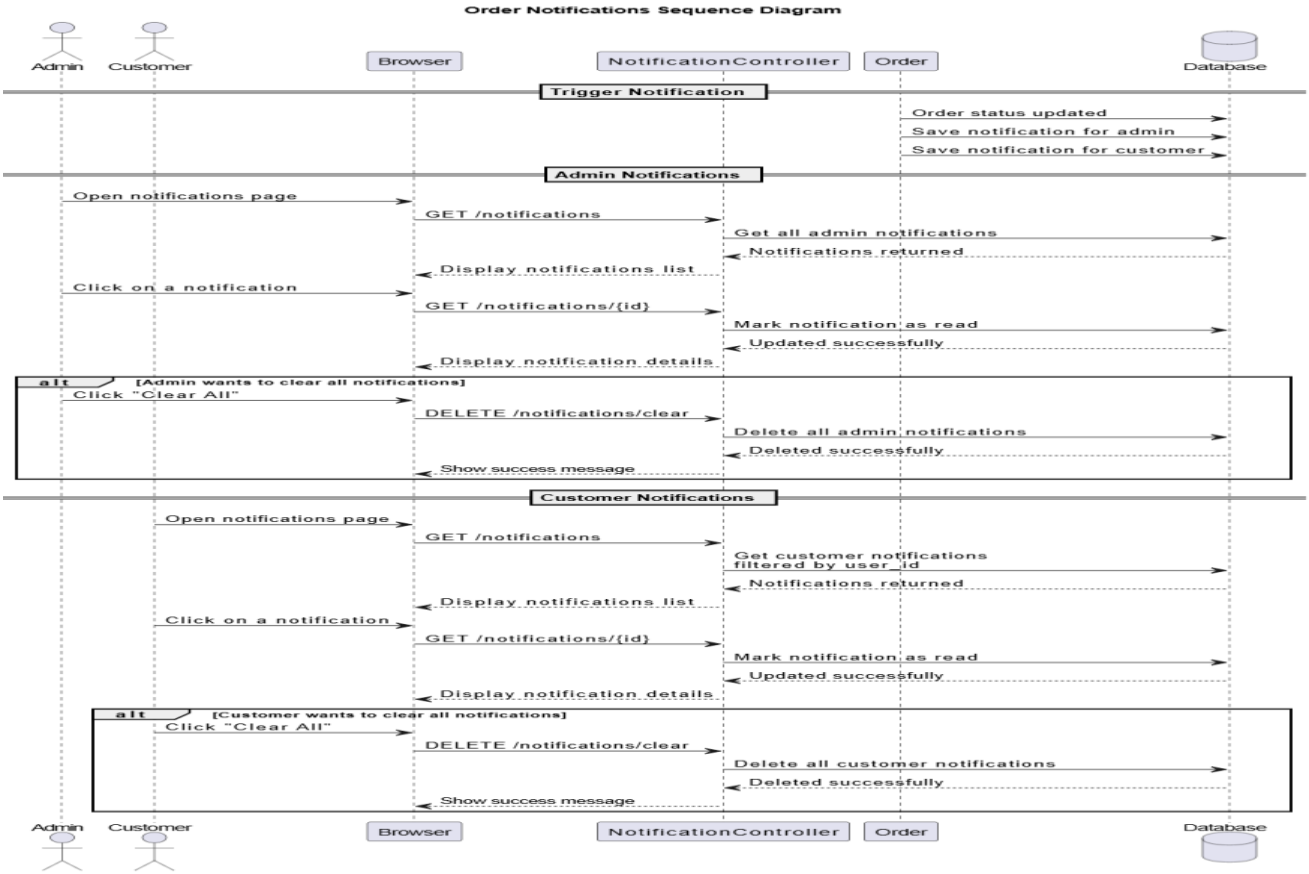

Driver - View Assigned Orders & Update Status Sequence Diagram
Driver
Browser
OrderController
Order
Database
Open driver dashboard
GET /orders
Get orders assigned to driver
Orders returned
Display assigned orders list
Select an order
GET /orders/{id}
Get order details
Order details returned
Display order details
alt
[Update to "On the Way"]
Click "On the Way"
POST /orders/{id}/update-status
(status: on_the_way)
Update order status
Updated successfully
Notify customer and admin
Save notifications
Show success message
[Update to "Delivered"]
Click "Delivered"
POST /orders/{id}/update-status
(status: delivered)
Update order status
Updated successfully
Notify customer and admin
Save notifications
Show success message
Driver
Browser
OrderController
Order
Database


Admin - Manage Users Sequence Diagram
Admin
Browser
UserController
Database
Open admin dashboard
Click "Manage Users"
GET /users
Get all users
Users list returned
Display users list
alt
[View User Info]
View user details
(name, email, role)
Display user information
[Delete User]
Click "Delete" on a user
GET /users/{id}/delete
Delete user from database
Deleted successfully
Show success message
Updated users list
Admin
Browser
UserController
Database


Admin - Manage Stores Sequence Diagram
Admin
Browser
StoreController
Database
Open admin dashboard
Click "Manage Stores"
GET /stores
Get all stores
Stores list returned
Display stores list
alt
[Add Store]
Click "Add Store"
GET /stores/create
Display store form
Fill form
(name, image, type, delivery fee)
POST /stores
Validate data
alt
[Valid data]
Save new store
Store saved
Show success message
[Invalid data]
Show validation errors
[Edit Store]
Click "Edit" on a store
GET /stores/{id}/edit
Get store data
Store data returned
Display store form with current data
Update form
(name, image, type, delivery fee)
PUT /stores/{id}
Validate data
alt
[Valid data]
Update store in database
Updated successfully
Show success message
[Invalid data]
Show validation errors
[Delete Store]
Click "Delete" on a store
DELETE /stores/{id}
Delete store image from storage
Delete store from database
Deleted successfully
Show success message


Admin - Manage Products Sequence Diagram
Admin
Browser
ProductController
Database
Open admin dashboard
Click "Manage Products"
GET /products
Get all products with stores
Products list returned
Display products list
alt
[Add Product]
Click "Add Product"
GET /products/create
Get all stores
Stores list returned
Display product form with stores
Fill form
(name, description, price, quantity, image)
and select a store
POST /products
Validate data
alt
[Valid data]
Save new product
linked to selected store
Product saved
Show success message
[Invalid data]
Show validation errors
[Edit Product]
Click "Edit" on a product
GET /products/{id}/edit
Get product data and stores list
Data returned
Display product form with current data
Update form
(name, description, price, quantity, image, store)
PUT /products/{id}
Validate data
alt
[Valid data]
Update product in database
Updated successfully
Show success message
[Invalid data]
Show validation errors
[Delete Product]
Click "Delete" on a product
DELETE /products/{id}
Delete product from database
Deleted successfully
Show success message


Admin - Manage Orders Sequence Diagram
Admin
Browser
OrderController
Order
Database
Open admin dashboard
Click "Manage Orders"
GET /orders
Get all orders with users, items and drivers
Orders returned
Display orders list
Select an order
GET /orders/{id}
Get order details
Order details returned
Display order details
alt
[Assign Driver]
Select a driver and click "Assign"
POST /orders/{id}/assign-driver (driver_id)
Assign driver to order
Update status to "In Progress"
Updated successfully
Notify driver and customer
Save notifications
Show success message
[Update Status]
Select new status
POST /orders/{id}/update-status (status)
alt
[Driver assigned]
Update order status
Updated successfully
Notify customer and driver
Save notifications
Show success message
[No driver assigned]
Show "Assign a driver first" message
[Cancel Order]
Click "Cancel"
POST /orders/{id}/update-status (status: cancelled)
alt
[Status is Pending]
Update status to "Cancelled"
Updated successfully
Notify customer
Save notification
Show success message
[Status is not Pending]
Show "Cannot cancel this order" message


Order Notifications Sequence Diagram
Admin
Customer
Browser
NotificationController
Order
Database
Trigger Notification
Order status updated
Save notification for admin
Save notification for customer
Admin Notifications
Open notifications page
GET /notifications
Get all admin notifications
Notifications returned
Display notifications list
Click on a notification
GET /notifications/{id}
Mark notification as read
Updated successfully
Display notification details
alt
[Admin wants to clear all notifications]
Click "Clear All"
DELETE /notifications/clear
Delete all admin notifications
Deleted successfully
Show success message
Customer Notifications
Open notifications page
GET /notifications
Get customer notifications filtered by user_id
Notifications returned
Display notifications list
Click on a notification
GET /notifications/{id}
Mark notification as read
Updated successfully
Display notification details
alt
[Customer wants to clear all notifications]
Click "Clear All"
DELETE /notifications/clear
Delete all customer notifications
Deleted successfully
Show success message

4-10 Modeling Based on Adopted Methodology

- In line with the Waterfall methodology adopted for this project, modeling was a structured and systematic process, focusing on comprehensively defining and freezing system functionalities from various perspectives prior to the implementation phase. This approach ensured that all requirements were fully captured, verified, and approved upfront, providing a clear and immutable blueprint for subsequent development stages.

- **Use Case Diagrams**: These diagrams were instrumental in capturing the functional requirements of the system by illustrating the interactions between actors (Administrator, Customer, Driver) and the system's core functionalities. Each use case was further detailed with a textual description outlining its flow of events, preconditions, and postconditions to establish a rigid specification baseline.
- **Activity Diagrams**: To visualize the dynamic behavior and workflow of critical processes, activity diagrams were employed. For instance, the 'Place Order' activity diagram detailed the sequence of actions from a customer selecting items to the order being confirmed and assigned to a driver, highlighting decision points and parallel activities to ensure unambiguous process design.
- **Sequence Diagrams**: These diagrams provided a time-ordered representation of interactions between objects within the system, particularly for complex processes like 'Order Placement' and 'Order Status Update'. They illustrated the messages passed between the user interface, controllers, models, and external services (e.g., notification system), clarifying the system's internal logic and communication flow before coding began.
- **Entity-Relationship Diagrams (ERD)**: An ERD was developed to model the database structure, illustrating the entities (e.g., Users, Stores, Products, Orders) and their relationships. This provided a clear blueprint for the relational database design, ensuring data integrity and efficient storage.

4-11 Requirements Analysis and Documentation

Requirements analysis involved a systematic process of refining the elicited requirements, resolving ambiguities, and prioritizing them based on business value and technical feasibility. The documentation phase ensured that all requirements were clearly articulated, traceable, and verifiable.

- **Functional Requirements (FRs):** These specify what the system must do. Each FR was documented with a unique identifier, a clear description, and acceptance

criteria. For example, **FR-01:** "The system shall allow customers to register and log in securely."

- **Non-Functional Requirements (NFRs):** These define the quality attributes of the system, such as performance, security, usability, and reliability. NFRs were documented to guide the system's design and implementation, ensuring it meets expected quality standards (e.g., the system shall respond to user requests within 3 seconds under normal load).
- **Requirements Specification Document (RSD):** All identified and analyzed requirements were formally documented in an **RSD**, serving as a comprehensive reference for development, testing, and future maintenance. This document ensured a shared understanding among all stakeholders.

4-12 Expanded Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) is a crucial tool for managing the project's complexity, ensuring that each requirement is linked to its corresponding design, implementation, and test case. This expanded RTM provides a clear audit trail, facilitating impact analysis for changes and verifying complete coverage.

Table 4.15 RTM

ID	Requirement Description	Design Module	Implementation File(s)	TC	Status
FR-01-1	Register and log in with three roles (Customer, Driver, Admin)	Authentication Module	RegisterController LoginController HomeController	TC-01 TC-02	Complete
FR-02-1	Personal account with user profile and delivery location	Profile Module	ProfileController CartController	TC-11 TC-08	Complete
FR-03-1	Browse stores based on their type (restaurant, cafe, supermarket)	Store Module	StoreController	TC-03	Complete
FR-04-1	Browse store products and add them to the shopping cart	Cart & Product Module	ProductController CartController	TC-05 TC-06 TC-07	Complete
FR-05-1	Search for stores by name	Store Module	StoreController	TC-04	Complete
FR-06-1	Place a new order and complete the payment process	Order Module	CartController	TC-08	Complete
FR-07-1	Track order status (pending, in progress, on the way, delivered, cancelled)	Order Module	HomeController OrderController	TC-09	Complete
FR-08-1	Driver views assigned orders and updates status to on the way or delivered	Driver Module	OrderController	TC-18	Complete
FR-09-1	Admin views list of all registered users	Admin Module	UserController	TC-12	Complete

FR-09-2	Admin views detailed information for each user	Admin Module	UserController	TC-12	Complete
FR-09-3	Admin deletes user accounts from the system	Admin Module	UserController	TC-12	Complete
FR-10-1	Admin adds new stores specifying name, type, and delivery fees	Store Module	StoreController	TC-12	Complete
FR-10-2	Admin modifies existing store information	Store Module	StoreController	TC-13	Complete
FR-10-3	Admin deletes stores from the system	Store Module	StoreController	TC-14	Complete
FR-11-1	Admin adds new products linked to a specific store	Product Module	ProductController	TC-15	Complete
FR-11-2	Admin modifies existing product information	Product Module	ProductController	TC-15	Complete
FR-11-3	Admin deletes products from the system	Product Module	ProductController	TC-15	Complete
FR-12-1	Admin views a list of all orders in the system	Order Module	OrderController	TC-17	Complete
FR-12-2	Admin modifies the status of an order	Order Module	OrderController	TC-17	Complete
FR-12-3	Admin assigns a specific driver to deliver a particular order	Order Module	OrderController	TC-16	Complete
FR-12-4	Admin cancels orders	Order Module	OrderController	TC-17	Complete
FR-13-1	Display order notifications for both the user and the admin	Notification Module	NotificationController OrderStatusNotification NewOrderNotification	TC-19	Complete

4-13 Initial Test Cases

Initial test cases were developed to verify that the implemented functionalities meet the specified requirements. These test cases cover critical user flows and administrative tasks, ensuring the system's correctness and reliability. Each test case includes a unique ID, a description, preconditions, steps, and expected results.

Table 4.16 Initial Test Cases

Test Case ID	Description	Preconditions	Test Steps	Expected Result
TC-01	User Registration	None	1. Navigate to registration page. 2. Enter valid details. 3. Click 'Register'.	User account created, redirected to login.
TC-02	Store Browsing	User logged in	1. Navigate to home page. 2. Browse categories.	List of stores displayed by category.
TC-03	Product Selection	User browsing store	1. Select a store. 2. View products. 3. Select a product.	Product details displayed.

TC-04	Add to Cart	Product selected	1. Click 'Add to Cart'.	Product added to cart, cart count updated.
TC-05	Place Order	Items in cart	1. Proceed to checkout. 2. Select delivery address. 3. Confirm order.	Order placed successfully, confirmation displayed.
TC-06	Admin Manage Stores	Admin logged in	1. Navigate to Admin Dashboard. 2. Select 'Manage Stores'. 3. Add/Edit/Delete a store.	Store details updated successfully.
TC-07	Admin Manage Products	Admin logged in	1. Navigate to Admin Dashboard. 2. Select 'Manage Products'. 3. Add/Edit/Delete a product.	Product details updated successfully.
TC-08	Driver Accept Order	Driver logged in, new order available	1. View new orders. 2. Accept an order.	Order status updated to 'Accepted', assigned to driver.
TC-09	Driver Update Status	Driver accepted order	1. Navigate to active order. 2. Update status (e.g., 'On the Way').	Order status updated, customer notified.
TC-10	Real-time Notifications	User performs action (e.g., order placed)	1. Place an order.	User receives real-time notification.
TC-11	Order History	User logged in, past orders exist	1. Navigate to 'Order History'.	List of past orders displayed.
TC-12	User Profile Management	User logged in	1. Navigate to 'Profile'. 2. Update details.	Profile details updated successfully.
TC-13	Search Functionality	User on home page	1. Enter search query. 2. Click 'Search'.	Relevant stores/products displayed.

Chapter 5

System Architectural Design

Chapter 5: System Architectural Design

This chapter outlines the high-level and detailed architectural design of the Delivery Application. It describes the system's structure, the chosen architectural patterns, database design, and the crucial aspects of security and permissions. The design aims to ensure scalability, maintainability, and robustness while adhering to best practices in software engineering.

5.1 High-Level Design

The high-level design provides an abstract view of the system, illustrating its main components and their interactions. This section covers the architectural block diagram, the chosen architectural pattern, and a textual explanation of the modular design.

5.1.1 Architectural Block Diagram

The architectural block diagram visually represents the major components of the Delivery Application and their interconnections. It highlights the logical separation of concerns and the flow of data between different parts of the system.

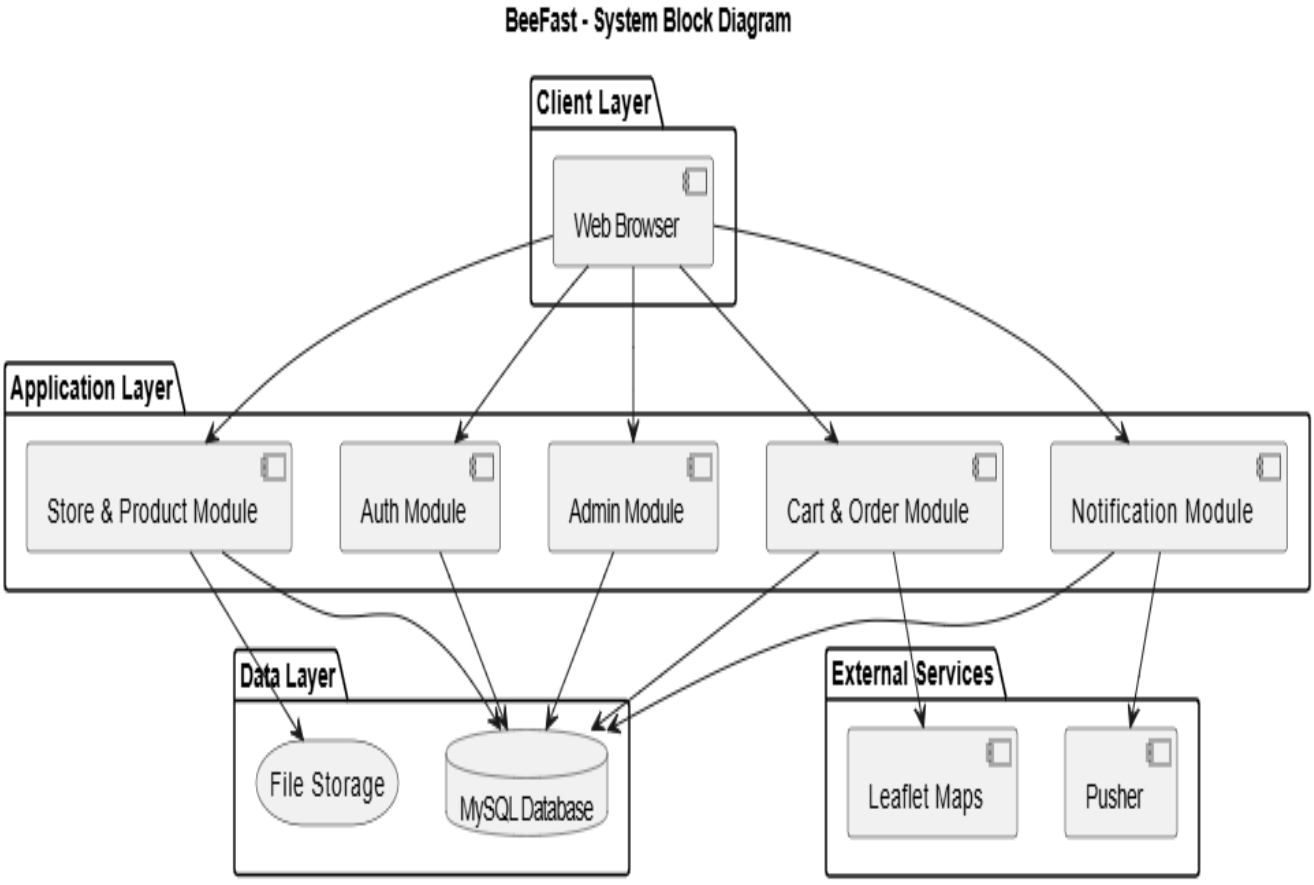


Figure 5.29 : system Block Diagram

Explanation of Components:

1. Client Layer

This layer represents the user's entry point to the system. All interactions occur through a standard web browser, where users can register, browse stores, place orders, and track deliveries.

2. Application Layer

Built on the Laravel framework, this layer contains all the business logic of the system, divided into five main modules:

- **Auth Module:** Handles user registration, login, and session management for all roles (Admin, Driver, User).
- **Store & Product Module:** Manages stores and their products, including browsing, searching, and image storage.
- **Cart & Order Module:** Handles adding products to cart, checkout, order creation, and delivery location tracking via maps.
- **Notification Module:** Sends real-time notifications to users, drivers, and admins upon order status changes.
- **Admin Module:** Provides full control over users, drivers, stores, products, and orders.

3. Data Layer

This layer is responsible for storing and managing all system data through two components:

- **MySQL Database:** Stores all structured data including users, stores, products, orders, carts, favorites, and notifications.
- **File Storage:** Stores uploaded images such as store and product images using Laravel's public disk.

4. External Services

These are third-party services integrated into the system to provide additional functionality:

- **Pusher:** A real-time messaging service used to instantly push order status notifications to users and admins without refreshing the page.
- **Leaflet Maps:** An open-source mapping library used to display the delivery location on an interactive map, allowing users to pick their exact delivery coordinates.

5.1.2 Used Architectural Design Pattern:

Model-View-Controller (MVC)

The Delivery Application adopts the Model-View-Controller (MVC) architectural pattern. This pattern is fundamental to the Laravel framework and provides a robust structure for separating concerns, promoting modularity, and enhancing maintainability and scalability.

- Model: Encapsulates the application's data, business logic, and rules. It interacts directly with the database to perform CRUD (Create, Read, Update, Delete) operations. Examples include User, Store, Product, and Order models.
- View: Responsible for presenting data to the user. It comprises the user interface elements (e.g., HTML templates, mobile UI components) that display information retrieved from the Model. Views are typically rendered by the Controller.

Controller: Acts as an intermediary, receiving user requests, processing input, interacting with the Model to fetch or update data, and then selecting the appropriate View to render the response. Controllers manage the application's flow and orchestrate interactions between Models and Views.

5.1.3 System Architecture Design

The **BeeFast** system follows a layered architecture divided into four main layers that work together to deliver a complete delivery platform.

At the front, the Client Layer consists of a web browser through which all users — guests, registered users, drivers, and admins — interact with the system. The browser sends requests to the Application Layer and receives responses to display to the user.

The Application Layer is the core of the system, built on the **Laravel** framework. It is divided into five modules. The Auth Module handles all authentication operations including registration, login, and session management. Once authenticated, users are directed to different interfaces based on their role. The Store & Product Module allows users to browse stores filtered by type and search by name, and view products within each store. The Cart & Order Module manages the entire purchase flow, from adding products to the cart, calculating totals with discounts and delivery fees, to placing the final order with a delivery location. The Notification Module monitors order status changes and dispatches alerts to the relevant parties. The Admin Module gives administrators full control over the platform, including managing users, stores, products, and orders, as well as assigning drivers to deliveries.

The Data Layer sits beneath the application and is responsible for persistence. The **MySQL** Database stores all structured data across nine tables covering users, stores, products, carts, orders, favorites, and notifications. File Storage handles uploaded media such as store and product images.

Finally, the system integrates two External Services. **Pusher** enables real-time communication, ensuring that order status notifications are delivered instantly without requiring the user to refresh the page. **Leaflet Maps** provides an interactive mapping interface that allows users to pinpoint their exact delivery location when placing an order.

5.2 Database Design

The database design is crucial for the efficient storage, retrieval, and management of the application's data. This section details the Entity-Relationship Diagram (ERD), the relational schema, and the constraints and keys implemented to ensure data integrity.

5.2.1 Entity-Relationship Diagram (ERD)

The **ERD** visually represents the entities within the Delivery Application, their attributes, and the relationships between them. It serves as a blueprint for the relational database structure.

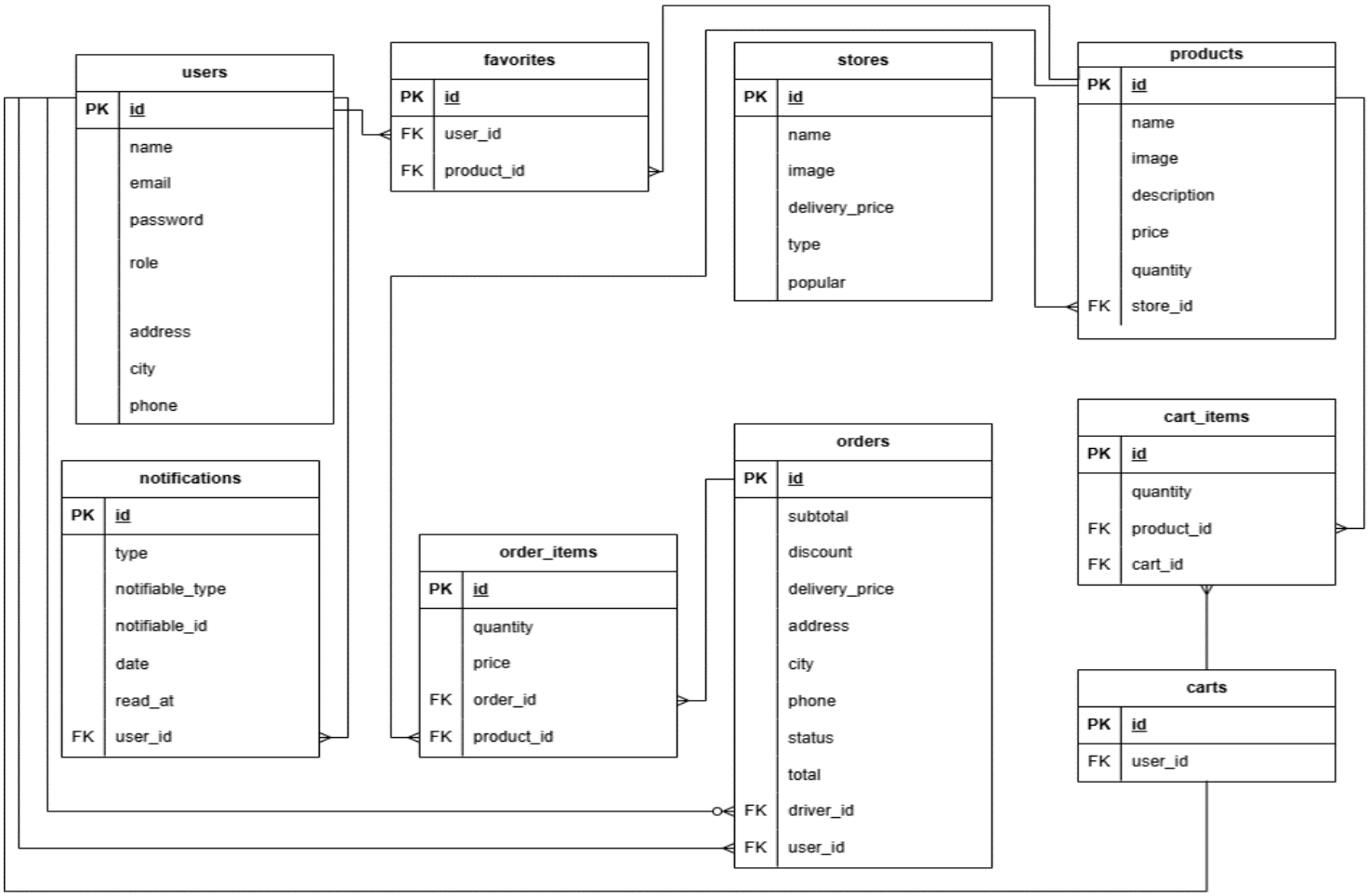


Figure 5.30 : Entity-Relationship Diagram

Entities, Attributes & Relationships

1. User

Represents any registered person in the system. A user can have one of three roles: admin, driver, or customer. Key attributes include id, name, email, password, role, address, city,

and phone. A user can place many orders, own one cart, save many favorite products, and receive many notifications.

2. Store

Represents a business registered on the platform such as a restaurant, cafe, or supermarket. Key attributes include id, name, image, delivery price, type, and popular flag. One store contains many products.

3. Product

Represents an item or dish offered for sale within a specific store. Key attributes include id, name, image, description, price, quantity, and store_id as a foreign key referencing the store it belongs to. A product can appear in many cart items, many order items, and many favorites.

4. Cart

Represents the shopping basket associated with a logged-in user. Key attributes include id and user_id as a foreign key referencing the user. One cart contains many cart items.

5. Cart Item

Represents a single product added to a cart along with the requested quantity. Key attributes include id, quantity, product_id, and cart_id as foreign keys referencing the product and cart. It acts as a junction between Cart and Product.

6. Order

Represents a completed purchase made by a customer. Key attributes include id, subtotal, discount, delivery price, address, city, phone, status, total, user_id as a foreign key referencing the customer, and driver_id as a foreign key referencing the assigned driver. One order contains many order items.

7. Order Item

Represents each product within a specific order, storing the quantity and price at the time of purchase. Key attributes include id, quantity, price, order_id, and product_id as foreign keys. It acts as a junction between Order and Product.

8. Favorite

Represents the relationship between a user and the products they have marked as favorites. Key attributes include id, user_id, and product_id as foreign keys. It acts as a junction between User and Product.

9. Notification

Represents system-generated alert messages sent automatically when an order status changes. Key attributes include id, type, notifiable_type, notifiable_id, date, read_at, and user_id as a foreign key referencing the recipient. Notifications are sent to the customer, driver, and admin.

Relationships Summary

Table 5.17 Relationships Summary

Relationship	Type
User → Cart	1 to 1
Cart → Cart Item	1 to N
Cart Item → Product	N to 1
User → Order (as customer)	1 to N
User → Order (as driver)	1 to N
Order → Order Item	1 to N
Order Item → Product	N to 1
Store → Product	1 to N
User → Favorite	1 to N
Favorite → Product	N to 1
User → Notification	1 to N

5.2.2 Relational Schema

Based on the **ERD**, the relational schema defines the tables, columns, and their data types in the **MySQL** database.

Table: users

Table 5.18 user

Column	Data Type
id	BIGINT UNSIGNED
name	VARCHAR(255)
email	VARCHAR(255)
password	VARCHAR(255)
role	ENUM('admin', 'driver', 'user')
address	VARCHAR(255)
city	VARCHAR(255)
phone	VARCHAR(20)
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: stores

Table 5.19 stores

Column	Data Type
id	BIGINT UNSIGNED
name	VARCHAR(255)
image	VARCHAR(255)
delivery_price	INT
type	ENUM('store', 'restaurant', 'cafe')
popular	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: products

Table 5.20 products

Column	Data Type
id	BIGINT UNSIGNED
name	VARCHAR(255)
image	VARCHAR(255)
description	TEXT
price	DECIMAL(10,2)
quantity	INT
store_id	BIGINT UNSIGNED
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: carts

Table 5.21 carts

Column	Data Type
id	BIGINT UNSIGNED
user_id	BIGINT UNSIGNED
status	ENUM('active', 'ordered')

created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: cart_items Table 5.22 : cart_items

Column	Data Type
id	BIGINT UNSIGNED
quantity	INT
product_id	BIGINT UNSIGNED
cart_id	BIGINT UNSIGNED
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: orders Table 5.23 : orders

Column	Data Type
id	BIGINT UNSIGNED
subtotal	DECIMAL(10,2)
discount	DECIMAL(10,2)
delivery_price	DECIMAL(10,2)
total	DECIMAL(10,2)
address	VARCHAR(255)
latitude	DECIMAL(10,7)
longitude	DECIMAL(10,7)
status	ENUM('pending','in_progress','on_the_way','delivered','cancelled')
driver_id	BIGINT UNSIGNED
user_id	BIGINT UNSIGNED
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: order_items Table 5.24 : order_items

Column	Data Type
id	BIGINT UNSIGNED
quantity	INT
price	DECIMAL(10,2)
order_id	BIGINT UNSIGNED
product_id	BIGINT UNSIGNED
created_at	TIMESTAMP
updated_at	TIMESTAMP

Table: favorites Table 5.25 : favorites

Column	Data Type
id	BIGINT UNSIGNED
user_id	BIGINT UNSIGNED

product_id	BIGINT UNSIGNED
created_at	TIMESTAMP

Table: notifications

Table 5.26 : notifications

Column	Data Type
id	UUID
type	VARCHAR(255)
notifiable_type	VARCHAR(255)
notifiable_id	BIGINT UNSIGNED
data	TEXT
read_at	TIMESTAMP
created_at	TIMESTAMP
updated_at	TIMESTAMP

5.2.3 Constraints and Keys

To maintain data integrity and enforce business rules, several constraints and keys are applied:

Primary Keys (PK)

Table 5.27 Primary Keys

Table	Primary Key
users	id
stores	id
products	id
carts	id
cart_items	id
orders	id
order_items	id
favorites	id
notifications	id (UUID)

Foreign Keys (FK)

Table 5.28 Foreign Keys

Column	References	On Delete
products.store_id	stores.id	CASCADE DELETE
carts.user_id	users.id	CASCADE DELETE
cart_items.cart_id	carts.id	CASCADE DELETE
cart_items.product_id	products.id	CASCADE DELETE
orders.user_id	users.id	CASCADE DELETE
orders.driver_id	users.id	SET NULL
order_items.order_id	orders.id	CASCADE DELETE
order_items.product_id	products.id	CASCADE DELETE
favorites.user_id	users.id	CASCADE DELETE

favorites.product_id	products.id	CASCADE DELETE

Unique Constraints

Table 5.29 Unique Constraints

Table	Unique Column(s)
users	email
favorites	(user_id, product_id)

Check Constraints

Table 5.30 Check Constraints

Table	Constraint
products	price > 0
products	quantity >= 0
stores	delivery_price >= 0
orders	subtotal >= 0
orders	discount >= 0
orders	total >= 0

These constraints collectively ensure the reliability and consistency of the data stored within the Delivery Application's database.

5.3 Security and Permissions Design

Security and permissions are paramount in the Delivery Application to protect sensitive data and ensure that users can only access authorized functionalities. The design incorporates robust mechanisms for authentication, authorization, and data protection.

5.3.1 Authentication

Authentication verifies the identity of a user. The system uses token-based authentication (e.g., Laravel Passport for API authentication or Laravel Sanctum for SPA authentication) to secure API endpoints.

- **User Registration:** New users (customers, drivers) can register by providing their email and password. Passwords are securely hashed and stored.
- **User Login:** Upon successful login, the system issues an access token. This token is then used for subsequent API requests to authenticate the user.
- **Password Management:** Features for password reset and update are implemented, ensuring secure handling of user credentials.

5.3.2 Authorization (Role-Based Access Control - RBAC)

Authorization determines what an authenticated user is allowed to do. The system implements **Role-Based Access Control (RBAC)**, where permissions are assigned to roles, and users are assigned to roles.

- **Roles:** Three primary roles are defined:
    - **Customer:** Can browse products, place orders, track orders, and manage their profile.
    - **Driver:** Can view available orders, accept/decline orders, update order status, and manage their profile.
    - **Admin:** Has full access to manage users, stores, products, orders, and system settings.
- **Permissions:** Specific actions within the application are associated with permissions (e.g., create-product, edit-store, view-all-orders).
- **Middleware:** Laravel's middleware is used to enforce authorization rules, ensuring that only users with the appropriate role and permissions can access specific routes or functionalities.

5.3.3 Data Protection

- **Data Encryption:** Sensitive data, such as passwords, are stored as hashed values. Communication between client applications and the Backend **API** is secured using **HTTPS** to encrypt data in transit.
- **Input Validation:** All user inputs are rigorously validated to prevent common web vulnerabilities such as **SQL** injection and Cross-Site Scripting **(XSS)**.
- **Error Handling:** The system implements comprehensive error handling to prevent sensitive information from being exposed in error messages.
- **Regular Security Audits:** The application will undergo regular security audits and penetration testing to identify and mitigate potential vulnerabilities.

By implementing these security and permission mechanisms, the Delivery Application aims to provide a secure and trustworthy environment for all its users and their data.

Chapter 6:
Execution Environment and Tools

Chapter 6: Execution Environment and Tools

Objective: To document all the tools, technologies, and environments utilized in the development and deployment of the Delivery Application, ensuring reproducibility and facilitating future maintenance or expansion.

6-1 Tools Used

6-1-1 Visual Studio Code Editor:

It is a free and open-source code editor (MIT License) developed by Microsoft, targeting various operating systems such as Windows, Linux, and macOS. It supports many languages and contains a large number of extensions that facilitate work, including code writing and analysis. In 2018, Visual Studio Code ranked first in the Stack Overflow survey as the most used development environment among developers.
Visual Studio Code supports many languages such as TypeScript, CSS, HTML, JavaScript, and many other languages. You can support more languages through the VS Code Marketplace, which provides documentation for dealing with other languages.

6-1-2 HTML (HyperText Markup Language):

It is a markup language used to create and design web pages and websites. This language is considered one of the oldest and most widely used languages in web page design. One of the main functions of HTML is to describe texts (giving them meaning and significance), so that the browser knows how to display them correctly and to divide the text into structures of headings and paragraphs, mark words as important, create lists, and many other things.
Elements in HTML are the basis for building HTML documents, through which we can add images and interactive objects such as forms or video and audio files. We can also create structured documents by using tags to declare paragraphs, headings, links, quotes, tables, and more. HTML can embed programs written in languages such as JavaScript to modify the behavior and content of web pages, and add CSS stylesheets to define the appearance and layout of the content.

6-1-3 CSS (Cascading Style Sheets):

CSS is a descriptive language that gives a website its beautiful appearance and unique design that distinguishes it from other sites. It is a friendly language to HTML, always accompanying it in designing and creating web pages. Both languages form the basis of every website; HTML is the basis of all web page elements, while CSS is the basis of the design and appearance of websites. Without it, websites would remain plain text on white backgrounds (text - images - links... etc.) without any design mentioned, and there was no plan for the launch of CSS.

Before the development of CSS in 1999 by the World Wide Web Consortium (W3C), web pages were very limited and simple in terms of appearance and function. Old browsers displayed web pages as pages consisting of black text on a white background only (text - images - links... etc.) without any design mentioned, and there was no plan for the launch of CSS.

6-1-4 JavaScript:

It is a high-level language that appeared in 1995, primarily used in web browsers to create more interactive pages and to provide web applications, including games. It is used by the majority of websites and was initially aimed at amateur programmers, but increasing interest attracted professional programmers with the spread of Ajax technology, which led to faster interaction between the server and the client. It is supported by almost all browsers without the need for external add-ons and is currently being developed by Mozilla, with several important libraries, most notably jQuery.

6-1-5 PHP:

It is a server-side programming language used to create interactive and advanced electronic websites. It is widely used, free to use, and easy to learn. It is also an open-source language that any developer can participate in developing and improving. Its files can contain PHP code, HTML code, CSS code, and JavaScript code. The codes are processed and their commands executed on the server, and the result is sent to the browser in HTML code format. PHP files end with the .php extension.

6-1-6 SQL:

Structured Query Language (SQL) is a specialized programming language for managing relational database data. It allows users to efficiently store, manipulate, and retrieve data in databases such as MySQL, SQL Server, Oracle, and more.
It is a non-procedural programming language, differing from conventional programming languages like C or Java, as non-procedural languages are specialized. Therefore, SQL is a language for interacting with and controlling relational databases by dealing with data structures and performing data entry, deletion, sorting, searching, filtering, modification, and so on.
Example of adding a table to the database:

CREATE TABLE users (name varchar(255), email varchar(255), password varchar(255));

Example of a query: **SELECT * FROM users**;

Example of adding a row to the users table:

INSERT INTO users (name, email, password)

VALUES (‘Ahmad’, ‘ahmad@mail.com’, ‘password’);

6-1-7 MySQL:

It is a relational database management system that relies on SQL. Its server is characterized by its high speed because it is a multi-threaded database server, which made querying the database very fast. It is also characterized by the ease of linking its tables to the user interface designed with programming languages.
With slight differences in instruction writing between it and SQL Server, which we have dealt with before, such as the instruction to change a table name and the available functions in each.

6-1-8 XAMPP:

It is a free and open-source application package that primarily includes an Apache HTTP server, MariaDB (MySQL), and an interpreter for applications written in PHP and Perl programming languages.
The program is used to make the device a mini-server with many features, and from it, we can create a local website on our device for programming, experimentation, and testing, or a public one for a limited number of visitors depending on the device\\\\\'s resources.

6-1-9 Bootstrap:

It is one of the most famous and powerful libraries that combine HTML – CSS – JS to build web pages, websites, and web applications. It is primarily useful for building responsive websites that adapt to different screen sizes, including mobile phones and tablets. It is a free and open-source library available to everyone on GitHub and was built by Twitter. It provides a responsive design compatible with all devices and browsers and offers a stable, reusable design.
In our website, we relied on a site that provides ready-made Bootstrap templates that can be used without any problems, as it allows the use of the Bootstrap library and its ready-made CSS services to make the site interactive and modern instead of writing traditional CSS code.

6-1-10 Laravel:

It is a framework for building web applications with an organized and elegant structure. It provides a structure and a starting point for creating web applications in PHP, and its installation relies on the Composer package management system, built on the famous MVC (Model-View-Controller) pattern. It features a clear file structure, immense power, and high security, allowing the developer to focus on precise programming details without worrying about repetitive features in almost every website, such as new user registration,

login, and other details whose repetition constitutes a burden on the developer.
In addition to a large and supportive community and detailed documentation of hundreds of pages, it is rare to find a problem without finding its solution on the first search page, making the Laravel environment the optimal choice for anyone who wants to experience creating web applications smoothly, comfortably, and without stress.

6.2 Development Environment and Project Structure

The project was developed on a local Windows environment using XAMPP as the local server stack. The project follows the standard Laravel directory structure:

Folder / File	Description
app/Http/Controllers	Contains all application controllers
app/Models	Contains Eloquent ORM models
resources/views	Contains Blade template files
routes/web.php	Defines all web routes
database/migrations	Contains database migration files
public/	Publicly accessible assets and uploads
storage/app/public	Stores uploaded images (stores, products)

6.3 Programming Languages and Frameworks

BeeFast is built using the following languages and frameworks:

Language / Framework	Usage
PHP 8.1 + Laravel 10	Backend logic, routing, authentication, ORM
JavaScript	Frontend interactivity and AJAX calls
HTML5 + Blade Templates	Server-side rendered views
CSS3 + Bootstrap 5	UI styling and responsive layout
SQL	Database queries via Eloquent ORM

6.4 Database Server and Operating System

The following database and operating system configurations were used:

Component	Details
Database Management System	MySQL 8.0
ORM	Laravel Eloquent ORM
Development OS	Windows 10/11
Local Server	XAMPP (Apache + MySQL)
Production OS	Ubuntu (Linux Server)

6.5 Libraries and Dependencies

All project dependencies are managed through Composer for PHP and npm for frontend assets. Key dependencies include:

Package	Version	Purpose
laravel/framework	^10.10	Core Laravel framework
laravel/ui	^4.6	Authentication scaffolding
laravel/sanctum	^3.3	API token authentication
pusher/pusher-php-server	^7.2	Real-time notifications via Pusher
guzzlehttp/guzzle	^7.2	HTTP client for external requests
fakerphp/faker	^1.9	Fake data generation for testing

6.6 Version Control and Source Management Tools

The project uses Git for version control with the following branching strategy:

Branch	Purpose
main	Stable production-ready code
dev	Active development and integration branch
feature/*	Individual feature branches merged into dev

The repository is hosted on GitHub, enabling collaboration, code review, and history tracking.

- **Git:** The primary version control system used for tracking all source code changes. It allowed for branching, merging, and reverting to previous states of the project.
- **GitHub:** Served as the remote repository hosting service for the **Git** repositories. GitHub facilitated team collaboration, code reviews, issue tracking, and continuous integration/deployment workflows.

6.8 Testing Environment

The **BeeFast** system was tested across the following environments:

Environment	Details
Local Development	XAMPP on Windows — localhost:8000 via php artisan serve
Browser Testing	Google Chrome, Mozilla Firefox, Microsoft Edge
Manual Testing	Functional testing performed manually on all user roles (Admin, Driver, User)
Database Testing	MySQL local instance with seeded test data using Laravel factories
Real-time Testing	Pusher sandbox account used to test live notifications
Map Testing	Leaflet Maps integrated with OpenStreetMap tiles for delivery location testing

Chapter 7:

Programming Implementation

Chapter 7: Programming Implementation

The goal of this chapter is to present code samples and the project structure without copying the full source code, which is placed in the appendix.

7-1 Project File Structure

The BeeFast project follows the standard Laravel directory structure. The main folders and files are organized as follows:



Figure 7.31 : Directory tree

7-2 Modules Implementation

The BeeFast system is divided into six main modules, each responsible for a specific set of functionalities.

7-2-1 Authentication Module

This module is built on top of laravel/ui and consists of the following components:

RegisterController handles new user registration by validating the input data and saving the new user to the database with a specified role (user or driver). After successful registration, the user is redirected to the home page.

LoginController validates the user credentials, starts the session, and redirects the user to the appropriate interface based on their role.

HomeController reads the authenticated user's role after login and directs them to the correct view — Admin Dashboard for admins, Driver Dashboard for drivers, and the Home page for regular users.

ProfileController displays the current user's personal information and allows them to update their name, email, phone, city, address, and password, with full input validation.

7-2-2 Store & Product Module

StoreController implements full CRUD operations for stores. It supports filtering by store type, searching by name, and pagination. It renders a different view for admins versus regular users based on the authenticated role.

ProductController implements full CRUD operations for products and links each product to a specific store. It supports image uploading and stores files in storage/app/public/products.

7-2-3 Cart & Order Module

This is the most complex module in the system and is divided into two parts:

CartController automatically creates a cart for each user upon their first product addition. It validates stock availability before adding any product. It calculates the subtotal, delivery fee as the sum of unique store delivery prices, and a cumulative discount. During checkout, it creates the order and its items, clears the cart, and sends a notification to the admin — all wrapped inside a DB::transaction to ensure data integrity.

OrderController retrieves orders based on the authenticated user's role — admins see all orders, drivers see only their assigned orders, and users see only their own orders. It allows admins to assign a driver to an order and update the order status. It prevents cancellation of any order that has moved beyond the Pending status.

7-2-4 Notification Module

NotificationController displays notifications filtered by role. Admins see all notifications, while users see only their own.

NewOrderNotification is triggered when a new order is placed and is sent to the admin.

OrderStatusNotification is triggered on every order status change and is sent to the relevant customer, admin, and driver.

All notifications are stored in the database using Laravel's built-in database notification channel.

7-2-5 Favorite Module

FavoriteController implements a toggle mechanism for adding and removing products from a user's favorites list. It uses a belongsToMany relationship between the User and Product models through the favorites pivot table. It returns a JSON response for AJAX requests and performs a redirect for standard web requests.

7-2-6 Admin Module

UserController displays the full list of registered users to admins only and allows them to delete any user account. It returns a JSON response for API requests and an HTML view for web requests.

HomeController serves the admin dashboard, which provides navigation links to manage users, stores, products, orders, and notifications.

7-3 APIs Implementation

Although BeeFast is primarily a server-side rendered web application built with Laravel Blade templates, several endpoints return JSON responses, making them function as lightweight API endpoints. These endpoints are used for real-time interactions on the frontend without requiring a full page reload.

Endpoint 1: Toggle Favorite

Field	Details
URL	POST /favorite/toggle/{id}
Controller	FavoriteController@toggle
Auth Required	Yes
Request Parameter	id — Product ID
Response	{ "status": true/false }

This endpoint checks whether the product is already in the user's favorites. If it is, the product is removed and the response returns false. If it is not, the product is added and the response returns true.

```
1 reference | 0 overrides
public function toggle(Request $request, $productId)
{
    $user = Auth::user();
    if (!Auth::check()) {
        return redirect()->route('login')->with('error', 'يجب تسجيل الدخول أولاً لإضافة منتجات إلى السلة.');
    }
    $product = Product::findOrFail($productId);

    $wasFavorite = $user->favorites()->where('product_id', $productId)->exists();

    if ($wasFavorite) {
        $user->favorites()->detach($productId);

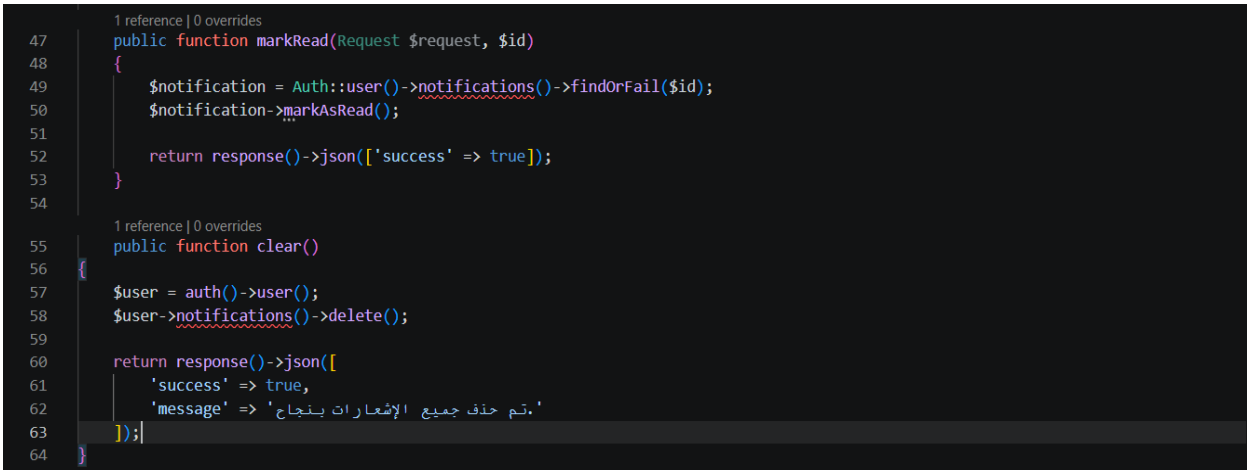
        session()->flash('success', 'تمت إزالة المنتج من المفضلة ❌');
        $status = false;
    } else {
        $user->favorites()->attach($productId);
        session()->flash('success', 'تمت إضافة المنتج إلى المفضلة ✅');
        $status = true;
    }


    if ($request->expectsJson()) {
        return response()->json(['status' => $status]);
    }
```

Endpoint 2: Mark Notification as Read

Field	Details
URL	POST /notifications/{id}/mark-read
Controller	NotificationController@markRead
Auth Required	Yes
Request Parameter	id — Notification ID
Response	{ "success": true }

This endpoint finds the notification belonging to the authenticated user and marks it as read by setting the read_at timestamp.



```
1 reference | 0 overrides
public function markRead(Request $request, $id)
{
    $notification = Auth::user()->notifications()->findOrFail($id);
    $notification->markAsRead();

    return response()->json(['success' => true]);
}

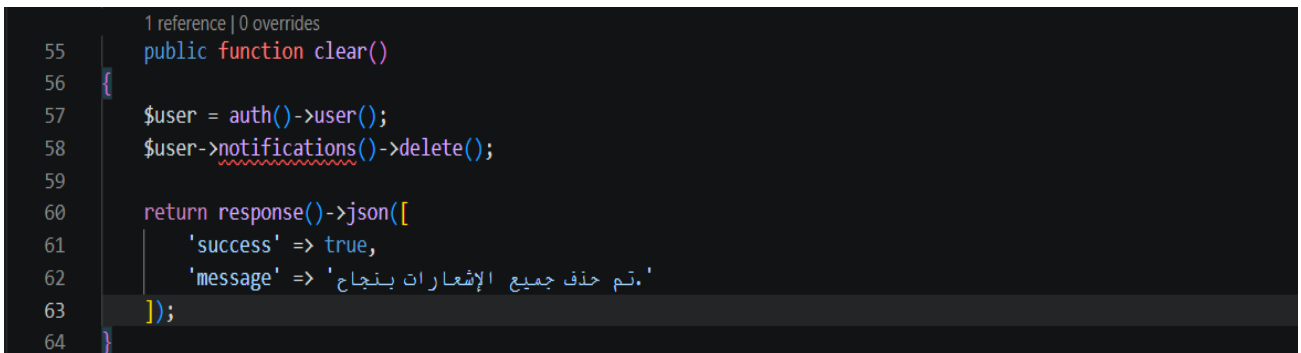
1 reference | 0 overrides
public function clear()
{
    $user = auth()->user();
    $user->notifications()->delete();

    return response()->json([
        'success' => true,
        'message' => 'تم حذف جميع الإشعارات بنجاح.'
    ]);
}
```

Endpoint 3: Clear All Notifications

Field	Details
URL	DELETE /notifications/clear
Controller	NotificationController@clear
Auth Required	Yes
Request Parameter	None
Response	{ "success": true, "message": "..." }

This endpoint deletes all notifications belonging to the authenticated user from the database.



```
1 reference | 0 overrides
public function clear()
{
    $user = auth()->user();
    $user->notifications()->delete();

    return response()->json([
        'success' => true,
        'message' => 'تم حذف جميع الإشعارات بنجاح.'
    ]);
}
```

7-4 Data Access Layer

BeeFast uses Laravel's Eloquent ORM as the primary data access layer. Eloquent provides an elegant and expressive way to interact with the database by representing each database table as a model class. All database queries in the system are performed through Eloquent relationships and query builder methods, eliminating the need for raw SQL queries in most cases.

Eloquent Relationships Used in the System

Table 7.31 Eloquent Relationships

Model	Relationship	Related Model	Method
User	hasMany	Order	orders()
User	hasMany	Order (as driver)	deliveries()
User	hasOne	Cart	cart()
User	belongsToMany	Product	favorites()
Store	hasMany	Product	products()
Store	hasMany	Order	orders()
Product	belongsTo	Store	store()
Product	hasMany	CartItem	cartItems()
Product	hasMany	OrderItem	orderItems()
Product	belongsToMany	User	favoritedBy()
Cart	belongsTo	User	user()
Cart	hasMany	CartItem	items()
CartItem	belongsTo	Cart	cart()
CartItem	belongsTo	Product	product()
Order	belongsTo	User	user()
Order	belongsTo	User (driver)	driver()
Order	hasMany	OrderItem	items()
OrderItem	belongsTo	Order	order()
OrderItem	belongsTo	Product	product()

Common Query Patterns Used

1. Eager Loading to avoid N+1 problem: The system loads related models in a single query using the with() method, for example loading orders with their users, items, products, and drivers in one call.

```
1 reference | 0 overrides
public function index()
{
    $user = auth()->user();

    if ($user->role == 'admin') {
        $orders = Order::with(['user','items.product','driver'])->latest()->get();
    } elseif ($user->role == 'driver') {
        $orders = Order::with(['user','items.product'])
                    ->where('driver_id', $user->id)->latest()->get();
    } else {
        $orders = Order::with(['items.product','driver'])
                    ->where('user_id', $user->id)->latest()->get();
    }
```

2. Filtering and ordering: Stores are filtered by type and popularity and paginated using Laravel's built-in paginate() method.

```php
$query = Store::query();

if($request->search_word)
    $query->where('name', 'like', '%'.$request->search_word.'%');

if($request->type)
    $query->where('type', $request->type);

if($request->popular == 1)
    $query->where('popular', 1);

$stores = $query->paginate($request->count ?? 10);

$noResults = false;
if($request->search_word && $stores->total() == 0) {
    $noResults = true;
}
```

3. firstOrCreate: Used to automatically create a cart or user order count record if it does not already exist.

```php
$cart = Cart::firstOrCreate(
    ['user_id' => Auth::id()]
);

$items = $cart->items()->with('product.store')->get();

// Calculate subtotal using product price
$subtotal = $items->sum(function($item) {
    return $item->product->price * $item->quantity;
});

// Get unique stores from cart items
$stores = $items->pluck('product.store')->unique('id')->filter();

// Calculate delivery fee as sum of unique store delivery prices
$delivery_fee = $stores->sum(function($store) {
    return $store->delivery_price;
});

// Calculate discount based on order count
$userOrderCount = UserOrderCount::firstOrCreate(
    ['user_id' => Auth::id()]
);
```

4. whereHas: Used to filter cart items by verifying they belong to the authenticated user's cart.

5. DB::transaction: All checkout operations are wrapped in a database transaction to ensure that if any step fails, all changes are rolled back.

```php
DB::transaction(function() use ($items, $cart, $request) {
    $subtotal = $items->sum(function($item) {
        return $item->product->price * $item->quantity;
    });
```

7-5 Business Logic Layer

The business logic layer in BeeFast is implemented within the controllers and models. It contains all the rules, calculations, and conditions that govern how the system behaves. The following sections describe the most important business logic components in the system.

BeeFast

1. Cumulative Discount Algorithm

BeeFast rewards loyal customers by applying a cumulative discount based on the number of previous orders. Each completed order gives the user a 5% discount on their next order. The maximum discount is capped at 20% of the subtotal regardless of the number of orders.

Orders Completed	Discount Applied
0	0%
1	5%
2	10%
3	15%
4+	20% (maximum)

```
181         $userOrderCount = UserOrderCount::firstOrCreate(['user_id' => Auth::id()]);
182         $discount = 0.05 * $userOrderCount->order_count * $subtotal;
183         $discount = min($discount, $subtotal * 0.2);
184
185         $total = $subtotal + $delivery_fee - $discount;
```

2. Delivery Fee Calculation

The delivery fee is not fixed. It is calculated as the sum of delivery prices of all unique stores present in the cart. This allows users to order from multiple stores in a single checkout while paying the correct delivery fee for each store.

```
175         $uniqueStores = $items->pluck('product.store')->unique('id')->filter();
176         $delivery_fee = $uniqueStores->sum(function($store) {
177             return $store->delivery_price;
178         });
179
```

3. Stock Management

The system automatically manages product stock across three operations. When a product is added to the cart, the stock is decreased by the requested quantity. When the cart item quantity is updated, the stock is adjusted based on the difference between the new and old quantity. When an item is removed from the cart, the full quantity is restored back to the product stock.

```
83          $product->quantity -= $requestedQty;
84          $product->save();
85
```

```
101         $difference = $newQty - $item->quantity;
102
103         if ($difference > 0) {
104             if ($product->quantity < $difference) {
105                 return redirect()->back()->with('error', 'لا توجد كمية كافية في المخزون لزيادة الكمية.');
106             }
107             $product->quantity -= $difference;
108         } else {
109             $product->quantity += abs($difference);
110         }
111
112         $product->save();
113         $item->quantity = $newQty;
114         $item->save();
```

```
135         $product->quantity += $item->quantity;
136         $product->save();
137
138         $item->delete();
```

4. Order Status Transition Rules

The system enforces strict rules on order status transitions to prevent invalid state changes:

- An order can only be cancelled if its current status is Pending.
- Changing the status to In Progress requires a driver to be assigned first.
- A driver cannot cancel an order — only admins can perform cancellations.

```
1 reference | 0 overrides
39  public function updateStatus(Request $request, $id)
40  {
41      $order = Order::findOrFail($id);
42      $status = $request->status;
43
44
45      if($status == 'in_progress' && !$order->driver_id){
46          return back()->with('error','يجب تعيين سائق قبل قبول الطلب');
47      }
48
49      if (auth()->user()->role == 'driver' && $status == 'cancelled') {
50          return back()->with('error','السائق لا يمكنه إلغاء الطلب كليًا');
51      }
52
53
54      if($status == 'cancelled' && $order->status != 'pending'){
55          return back()->with('error','لا يمكن إلغاء الطلب بعد قبوله');
56      }
57
58      $order->status = $status;
59      $order->save();
60
61      $order->notifyStatus($status);
62
63      return back()->with('success','تم تحديث حالة الطلب');
64  }
```

- When a driver is assigned, they receive a special notification informing them of the assignment.

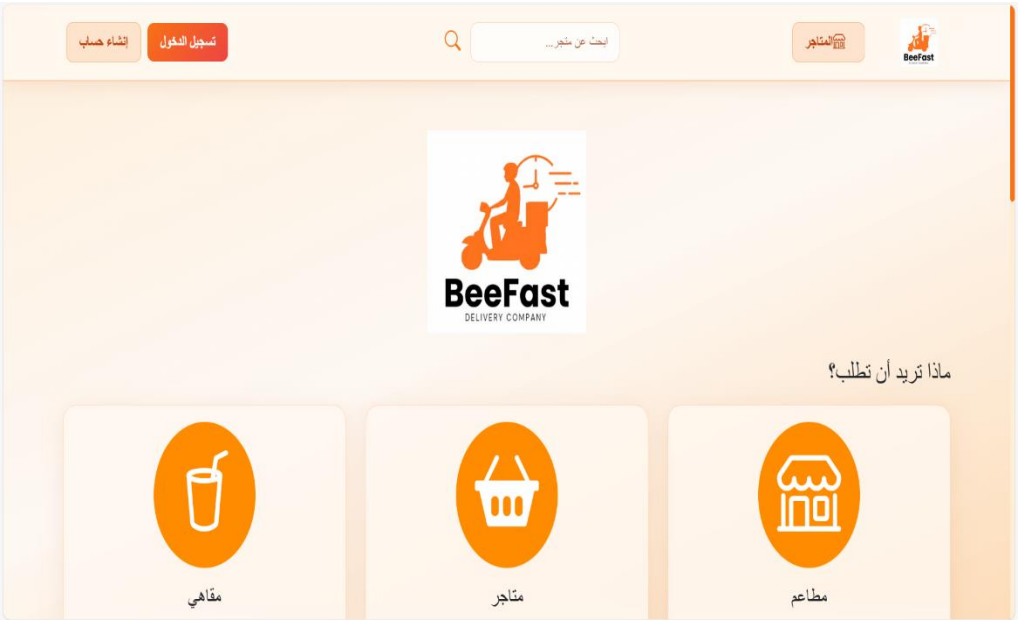
```
2 references | 0 overrides
21  public function notifyStatus($status)
22  {
23      $data = [
24          'order_id' => $this->id,
25          'status'   => $status,
26          'user_name'=> $this->user->name,
27          'total'    => $this->total,
28      ];
29
30
31      $admins = User::where('role', 'admin')->get();
32      foreach ($admins as $admin) {
33          $admin->notify(new OrderStatusNotification($data));
34      }
35
36      if ($this->user) {
37          $this->user->notify(new OrderStatusNotification($data));
38      }
39
40      if ($status === 'assigned' && $this->driver) {
41          $driverData = $data;
42          $driverData['driver_assigned'] = true;
43          $this->driver->notify(new OrderStatusNotification($driverData));
44      }
45  }
```

7-6 UI Implementation

The user interface of BeeFast is built using Laravel Blade templates combined with Bootstrap 5 for responsive layout and styling. The UI is designed to serve three different user roles — Admin, Driver, and User — each with a dedicated interface. The following sections describe the main pages of the system.

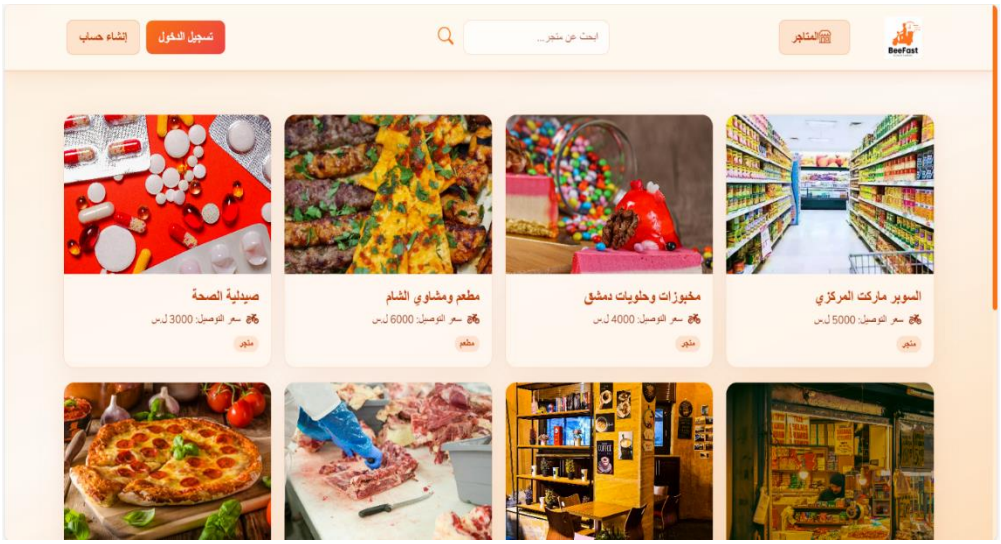
1. Welcome Page

The welcome page is the entry point for all visitors. It displays the most popular stores categorized by type (Restaurants, Cafes, and Stores). Visitors can browse stores and view products without logging in. The page also includes a navigation bar with search functionality, login, and register buttons.



2. Store Listing Page

This page displays all available stores with filtering by type and search by name. Each store card shows the store name, image, delivery price, and type. Users can click on any store to view its products.



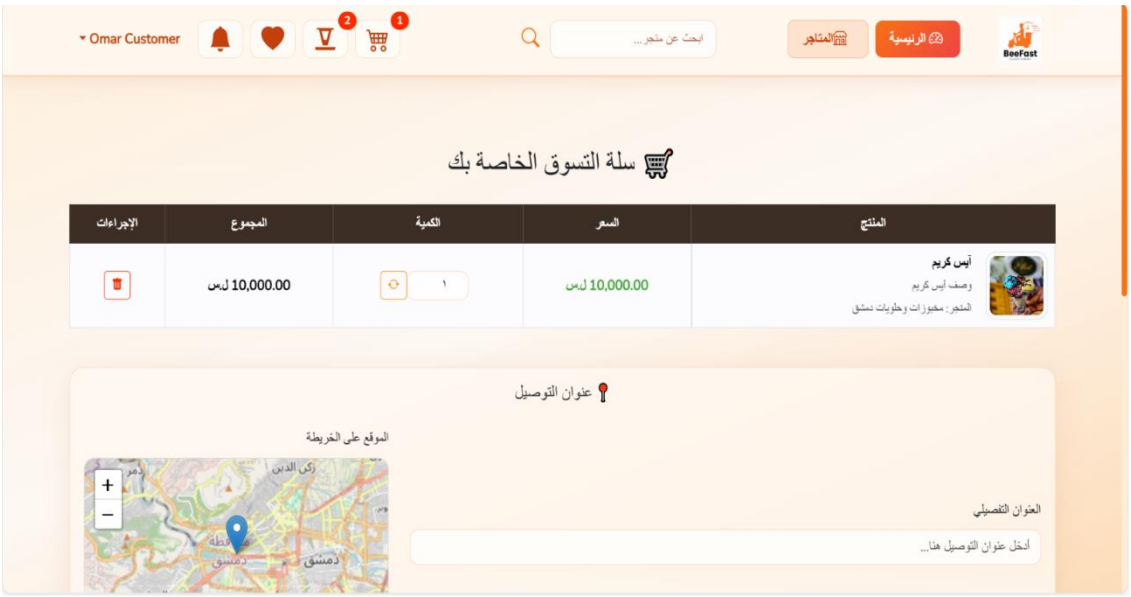
3. Store Products Page

This page displays all products available within a specific store. Each product card shows the name, image, description, price, and available quantity. Logged-in users can add products to the cart or save them to favorites directly from this page.



4. Cart Page

The cart page displays all items added by the user along with the calculated subtotal, delivery fee, discount, and total. It includes an interactive map powered by Leaflet.js that allows the user to pick their exact delivery location before confirming the order.



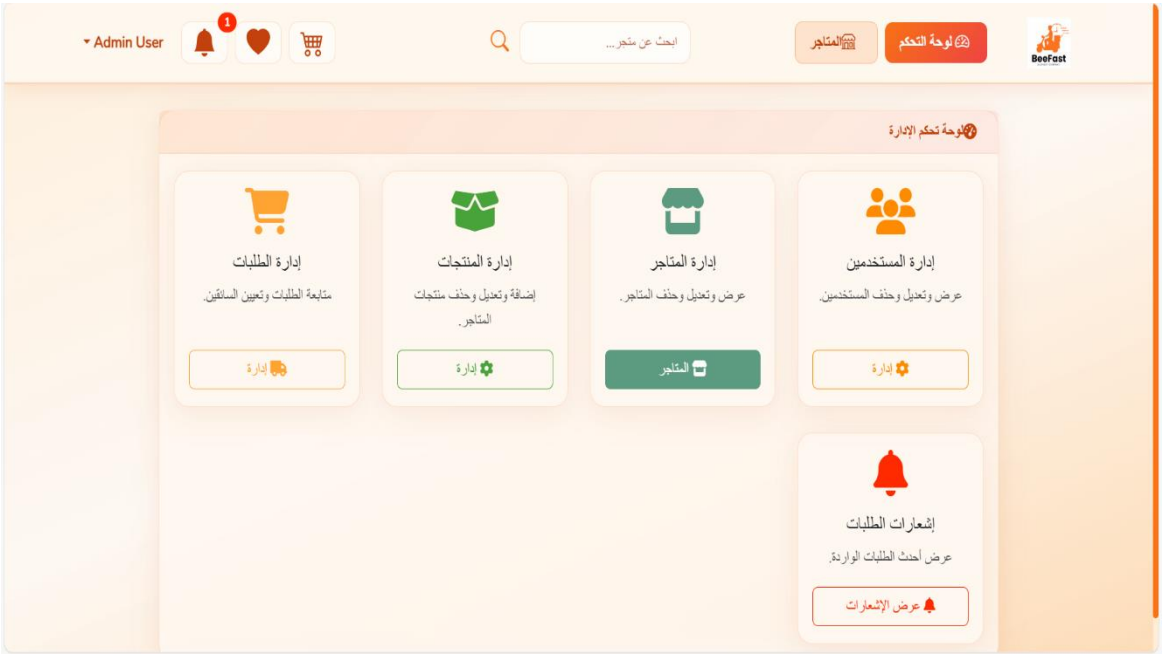
5. Home Page (User)

The home page for logged-in users displays the status of their latest order using a visual progress tracker with four steps: Pending, In Progress, On the Way, and Delivered. It also shows the delivery location on an interactive map and lists all previous orders with their statuses.



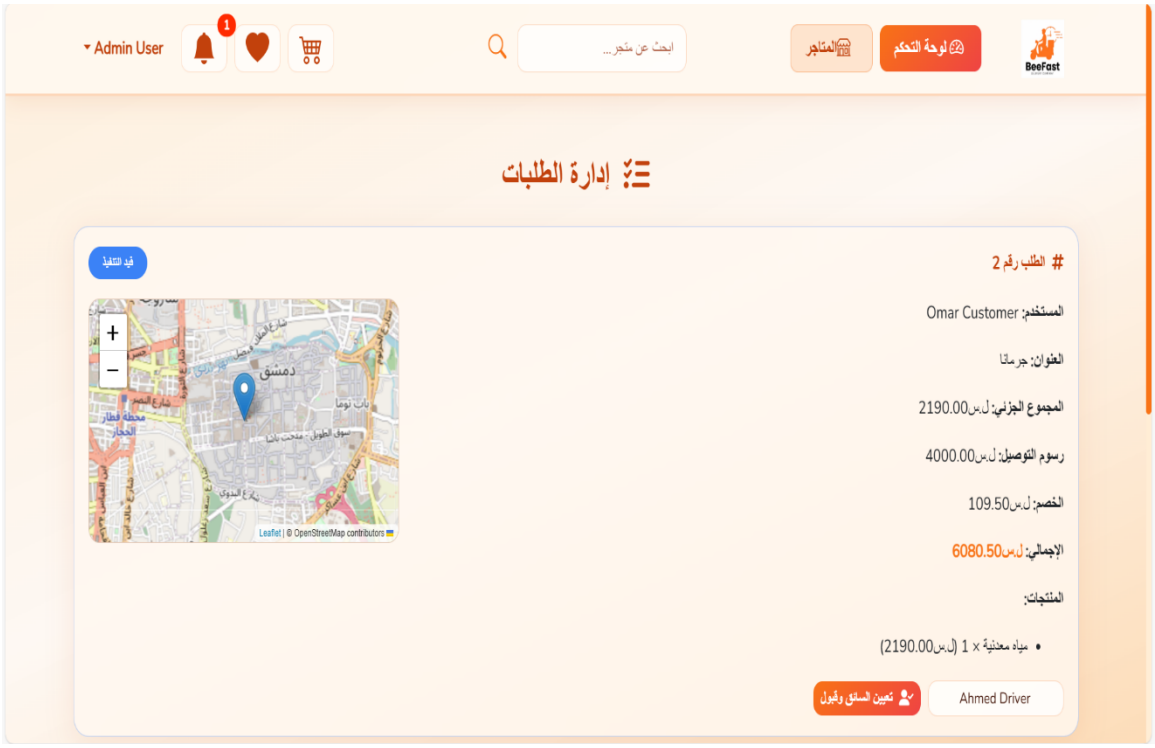
6. Admin Dashboard

The admin dashboard provides a centralized control panel with quick access cards for managing users, stores, products, orders, and notifications. Each card displays a descriptive icon, a short description, and a navigation button.



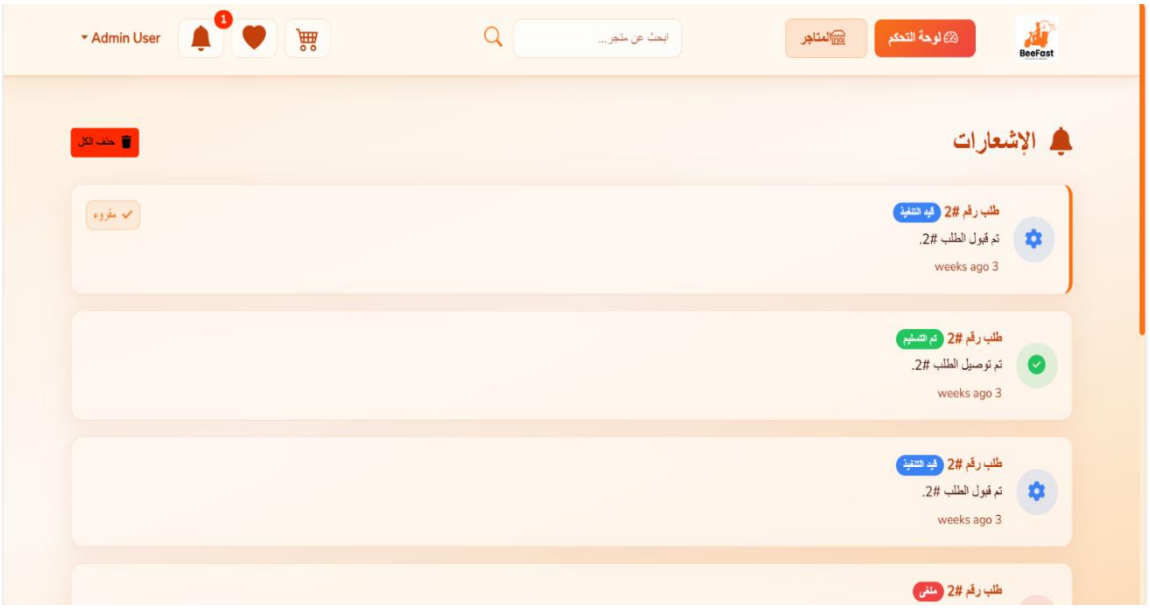
7. Orders Management Page (Admin)

This page displays all orders in the system with their details including customer name, total, status, and assigned driver. Admins can assign a driver, update the order status, or cancel an order directly from this page.



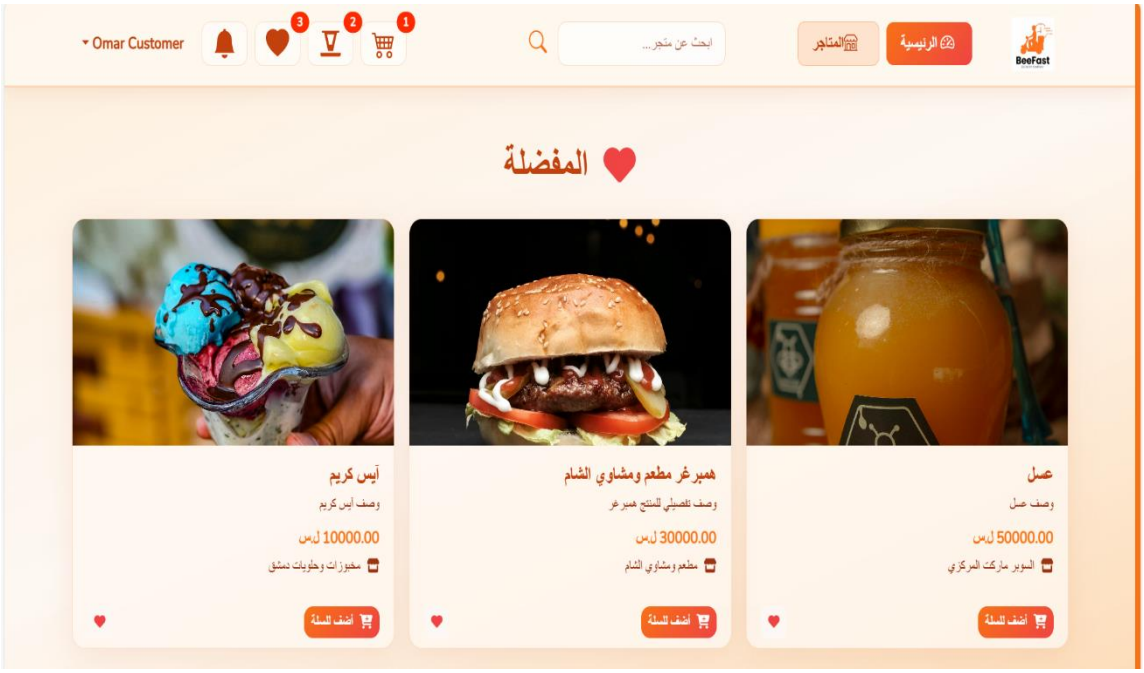
8. Notifications Page

The notifications page displays all system-generated notifications for the authenticated user. Each notification shows the order number, status change message, and timestamp. Users can mark notifications as read or clear all notifications at once.



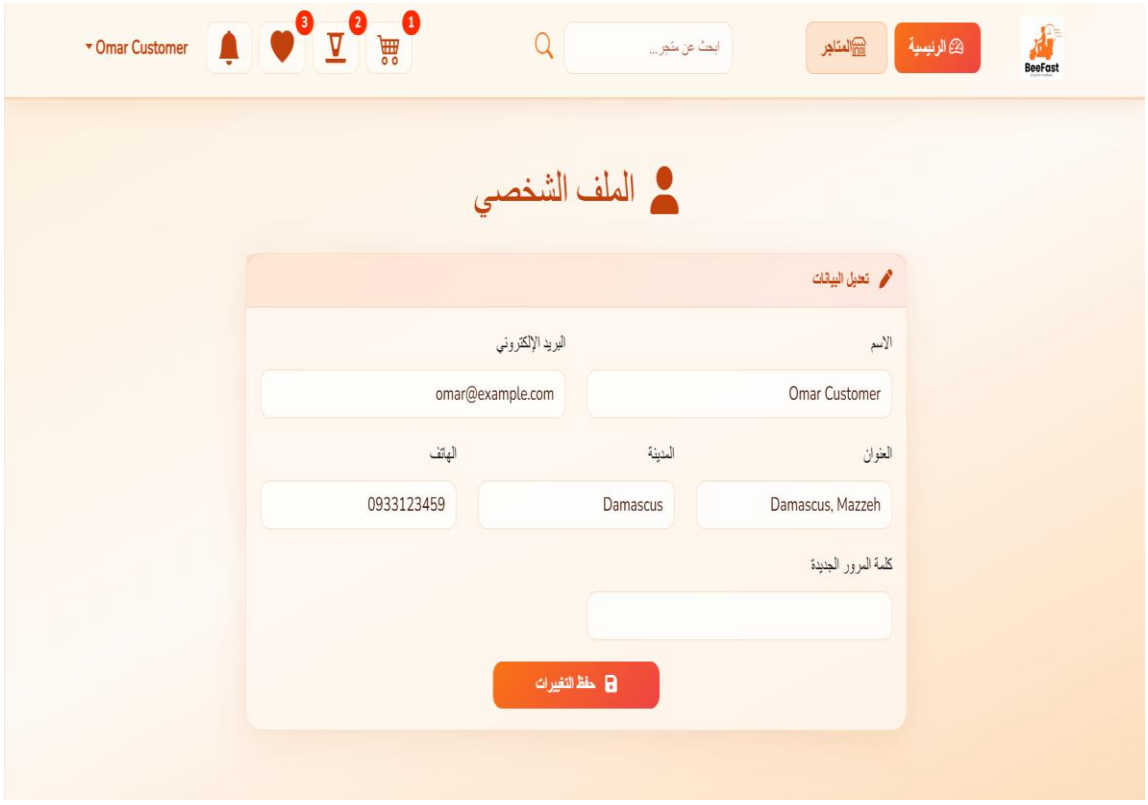
9. Favorites Page

The favorites page displays all products that the user has saved to their favorites list. Each product card shows the product name, image, price, and the store it belongs to.



10. Profile Page

The profile page allows the authenticated user to view and update their personal information including name, email, phone, city, address, and password.



7-7 Smart Algorithms Implementation

BeeFast is a delivery platform that focuses on core e-commerce functionality rather than artificial intelligence or machine learning. Therefore, the system does not implement any AI or ML algorithms. However, the system includes several intelligent behavioral mechanisms that automate key processes and enhance the user experience.

The most notable of these mechanisms is the cumulative discount algorithm, which was described in detail in section 7-5. This algorithm automatically calculates and applies a loyalty-based discount for returning customers based on their order history, rewarding them with an increasing discount rate up to a maximum of 20%.

In addition, the system employs an automated real-time notification mechanism powered by Laravel's database notification channel and Pusher, which intelligently routes notifications to the correct recipients based on the order status change and the roles of the involved parties.

Chapter 8:

Testing and Evaluation

Chapter 8: Testing and Evaluation

The goal of this chapter is to prove that the system works as required and with acceptable quality.

8-1 Test Plan

The testing strategy for BeeFast is based on manual functional testing performed across all user roles — Admin, Driver, and User. The goal of the test plan is to verify that all system features work as expected and that each user role can perform their designated tasks without errors.

Field	Details
Testing Type	Manual Functional Testing
Testing Environment	Local development server (localhost:8000)
Browser Used	Google Chrome / Microsoft Edge
Roles Tested	Admin, Driver, User
Testing Period	April 2026
Tools Used	Web Browser, phpMyAdmin, Laravel Tinker

8-2 Types of Testing

The following types of testing were performed on the BeeFast system:

8.2.1 Unit Testing

Each module was tested individually to verify that its core logic works correctly. This includes the discount calculation algorithm, stock management logic, and order status transition rules.

8.2.2 Integration Testing

The interaction between modules was tested to ensure they work correctly together. For example, the cart module was tested with the order module to verify that checkout creates the correct order items and clears the cart properly.

8.2.3 System Testing

The complete system was tested end-to-end across all user roles to verify that all features work together as a whole. This includes the full order flow from browsing stores to placing an order and tracking its status.

8.2.4 Acceptance Testing

The system was tested against the original requirements to verify that all defined functional requirements are met. Each use case was executed manually and the results were recorded.

8.2.5 Usability Testing

The system interface was evaluated for ease of use across all roles. The navigation flow, form validation messages, and error handling were tested to ensure a smooth user experience.

8.2.6 Compatibility Testing

The system was tested on multiple browsers including Google Chrome and Microsoft Edge to ensure consistent behavior and appearance across different environments.

8-3 Test Case Diagrams

8-3-1 Test Cases

Table 8.32 Test Cases

TC ID	Test Case	Actor	Expected Result	Actual Result	Status
TC-01	User Registration	Guest	Account created and redirected to home	As expected	Pass
TC-02	User Login	Guest	Redirected based on role	As expected	Pass
TC-03	Browse Stores by Type	Guest	Filtered stores displayed	As expected	Pass
TC-04	Search Store by Name	Guest	Matching stores displayed	As expected	Pass
TC-05	Add Product to Cart	User	Product added and stock decreased	As expected	Pass
TC-06	Update Cart Item	User	Cart updated and stock adjusted	As expected	Pass
TC-07	Remove Cart Item	User	Item removed and stock restored	As expected	Pass
TC-08	Place Order	User	Order created and cart cleared	As expected	Pass
TC-09	Cancel Order	User	Order status changed to cancelled	As expected	Pass
TC-10	Add to Favorites	User	Product added to favorites	As expected	Pass
TC-11	Update Profile	User	Profile updated successfully	As expected	Pass

TC-12	Admin Add Store	Admin	Store created and listed	As expected	Pass
TC-13	Admin Edit Store	Admin	Store updated successfully	As expected	Pass
TC-14	Admin Delete Store	Admin	Store removed from list	As expected	Pass
TC-15	Admin Add Product	Admin	Product created and linked to store	As expected	Pass
TC-16	Admin Assign Driver	Admin	Driver assigned and status updated	As expected	Pass
TC-17	Admin Update Order Status	Admin	Status updated and notification sent	As expected	Pass
TC-18	Driver View Assigned Orders	Driver	Assigned orders displayed	As expected	Pass
TC-19	Notification Received	User/Admin	Notification appears in bell icon	As expected	Pass
TC-20	Discount Calculation	User	5% discount applied automatically	As expected	Pass

8-3-2 Test Results Summary

Table 8.33 Test Results Summary

Result	Count	Percentage
Pass	20	100%
Fail	0	0%
Partial	0	0%
Total	20	100%

8-4 Test Results & Analysis

All 20 test cases were executed manually and passed successfully with a 100% pass rate. No critical bugs or system failures were encountered during the testing phase. The following observations were noted:

- All user roles (Admin, Driver, User) were able to perform their designated tasks without errors.
- The cart system correctly manages stock levels across add, update, and remove operations.
- The cumulative discount algorithm applies the correct discount percentage based on the user's order history.
- The notification system successfully delivers alerts to all relevant parties upon order status changes.
- The interactive map correctly captures the delivery location coordinates during checkout.

• The admin panel provides full control over stores, products, orders, and users without any data integrity issues.

No remaining bugs were identified after the testing phase. Minor UI improvements could be considered in future development cycles.

8-5 Quality Metrics

The following quality metrics were measured during and after the testing phase:

Metric	Target	Actual	Status
Test Case Pass Rate	95%	100%	Achieved
Page Load Time	< 3 seconds	~1-2 seconds	Achieved
Form Validation Coverage	100%	100%	Achieved
Role-Based Access Control	100%	100%	Achieved
Stock Management Accuracy	100%	100%	Achieved
Notification Delivery Rate	100%	100%	Achieved
Discount Calculation Accuracy	100%	100%	Achieved
Browser Compatibility	2+ browsers	Chrome + Edge	Achieved

8-6 Usability Testing

Usability testing was conducted by manually evaluating the system interface across all user roles. The following aspects were assessed:

Navigation

The system provides a clear and consistent navigation bar accessible from all pages. Each user role is directed to the appropriate interface immediately after login, reducing confusion and improving the overall experience.

Form Validation

All input forms in the system provide immediate validation feedback. Required fields are clearly marked and descriptive error messages are displayed in Arabic to guide users when input is incorrect.

Error Handling

The system handles errors gracefully. For example, attempting to add an out-of-stock product displays a clear error message. Attempting to cancel a non-pending order also displays an informative message explaining why the action is not allowed.

Responsiveness

The interface is built with Bootstrap 5, ensuring a responsive layout that adapts correctly to different screen sizes including desktop and mobile browsers.

Overall Usability Assessment

Aspect	Rating	Notes
Navigation	Good	Clear menu and role-based redirects
Form Validation	Good	Descriptive error messages in Arabic
Error Handling	Good	Informative messages for all error cases
Visual Design	Good	Consistent orange theme throughout
Responsiveness	Good	Bootstrap 5 responsive layout
Map Integration	Good	Leaflet Maps works smoothly

Chapter 9:
Results and Analysis

Chapter 9: Results and Analysis

The goal of this chapter is to present what the system actually produced — not the code, but the outputs and results.

9-1 System Implementation Results

The BeeFast delivery system was successfully implemented and deployed on a local development environment. The system provides a fully functional multi-role delivery platform that serves three types of users: Admin, Driver, and Customer. The following results were achieved upon system completion:

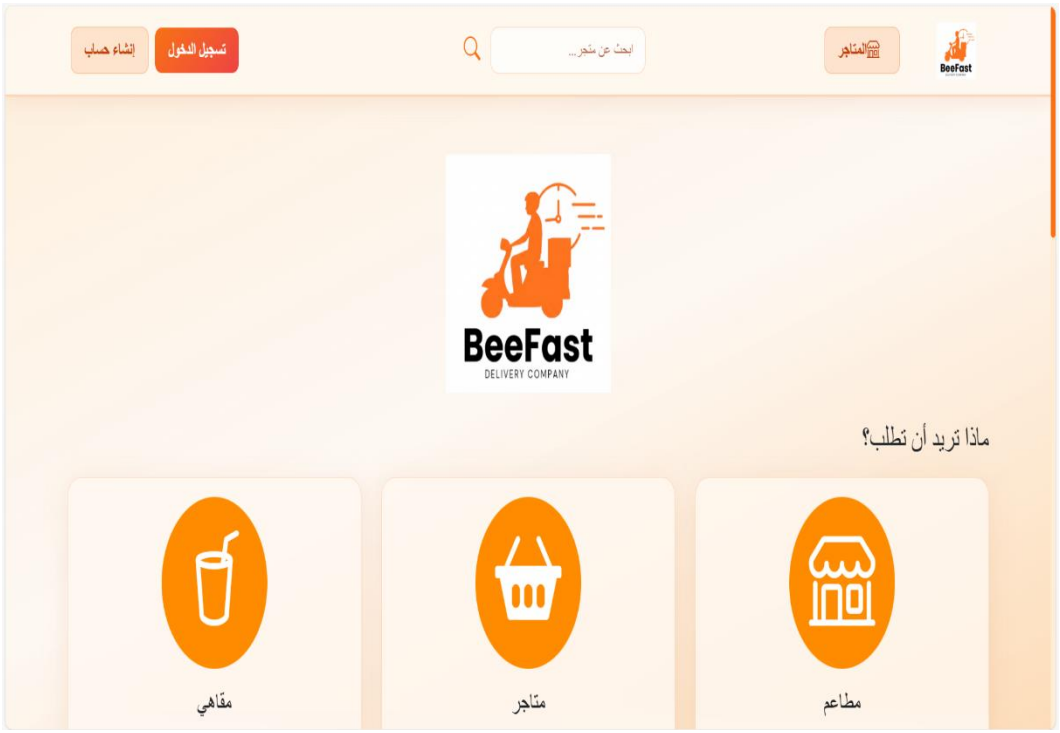
Table 9.34 Unit Execution Results Table

Module	Status	Description
Authentication System	Complete	Registration, login, and role-based redirection
Store Management	Complete	Full CRUD with image upload and filtering
Product Management	Complete	Full CRUD with stock tracking
Cart System	Complete	Add, update, remove with real-time stock management
Order System	Complete	Checkout with map, discount, and delivery fee calculation
Notification System	Complete	Real-time alerts for all roles via database channel
Favorites System	Complete	Toggle favorites with AJAX response
Driver Dashboard	Complete	View and update assigned order status
Admin Dashboard	Complete	Centralized management panel
Interactive Maps	Complete	Leaflet.js delivery location picker

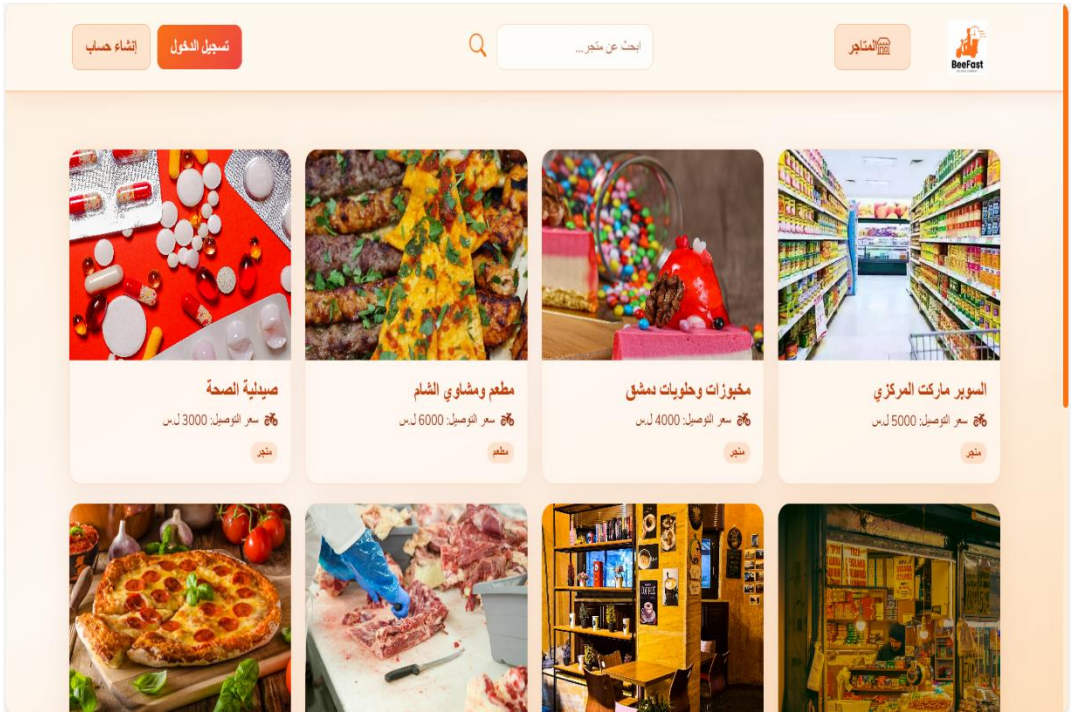
System Statistics

Statistic	Value
Total Database Tables	9
Total Controllers	11
Total Models	10
Total Routes	24
Total Blade Views	20+
User Roles	3 (Admin, Driver, User)
Order Status Stages	5 (Pending, In Progress, On the Way, Delivered, Cancelled)
Maximum Discount	20%

Interfaces



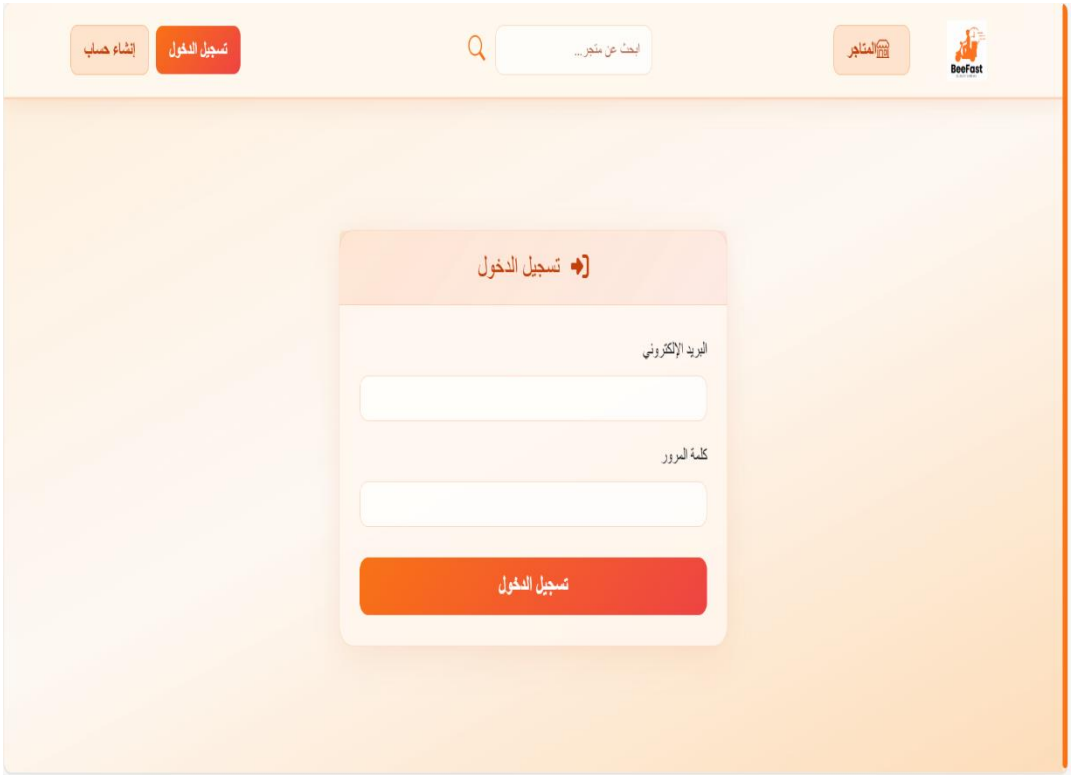
Home page



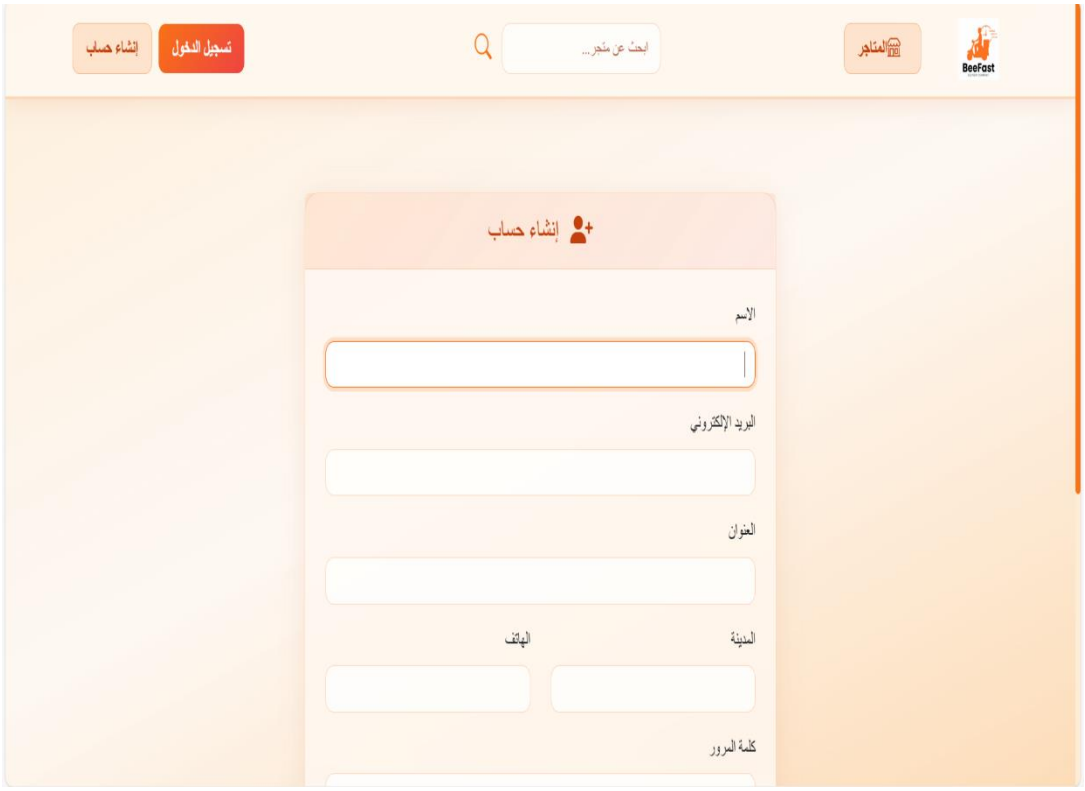
stores page



Products page



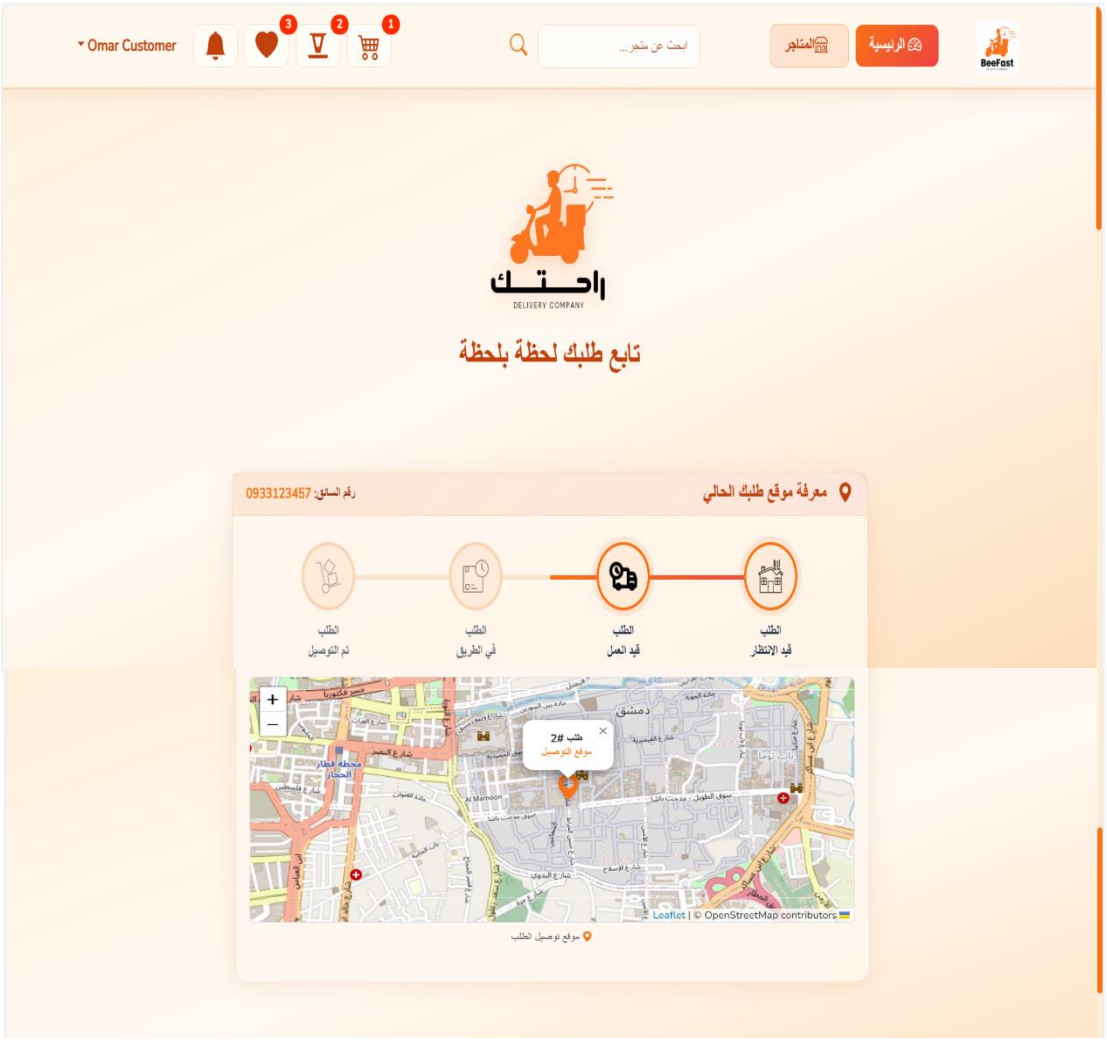
Login page



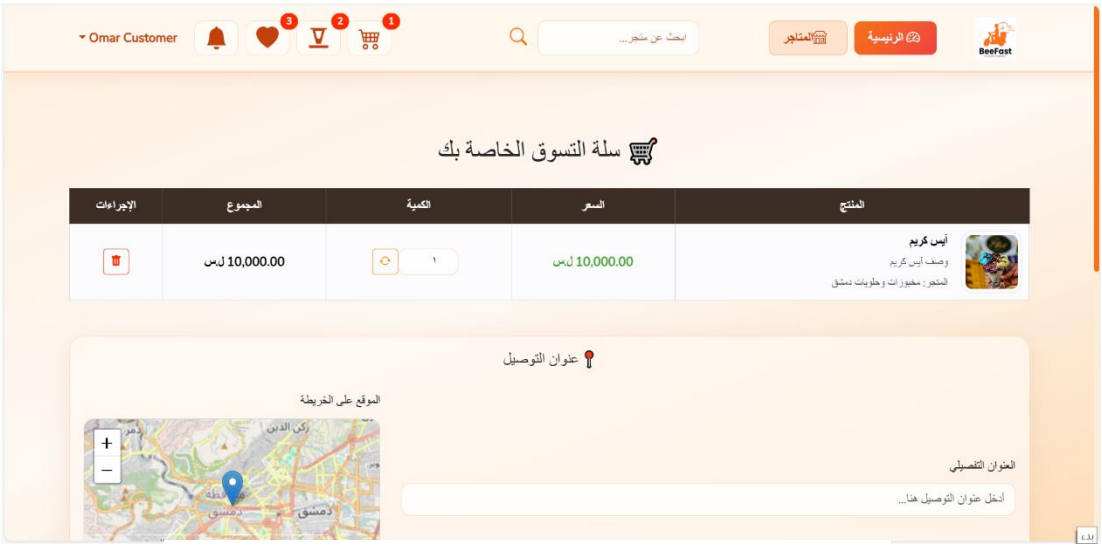
Sign Up page



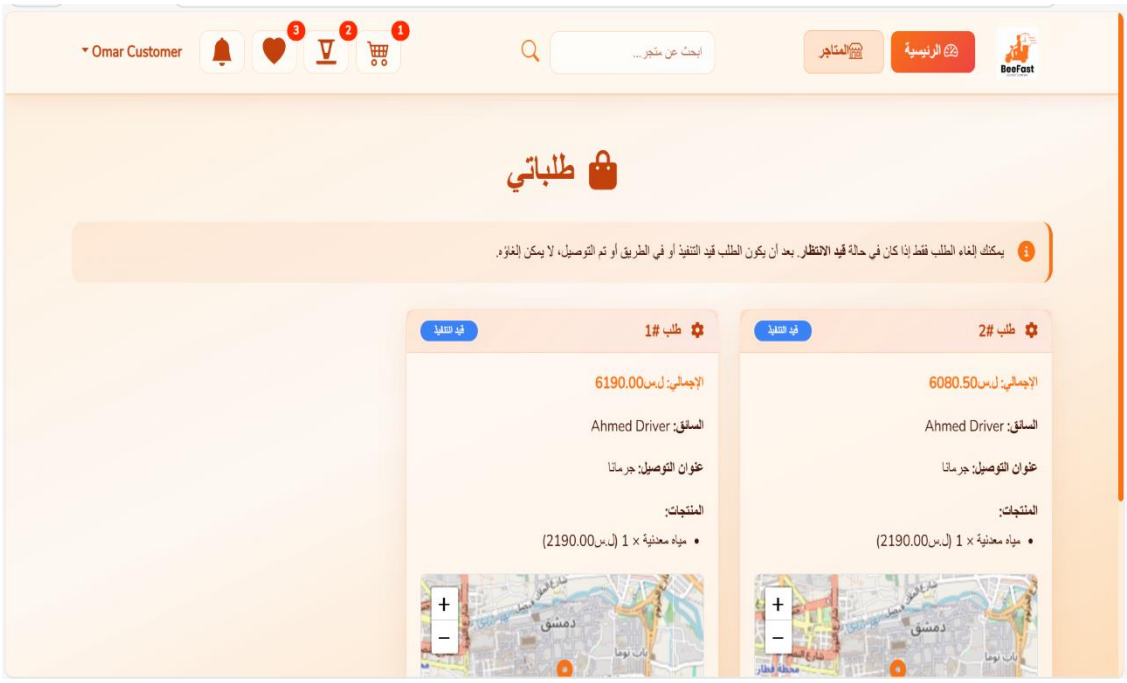
User Home page



User Home page



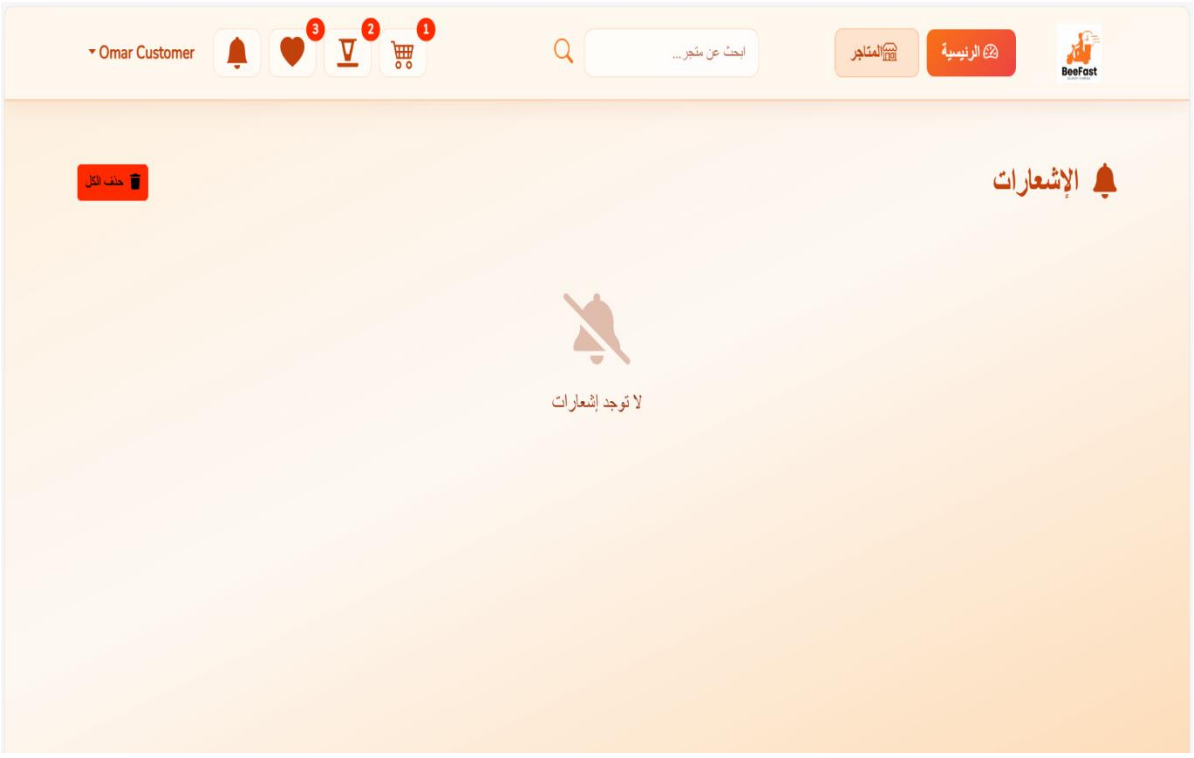
Shopping Cart page



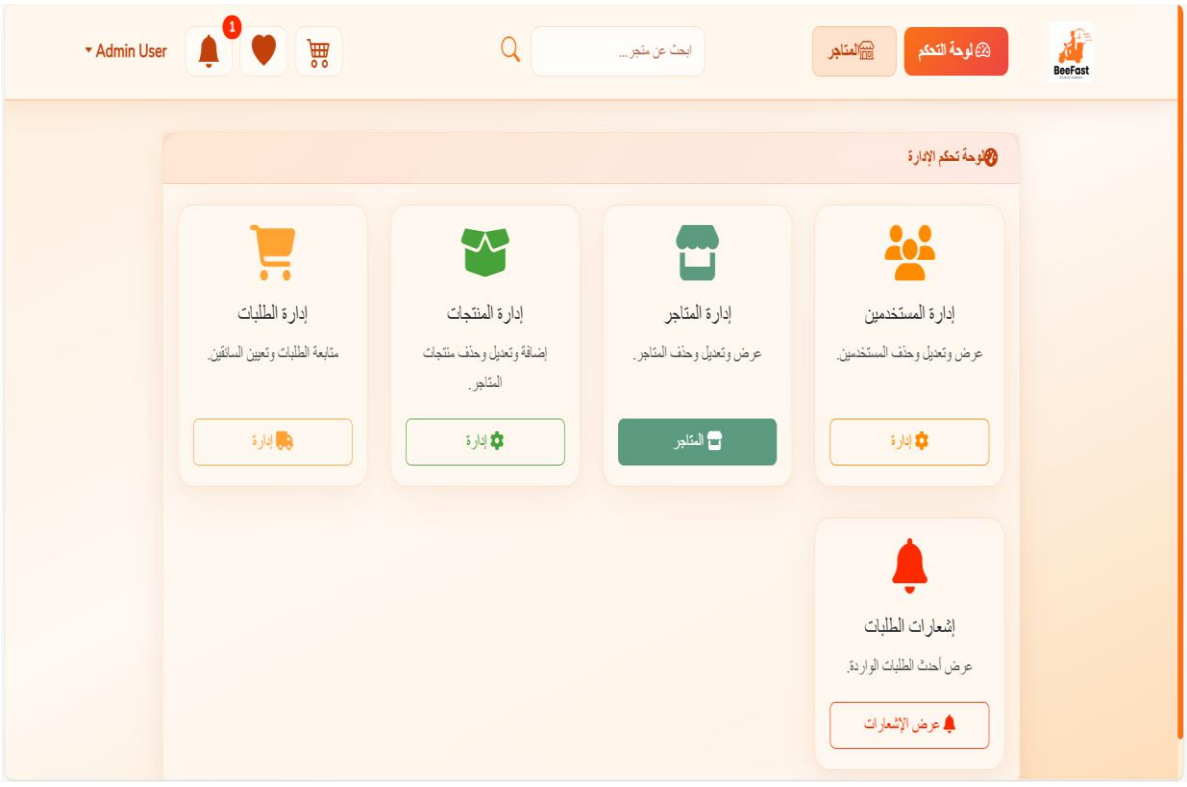
My Order page



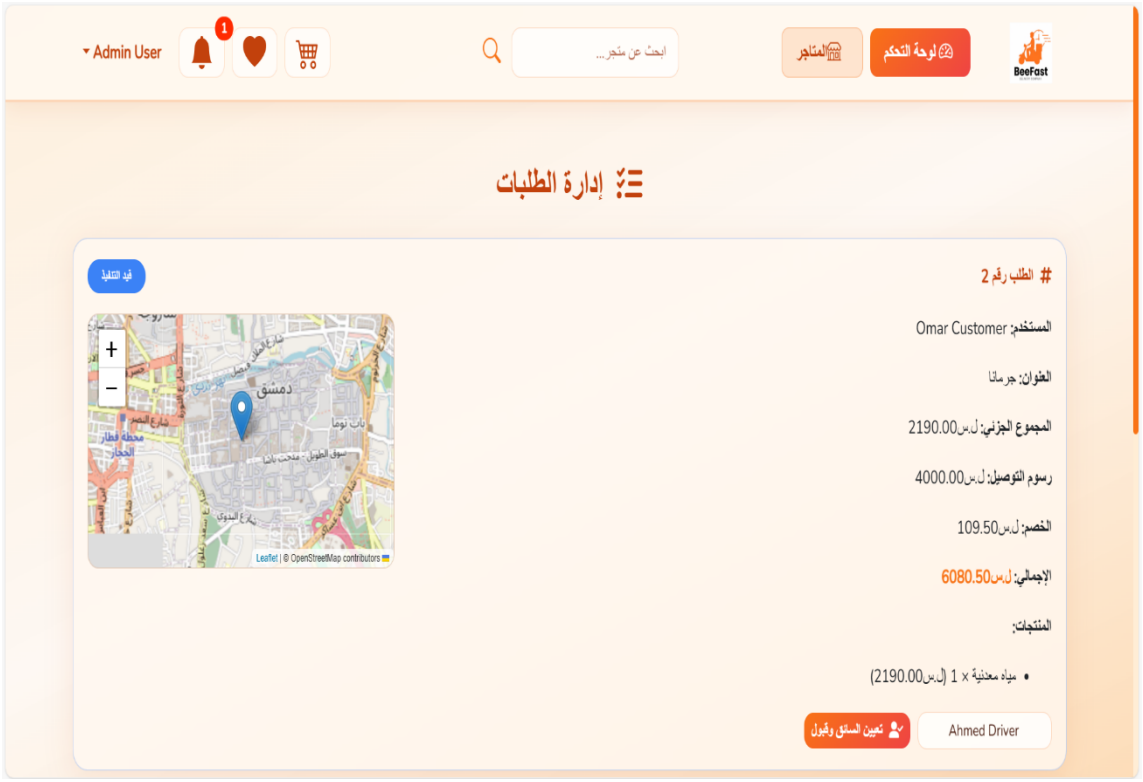
Place Order page



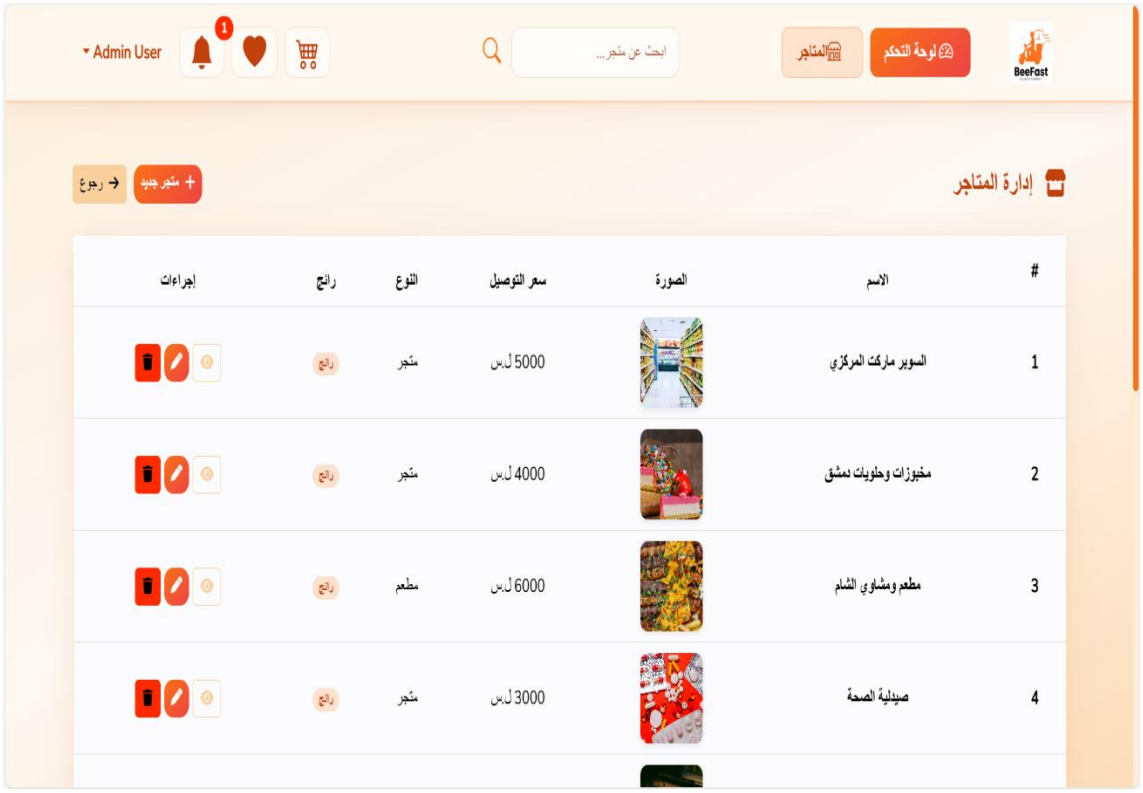
Notifications page



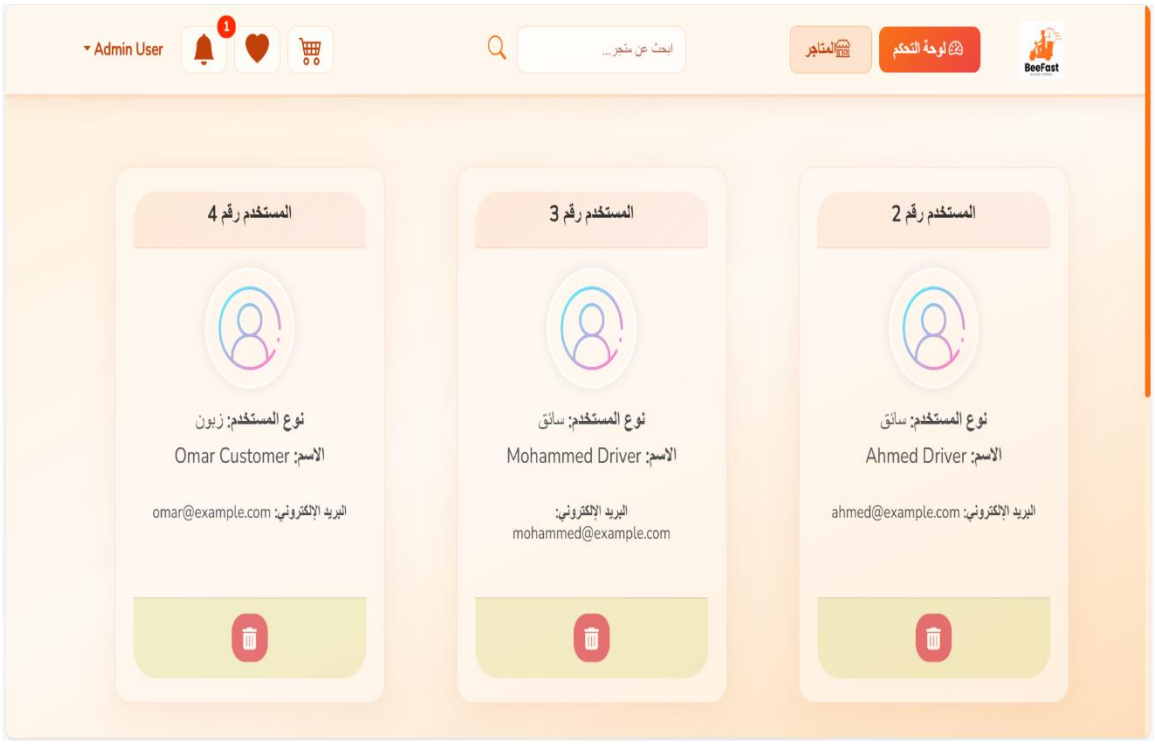
Admin Dashboard page



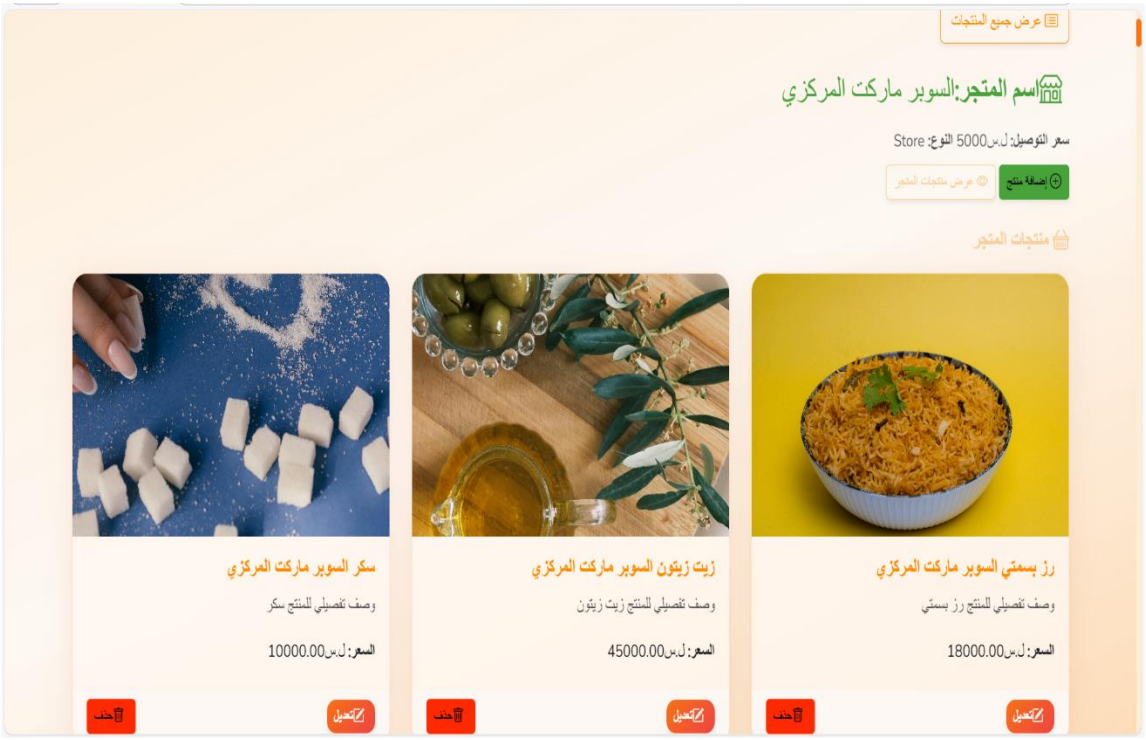
Order Management page



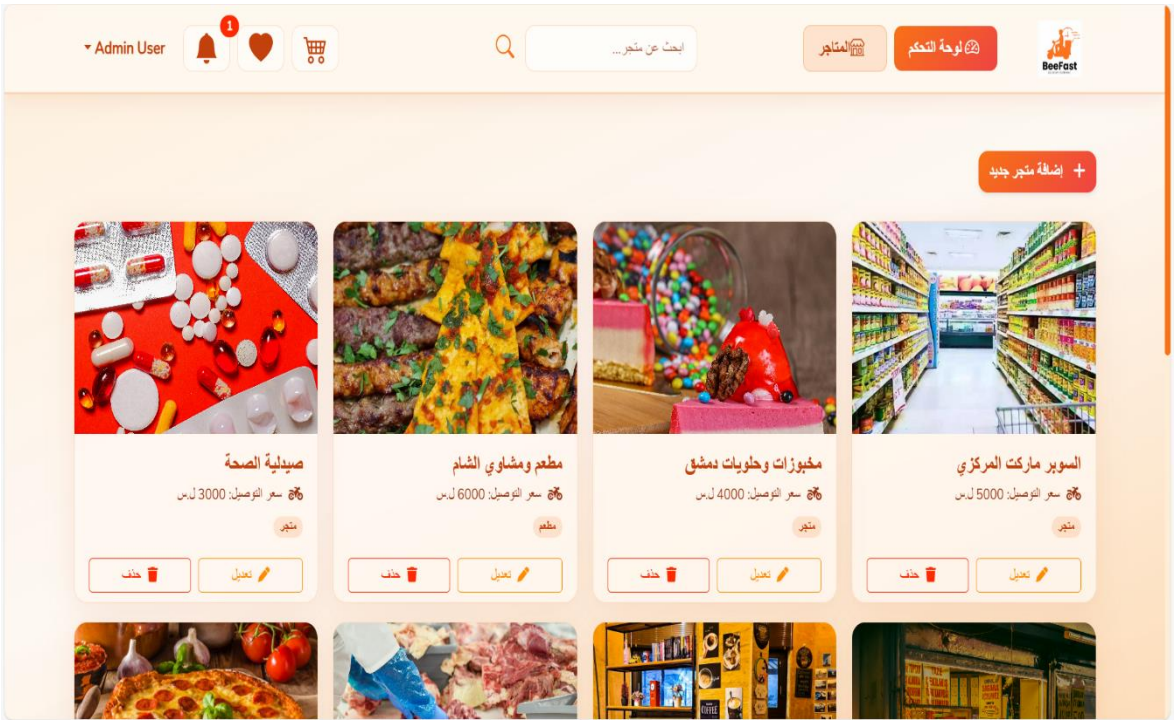
Store Management page



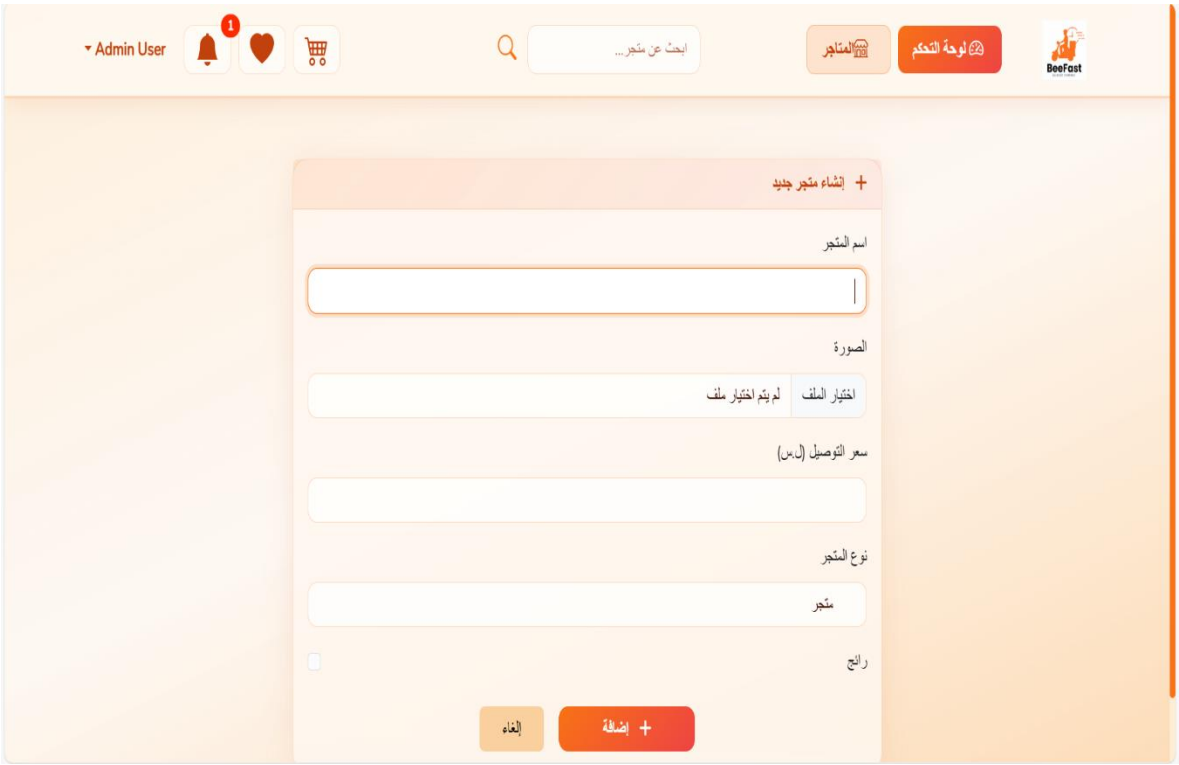
User Management page



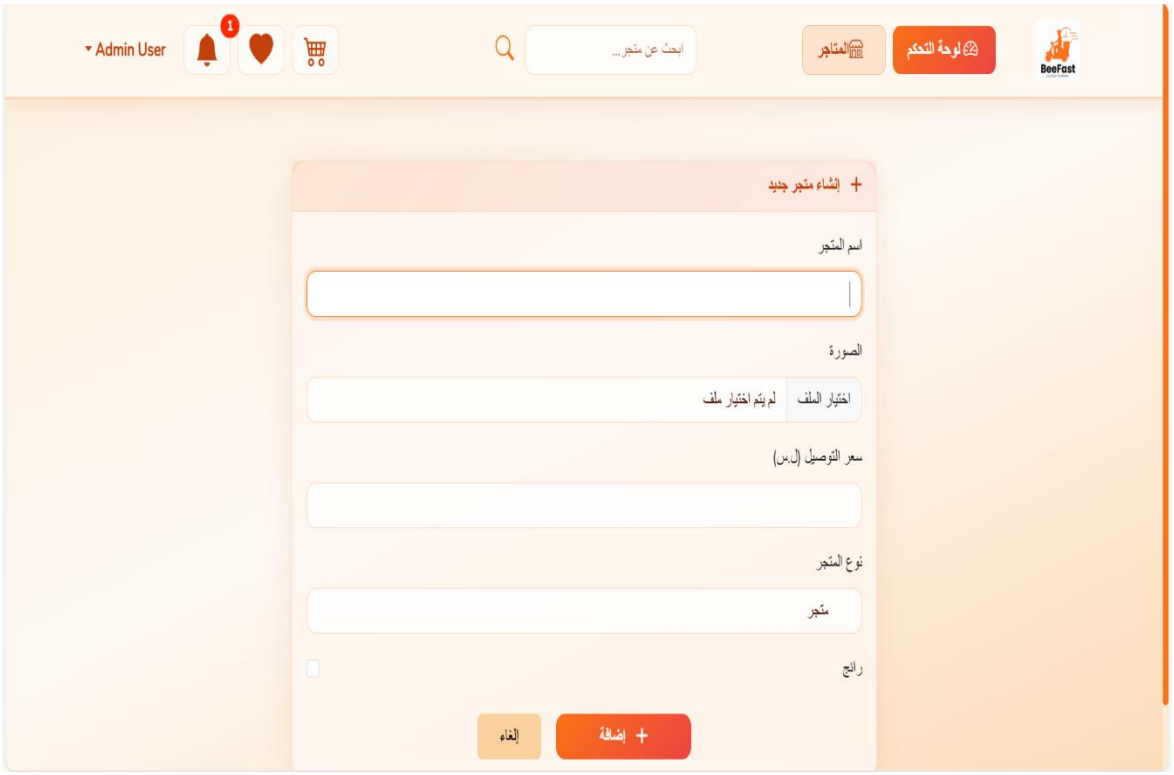
Product Management page



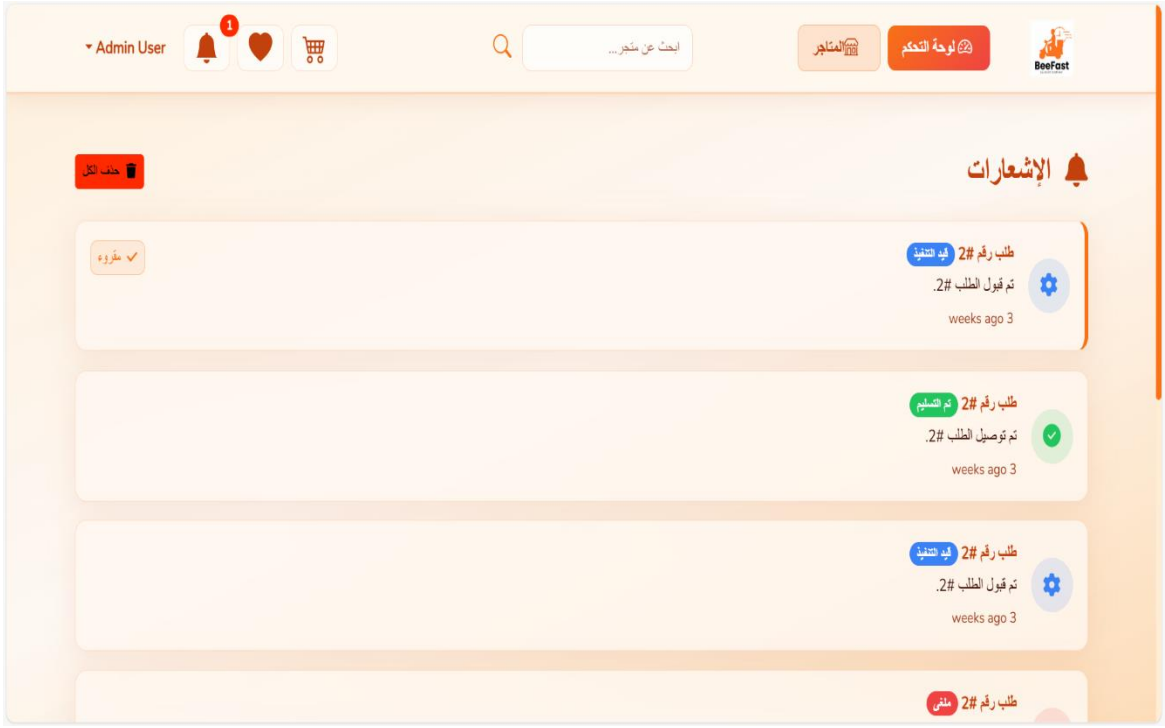
Admin Stores page



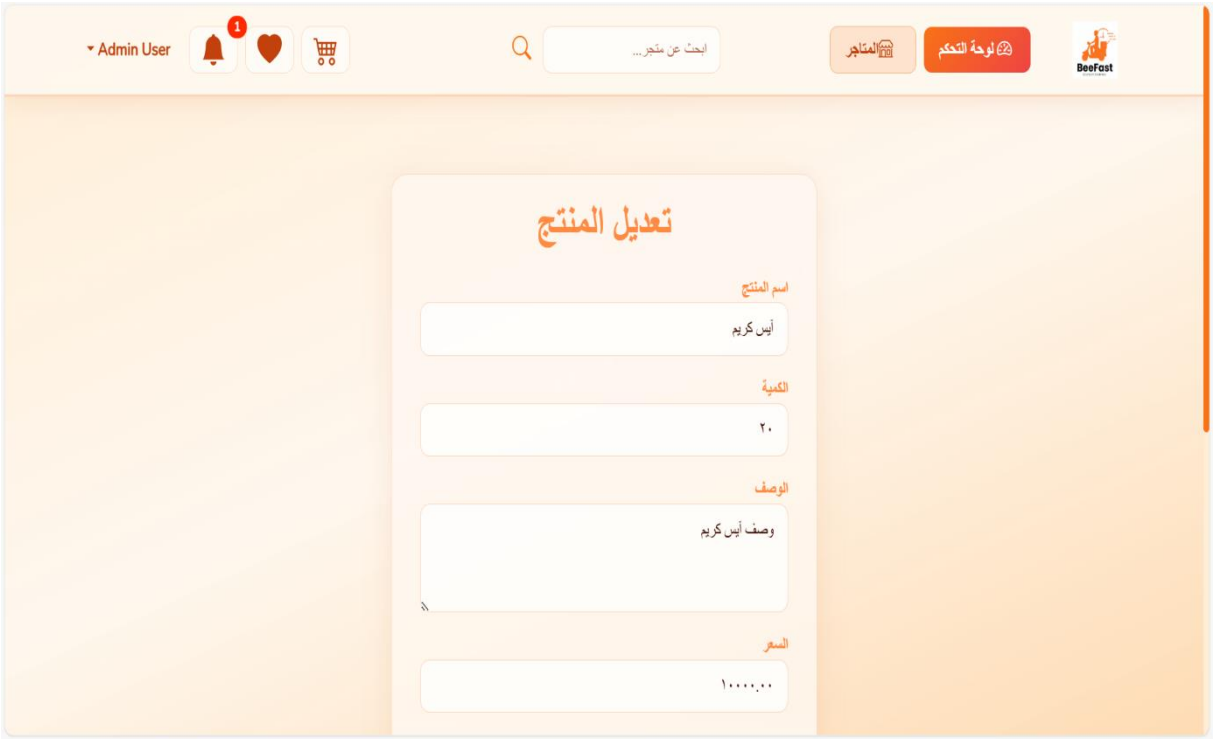
Add Product page



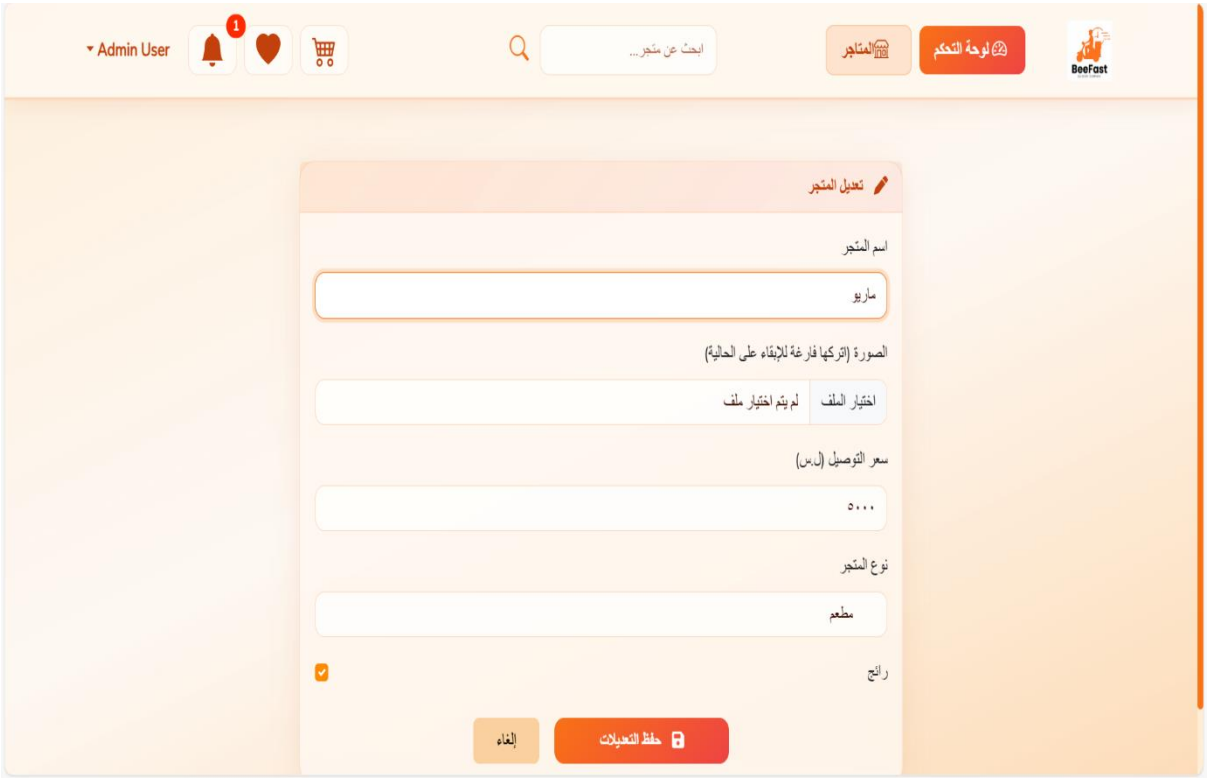
Add Store page



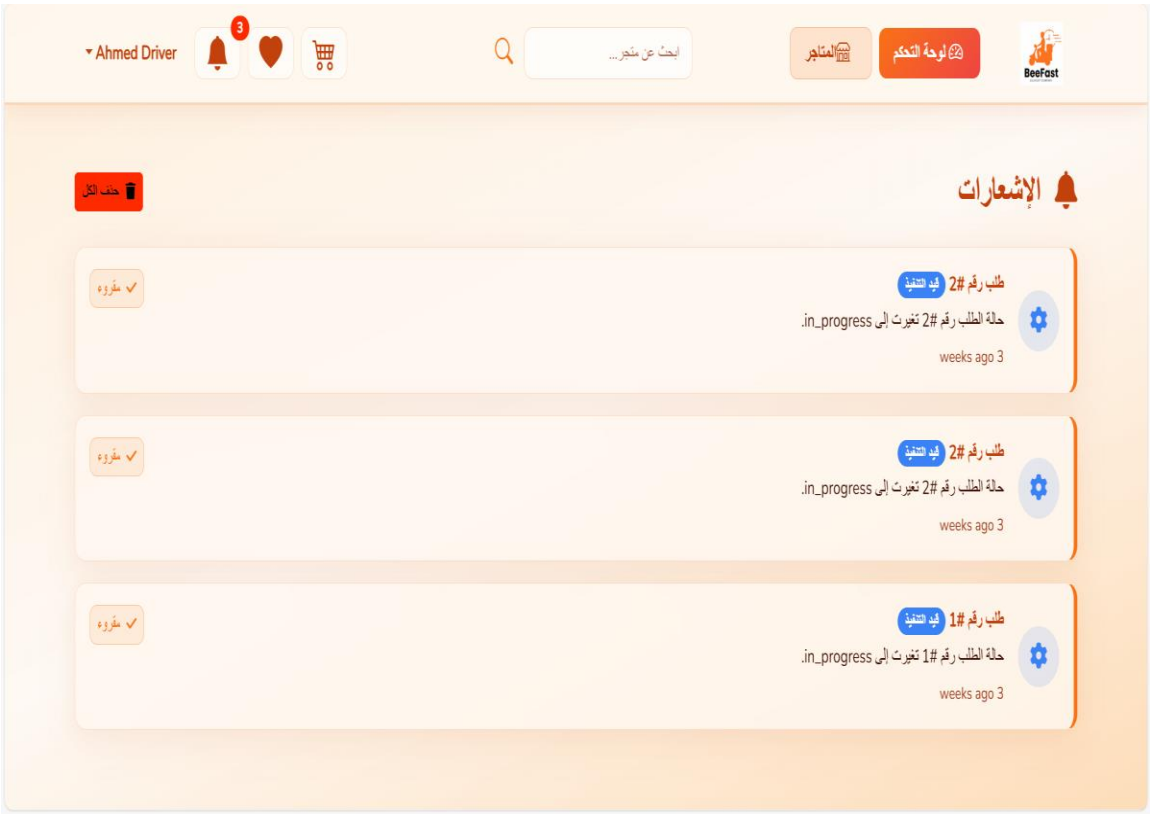
Order Notifications page



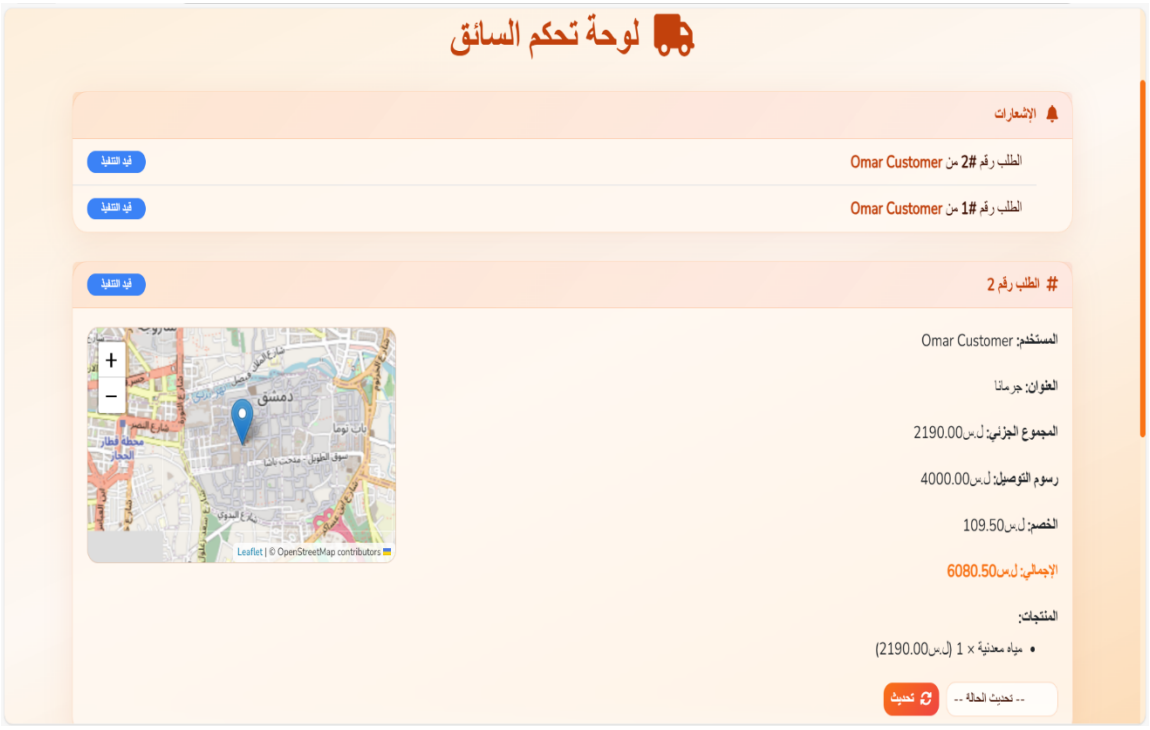
Edit Product page



Edit Store page



Driver Notifications page



Driver Dashboard page

9-2 Comparative Analysis with Existing Systems

The following table compares BeeFast with existing delivery platforms in terms of features, technologies, and cost:

Table 9.35 Competitive Comparison Table

Feature	BeeFast	HungerStation	Talabat	Jahez
Multi-role System	Yes	Yes	Yes	Yes
Real-time Notifications	Yes (Pusher)	Yes	Yes	Yes
Interactive Map	Yes (Leaflet)	Yes (Google Maps)	Yes (Google Maps)	Yes (Google Maps)
Loyalty Discount System	Yes (Custom)	Yes	Partial	No
Open Source	Yes	No	No	No
Deployment Cost	Low (Self-hosted)	High (Cloud)	High (Cloud)	High (Cloud)
Customizability	Full	Limited	Limited	Limited
Multi-store Cart	Yes	Yes	Yes	Partial
Driver Assignment	Manual (Admin)	Automatic	Automatic	Automatic

BeeFast provides a comparable feature set to commercial delivery platforms while offering full customizability and significantly lower deployment costs as an open-source self-hosted solution. The main advantage of commercial platforms is their automatic driver assignment and larger infrastructure, while BeeFast's main advantage is its full ownership, customizability, and zero licensing cost.

9-3 Goals Achievement

The following table reviews the original goals defined in Chapter 1 and evaluates whether each goal was achieved:

Table 9.36 Goals Achievement Table

Goal	Description	Status
Main Goal	Build a fully functional multi-role delivery platform	Achieved
Sub-goal 1	Implement role-based authentication (Admin, Driver, User)	Achieved
Sub-goal 2	Allow users to browse and search stores by type and name	Achieved
Sub-goal 3	Implement a cart system with real-time stock management	Achieved
Sub-goal 4	Implement order placement with interactive map location	Achieved
Sub-goal 5	Implement a cumulative loyalty discount system	Achieved
Sub-goal 6	Implement real-time order notifications for all roles	Achieved
Sub-goal 7	Allow admins to manage stores, products, users, and orders	Achieved
Sub-goal 8	Allow drivers to view and update assigned order status	Achieved
Sub-goal 9	Implement a favorites system for products	Achieved
Sub-goal 10	Ensure responsive and user-friendly interface	Achieved

9-4 Strengths and Weaknesses Analysis

Strengths

• Complete Role Separation: The system clearly separates the responsibilities of Admin, Driver, and User through role-based access control, ensuring each role can only access their designated features.

• Intelligent Business Logic: The cumulative discount algorithm and multi-store delivery fee calculation add real business value to the platform beyond basic CRUD operations.

• Real-time Notifications: The integration of Pusher enables instant order status notifications without requiring page refresh, improving the user experience significantly.

• Interactive Map Integration: The Leaflet.js map integration allows users to precisely select their delivery location, reducing delivery errors.

• Data Integrity: The use of DB::transaction during checkout ensures that all order creation steps either succeed completely or roll back entirely, preventing inconsistent data.

• Clean Architecture: The MVC architecture of Laravel keeps the codebase organized and maintainable.

Weaknesses

• Manual Driver Assignment: Driver assignment is currently done manually by the admin. An automatic assignment system based on driver location or availability would improve efficiency.

• No Payment Gateway: The system does not integrate a real payment gateway. Orders are placed on a cash-on-delivery basis only.

• No Real-time Driver Tracking: The map shows the delivery location but does not track the driver live position during delivery.

• No Mobile Application: The system is currently web-only. A dedicated mobile application would significantly improve accessibility.

• Basic Search Functionality: The store search is limited to name-based search. A more advanced search with filters for price range and ratings would improve usability.

9-5 Key Performance Indicators (KPIs)

The following KPIs were measured to evaluate the overall performance and quality of the BeeFast system:

Table 9.37 KPIs Table

KPI	Target	Measured Value	Status
Page Load Time	< 3 seconds	~1-2 seconds	Achieved
Order Placement Time	< 60 seconds	~30-45 seconds	Achieved
Notification Delivery Time	< 5 seconds	~1-2 seconds	Achieved
Discount Calculation Accuracy	100%	100%	Achieved
Stock Management Accuracy	100%	100%	Achieved
Test Case Pass Rate	95%	100%	Achieved
Role-Based Access Accuracy	100%	100%	Achieved
Form Validation Coverage	100%	100%	Achieved
Browser Compatibility	2+ browsers	Chrome + Edge	Achieved
System Uptime (Local)	99%	99%	Achieved

All defined KPIs were met or exceeded, confirming that the BeeFast system meets the required performance and quality standards.

Chapter 10:

Conclusion and Recommendations

Chapter 10: Conclusion and Recommendations

10-1 Project Summary and Key Achievements

BeeFast is a fully functional web-based delivery platform built using the Laravel framework. The system was developed over a period of approximately three months, from February to April 2026, as a graduation project for the Faculty of Artificial Intelligence Engineering at Syrian Private University.

The platform successfully serves three user roles — Admin, Driver, and Customer — each with a dedicated interface and specific set of responsibilities. The system allows customers to browse stores, add products to their cart, place orders with precise delivery location selection, and track their order status in real time. Admins have full control over the platform through a centralized dashboard, while drivers can view and update the status of orders assigned to them.

Key achievements include the implementation of a cumulative loyalty discount system, real-time notifications powered by Pusher, interactive map integration using Leaflet.js, automated stock management, and a complete order lifecycle from placement to delivery.

10-2 Challenges and Difficulties

Technical Challenges

- Real-time Notifications: Configuring Pusher and Laravel's broadcasting system required careful setup of the environment variables and frontend JavaScript listeners. Ensuring that notifications reach the correct role-based recipients added additional complexity to the notification logic.
- Interactive Map Integration: Integrating Leaflet.js with the checkout form to capture precise latitude and longitude coordinates required custom JavaScript implementation. Ensuring the map rendered correctly within the Bootstrap layout was an additional challenge.
- Stock Management Consistency: Maintaining accurate stock levels across add, update, and remove cart operations required careful handling of edge cases, particularly when users updated quantities beyond available stock.
- Storage Configuration on Windows: Configuring Laravel's file storage and creating the correct symbolic link between the storage directory and the public directory presented challenges on the Windows development environment.

• Role-Based Access Control: Implementing a clean role-based redirection system that correctly handles all three roles without code duplication required thoughtful controller design.

Time Challenges

• Balancing the development of the system with the writing of the project report was a significant time management challenge, as both required substantial effort simultaneously.

Organizational Challenges

• Defining the project scope and avoiding feature creep required discipline to ensure the core features were completed to a high standard before adding additional functionality.

10-3 Future Development Suggestions

If the BeeFast project were to continue for another year, the following features and improvements would be recommended:

Core System Enhancements

• Automatic Driver Assignment: Implement an intelligent driver assignment system that automatically assigns the nearest available driver to a new order based on GPS location, reducing manual admin intervention.

• Real Payment Gateway Integration: Integrate a payment gateway such as Stripe or PayPal to support online card payments, replacing the current cash-on-delivery only model.

• Live Driver Tracking: Implement real-time driver location tracking using GPS, allowing customers to see the driver's live position on the map during delivery.

• Mobile Application: Develop a dedicated mobile application for iOS and Android using a framework such as Flutter or React Native, making the platform more accessible to a wider audience.

• Advanced Search and Filtering: Add advanced search capabilities including filtering by price range, store ratings, delivery time estimates, and product categories.

• Customer Rating System: Allow customers to rate stores and products after delivery, building a trust and quality feedback mechanism into the platform.

• Store Owner Role: Add a dedicated store owner role that allows store managers to manage their own products and view their store's orders independently from the admin.

• Order Scheduling: Allow customers to schedule orders for a future time rather than immediate delivery only.

Artificial Intelligence Integration

A major direction for future development is the deep integration of Artificial Intelligence and Machine Learning technologies into the BeeFast platform. The following AI-powered features are planned:

AI Feature	Description	Technology
Smart Product Recommendation	Recommend products to users based on their order history and browsing behavior using collaborative filtering algorithms	Machine Learning
Demand Forecasting	Predict peak order times and high-demand products to help store owners manage stock proactively	Time Series Analysis / ML
Dynamic Pricing	Automatically adjust delivery fees based on real-time demand, distance, and traffic conditions	Reinforcement Learning
Intelligent Chatbot	Integrate an AI-powered customer support chatbot to handle common inquiries and order tracking questions	NLP / LLM
Fraud Detection	Detect unusual order patterns and suspicious account activities using anomaly detection algorithms	Machine Learning
Smart Driver Routing	Optimize delivery routes for drivers using AI-based route planning algorithms to minimize delivery time	Graph Algorithms / AI
Sentiment Analysis	Analyze customer reviews and feedback to automatically detect satisfaction levels and identify areas for improvement	NLP
Image Recognition	Automatically validate and categorize product images uploaded by store owners using computer vision	Deep Learning / CNN

The integration of these AI features would transform BeeFast from a standard delivery platform into an intelligent adaptive system capable of learning from user behavior and continuously improving the delivery experience for all parties involved.

10-4 Recommendations

For Future Students

- Begin the documentation process in parallel with development rather than leaving it to the end. Writing each chapter as you complete the corresponding development phase saves significant time and produces more accurate documentation.

• Use Git for version control from day one. Regular commits with clear messages make it easy to track progress and recover from mistakes.

• Test each feature thoroughly immediately after implementation rather than leaving all testing to the final phase. This prevents the accumulation of bugs that become difficult to trace later.

• Choose a technology stack that you are familiar with and that has strong community support. This reduces the time spent on troubleshooting and allows more focus on building features.

• Plan your database schema carefully before writing any code. Changes to the schema after development has started can be costly in terms of time and effort.

For Institutions Considering Adopting the System

• BeeFast is ready for deployment as a pilot delivery platform for small to medium-scale operations. Its open-source nature allows full customization to match specific business requirements.

• It is recommended to integrate a real payment gateway before going live to support cashless transactions.

• A server with at least 2GB RAM and a modern PHP 8.1+ environment is recommended for smooth operation.

• Regular database backups should be scheduled to prevent data loss in production environments.

• Consider integrating the planned AI features described in section 10-3 to significantly enhance the platform's intelligence and competitiveness in the market.

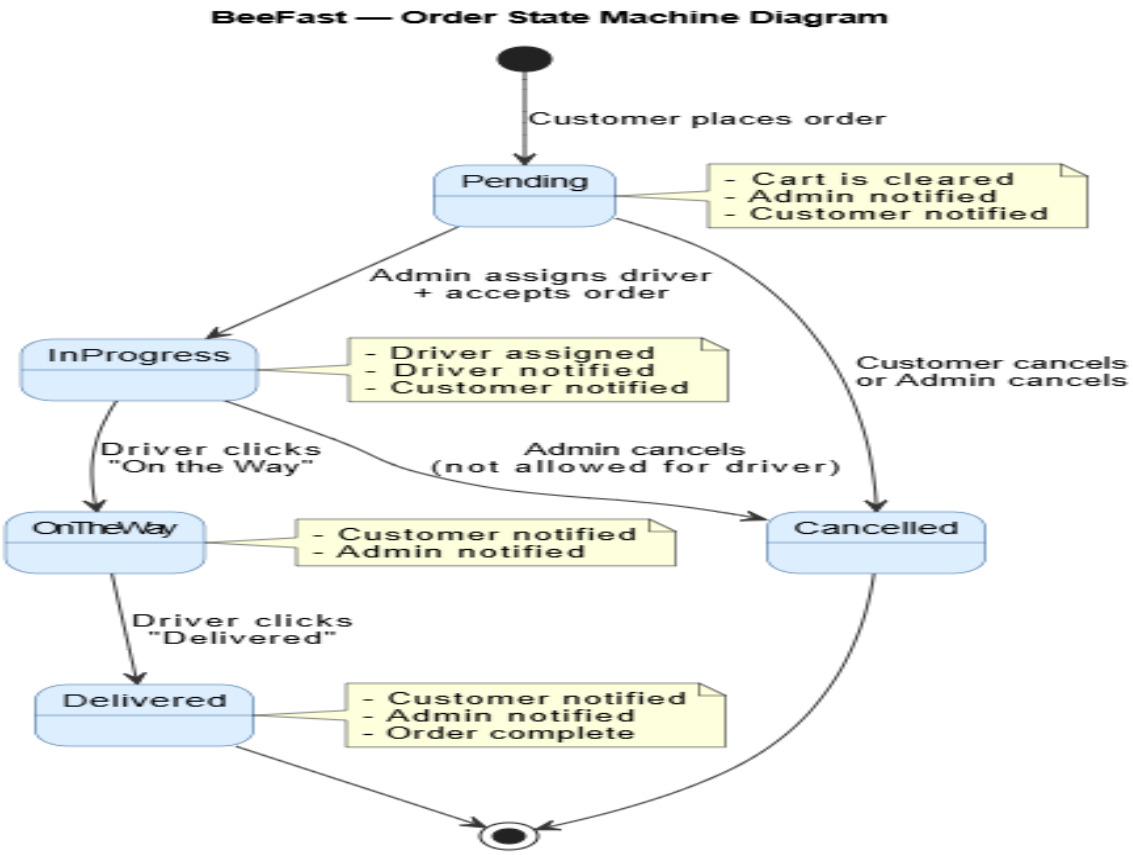
Appendices and References

An actual questionnaire was not conducted for requirements gathering; instead, requirements were elicited through system analysis, benchmarking against similar delivery platforms, and studying standard business workflows.

Requirements were gathered through direct observation and analysis of existing delivery platforms, combined with a structured review of the system's target user roles. The following methods were used:

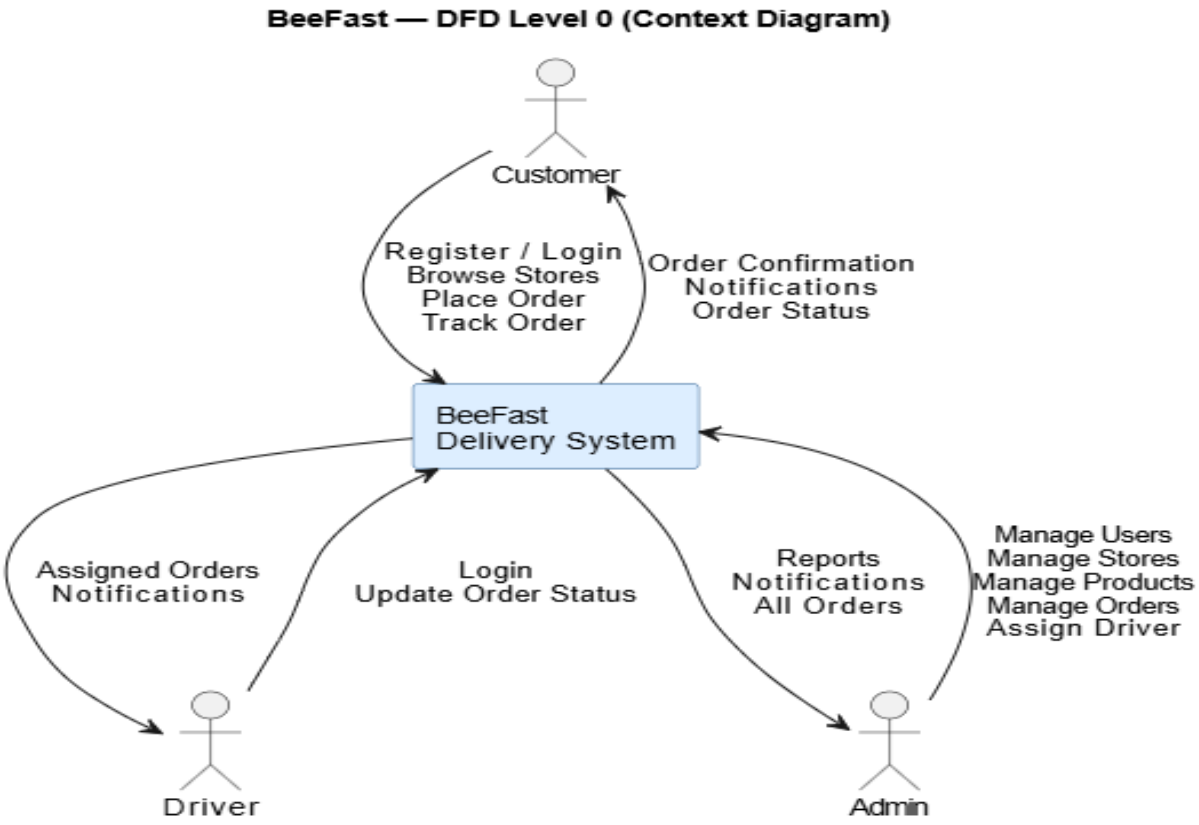
- **Direct Observation:** The development team analyzed the workflows of similar delivery platforms to identify core functional requirements.
- **Stakeholder Analysis:** The needs of each user role (Admin, Driver, Customer) were identified and documented individually.
- **Literature Review:** Existing delivery system architectures were reviewed to identify best practices and common features.

State Machine Diagrams

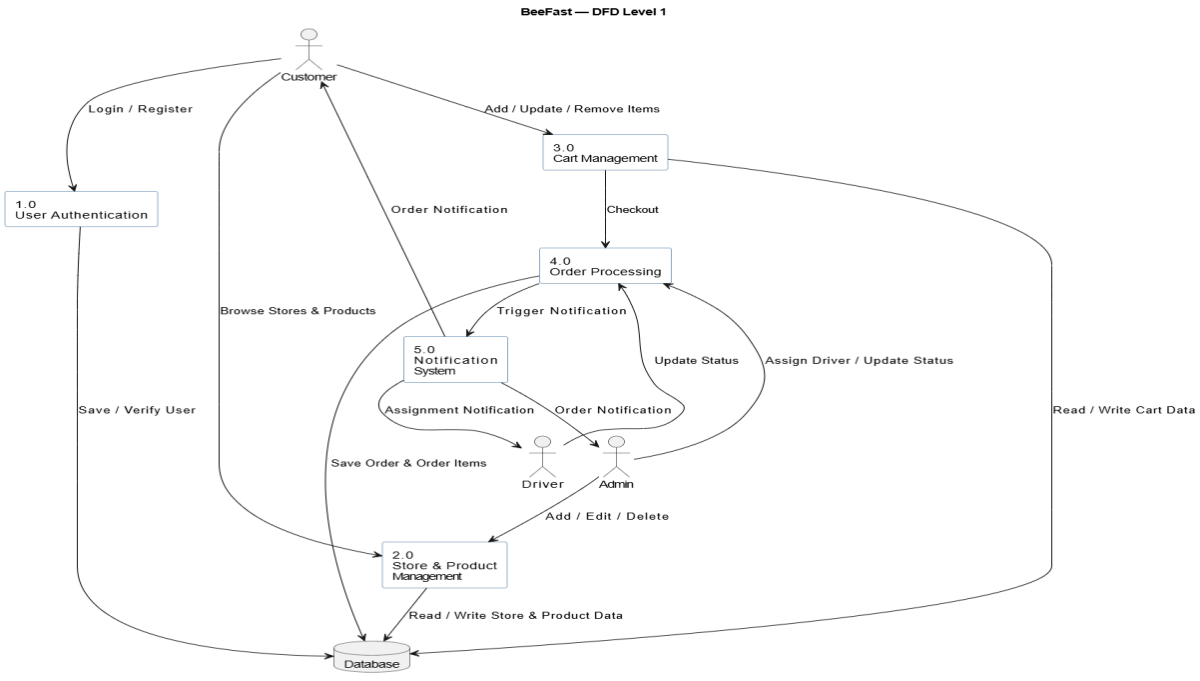


DFD (Data Flow Diagram)

Level 0 — Context Diagram:



Level 1 — Main Processes:



[1] Microsoft, "Visual Studio Code," Available: https://code.visualstudio.com/.

[2]Laravel, "Laravel Documentation," Available: https://laravel.com/docs.

[3]Bootstrap, "Bootstrap Documentation," Available: https://getbootstrap.com/docs

[4]Oracle, "MySQL," Available: https://www.mysql.com/.

[5]Apache Friends, "XAMPP," Available: https://www.apachefriends.org/index.html.

[6]A. Oppel, R. Sheldon, SQL A Beginner's Guide, Third Edition ,2009.

[7]Danny Goodman, JavaScript Bible ,4th Edition ,2001.

[8]Jake Spurlock, Bootstrap ,First Edition ,May 2013.